Oscar Turnbull

Centre number: 56120

Ai racer

Contents

Analysis:

Introduction

My project idea is to create and train a neural network using reinforced learning to be able to play a simple top-down 2D racing game and to achieve faster times as it is trained. My aim is to produce an AI which can make autonomous decisions while driving on a track of when to turn, accelerate, brake, etc to reach the finish line. I want to do this project specifically as I have been interested in reinforced learning as a method of machine learning for a long time and will gain a much deeper understanding of the process behind it than I already have by implementing reinforced learning myself inside of a game. To make the project outstanding, I could also implement additional features such as the ability for the user to race against the AI to make the game more engaging and fun to strengthen my users' interest in reinforced learning.

I have been inspired by watching videos and reading articles previously about how reinforced learning can be used especially to play games. Reinforced learning is suitable for this as this branch of machine learning can take positive or negative signals from the game based on how well it has performed and can use that to fine tune the neural network and perform better in the future.

To complete this project, I must research and learn the mathematics behind machine learning and how neural networks can be implemented in python by using various online and offline materials.

Similar ideas I have researched:

A similar project which uses reinforcement learning inside of a game is Andrej Karpathy's pong from 2016.

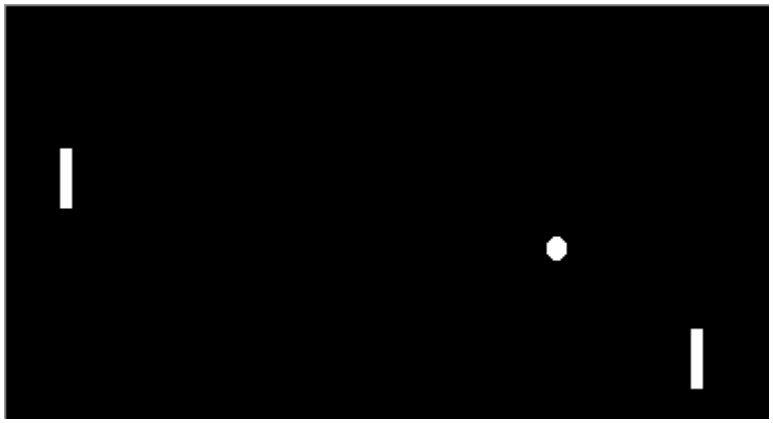


Figure 1 Karpathy's Pong from Pixels (https://karpathy.github.io/2016/05/31/rl/)

Here, Karpathy used reinforcement learning to give an array of every pixel in the game pong to a neural network. Then that neural network uses the input to decide if the paddle should move up or down to increase its chances of winning. This program allows the user to train the neural network from scratch and slowly improving by reinforcement learning and allows the user to watch the AI play a game against a hard coded pong bot.

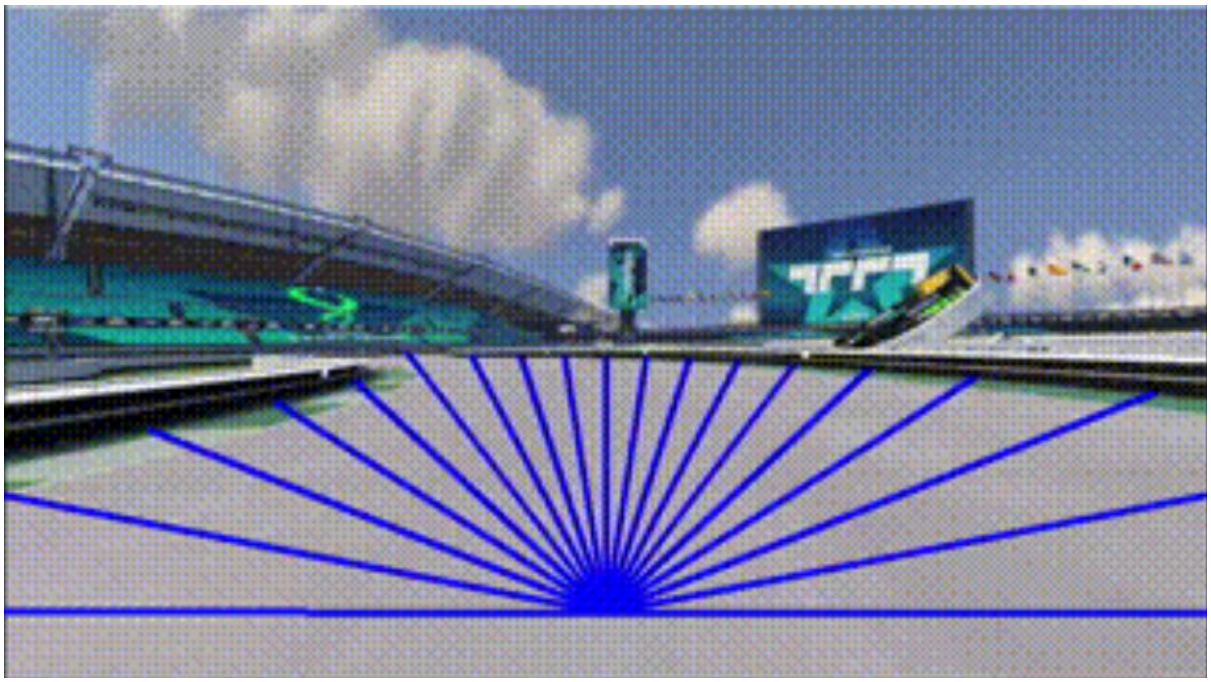
Positives	Negatives
The use of reinforcement learning to play a game is a very similar idea to what I want to implement.	The user interface is very bare and allows for very little interaction from the user.
It is very clear what is happening on screen due to its simple nature.	The colour scheme and design of the graphics is very simplistic which makes watching the AI less engaging.
I want to use and train a neural network to play a game and to update its internal values to become more efficient.	The game of Pong used is very simple with the only option for the neural network is to move up or down which is less enjoyable for the user.
Here, machine learning is used to optimally play a video game which is what I want to implement.	Karpathy chose to give the neural network the state of every pixel on screen each frame however this would result in the scope of the neural network to become too large for this project so I will reduce the number of inputs received to specific values in the game.
The AI is very strong at pong and can consistently beat a very good hard coded bot with enough training.	

Features I want to incorporate:

I want to use reinforcement learning to train a neural network instead of other methods of machine learning.

I want to use reinforcement learning to train a neural network to play a game specifically as this is an easy to visualise concept for the users.

Trackmania is a 3D racing game with an emphasis on user generated content such as custom tracks and cars. TMRL (Trackmania reinforcement learning) is a project created using the game where a neural network is trained using reinforcement learning to drive the car in a 3D environment. The neural network takes in various input parameters such as the speed of the car, the distances from the sides of the track and the image on the screen and uses it to calculate optimal values for the car's steering, acceleration and braking for its next frame.



Example of some information gathered by the AI about the game environment (trackmania-rl/tmrl: Reinforcement Learning for real-time applications - host of the TrackMania Roborace League)

Positives	Negatives
The use of reinforced learning inside of a racing game is especially like what I want to create and is an engaging concept.	The 3D racing game with advanced physics is far too complex to be used for this project.
The pretrained AI is very strong at playing the game and can beat many human players.	This AI uses many input parameters such as an image of the screen which is too complex to use for this project.
The graphics and user interface of trackmania are realistic and engaging and I would also like to use realistic colours	The AI itself comes with no user interface and requires editing of files by the user to configure which makes the program more difficult to use.

for my game to convey the environment more clearly to the user.	
TMRL comes with a pretrained AI so that users can experiment with it without requiring extensive testing as well as the ability to create one from scratch.	
The AI can be used effectively on a large variety of tracks.	

Features I want to incorporate:

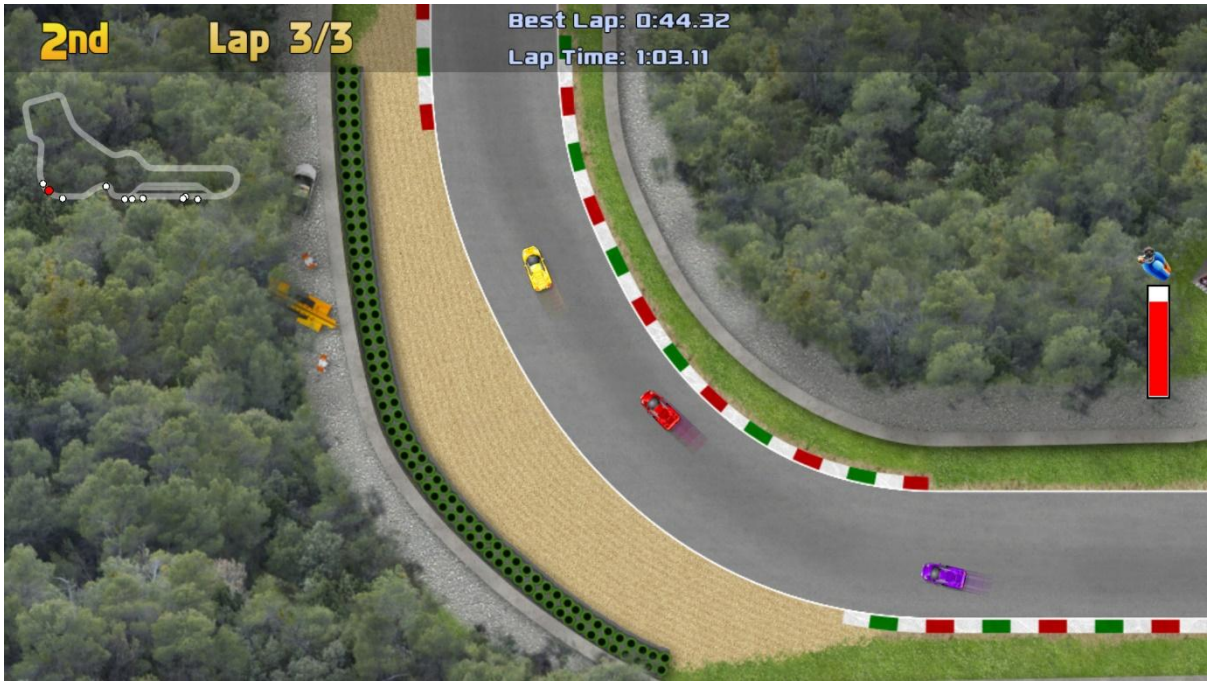
I want to use reinforcement learning inside of a racing game as this is both is an engaging concept and reinforcement learning is also suited to be used for video games.

I want to use a similar colour scheme with realistic colours to convey the environment so that it is easy to understand for the user and that the cars are visually distinct.

I want my project to contain a pretrained AI which can be immediately used by the user so that they do not have to sit through the long training process to use the AI.

I want my neural network to be able to drive correctly on tracks with different layouts so that it can learn the core aspects of the racing game and not just how to beat one specific track.

Ultimate Racing 2D is a top-down racing game featuring many unique tracks and realistic physics. This game does not feature any machine learning however the 2D racing game aspects are very similar to what I want to use in my own project as the game environment that the AI interacts with.



Screenshot of the gameplay from Ultimate Racing 2D (Save 70% on Ultimate Racing 2D on Steam)

Positives	Negatives
The game is a 2D racing game which is relatively easy to create while still allowing for some depth and enjoyment.	This game does not feature any machine learning which is a key feature I want to use in my project.
The car's physics are realistic which allows for a more enjoyable experienced from the user.	This game features different types of cars with different attributes such as speed which is not suitable for my project as each race should be similar so that the AI itself can be compared.
The cars have many mechanics such as drifting which adds a good level of complexity to the game.	The game's tracks are very long which would increase the time it would take to train an AI to play it.
The game's user interface is very simple but conveys all relevant information in the game.	
The game has many unique tracks and cars so that it is replayable.	
The game has simplistic but realistic graphics and art which I would be able to achieve something similar while still being easy to understand.	

Features I want to incorporate:

I want to create a similar 2D racing game for the AI to be able to play.

I want to add similar car physics and game mechanics so that there is more depth to the gameplay making it harder for the AI to perfect.

I want to use a similar art style and user interface so that all the information on screen is very clear to the user so that they can focus on the machine learning itself.

I want to have multiple different tracks in the game so that the AI can quickly adapt to new tracks created and is sufficient at using the core game mechanics instead of the series of inputs required to beat one specific level.

Stakeholders:

The stakeholders of my project are students who are interested in machine learning and artificial intelligence but who may not have personal experience with working with it. My target demographic is students aged between 14-18 as they are likely to have an initial curiosity about how machine learning can be used but not a full idea of how it works and may have not previously been encouraged to explore the concept independently. My program is targeted towards a male demographic as there is sadly a larger proportion of males who are interested in machine learning and the racing game setting is also a genre with a largely male player base however females interested in machine learning and AI are also able to use this program.

I intend for this project to be used to show my users an example of machine learning in action inside of a familiar and entertaining context. This project may be used by the users on a personal computer at home if they are intrigued by the concept and want to experiment with machine learning or it may be used by teachers in a classroom to provide their students with an enjoyable and educational lesson outside of the usual curriculum. My hope is that the project itself is engaging and fun for students so that it does not seem initially daunting to begin learning about AI but using the project inspires the students to research further into the topic and develop their own understanding and the program will contain a link to further resources if the users want to then learn about how machine learning works.

I want to involve my stakeholders in the initial design process by surveying students from my college with an interest in machine learning with what they would want to see from my program and in the final testing process to review if they feel the success criteria has been met and to get their feedback on a finished version of the program.

Survey results:

I asked computer science students at my college who are interested in machine learning and AI specifically questions from an online form.

1. Would you prefer a racing game played using machine learning to be more educational to teach people about machin e learning or more for fun? More details

Most stakeholders would like the project to be mostly for entertainment so an emphasis will be made to make the project engaging and fun however this is somewhat split between respondents so an emphasis will be placed on both areas.

2. How much prior knowledge do you have about AI and machine learning? More details

The stakeholders surveyed have a mixture of a fair knowledge base on AI and a vague idea about the concept so some basic knowledge may be assumed in my program about what AI and machine learning is and what it is used for however, to make my project as accessible and educational as possible a small amount of prior knowledge will be assumed for any educational aspects.

3. Have you heard of reinforcement learning before?

More details

Many stakeholders have heard of the concept of reinforced learning so it may not need to be completely introduced from scratch however to continue making the project more accessible no detailed knowledge about the process of reinforcement learning will be assumed from the users.

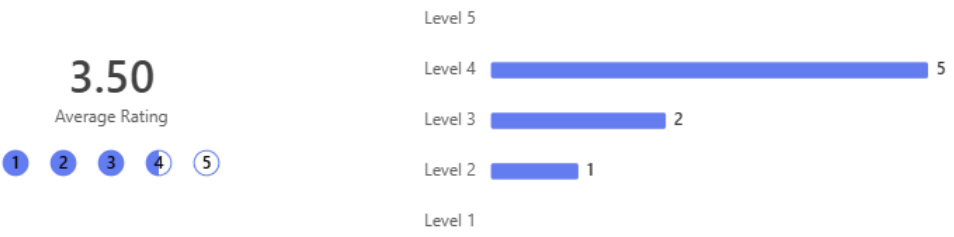
4. How important is it to you that the AI can consistently reach the finish line

More details

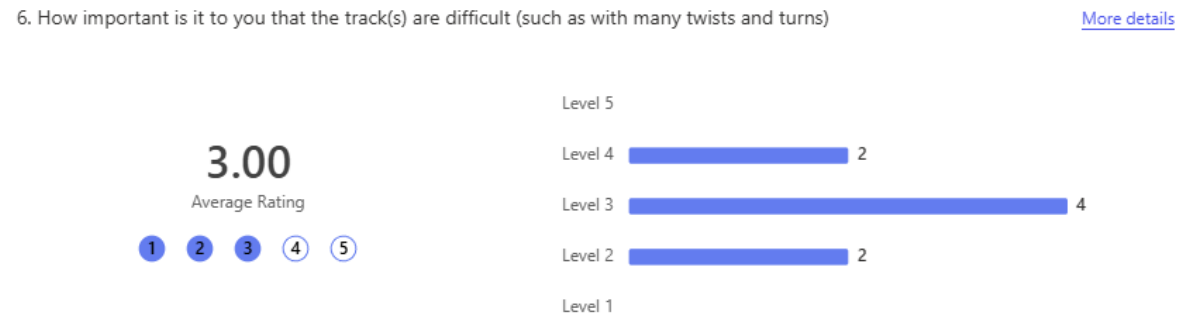
Most users view it as very important that the AI can consistently reach the finish line and complete a track so this will be the focus when training the neural network.

5. How important to you is it that the AI is able to drive quickly (without crashing)

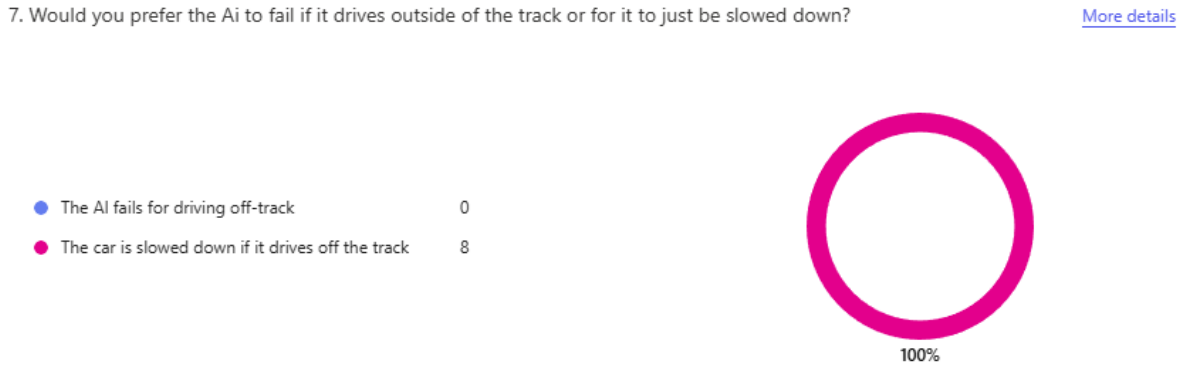
More details



It is viewed as less important that the AI can drive quickly with 0 stakeholders rating it as the highest priority so when training the AI, the speed of it will be a lower priority but still paid attention to.



The complexity of the tracks is viewed as less important than the performance of the AI so there will be some focus on this area but not enough to inhibit the AI from performing quickly or consistently finishing the race.



Everybody surveyed stated that they would prefer the AI to just be slowed down when it drives off of the track so this will be implemented as a feature.

Out of the non-essential features asked about, the ability for the user to race against the AI and more advanced racing physics and abilities were highly sought after so they are important to be completed whereas the other features will have less of an emphasis to be developed.

Requirements:

	Feature	Explanation	Justification	Importance
1.	Display window.	The car racing along the track will be visually shown in a main window which will also contain the user interface.	This must be displayed to the user so that they can see and understand the AI's decisions.	Essential
2.	A car which can be controlled.	There will be a simple car which can be controlled in a top down 2d environment with simple abilities such as accelerating/ decelerating and turning.	Having a car to be controlled by the Ai is a crucial part of the project as this is what the Ai will learn to use.	Essential
3.	A racetrack.	There will be a track for the car to race on with a defined start, finish and sides of the track.	Having a complex track layout with turns will develop a more sufficient Ai which can respond to changes on the track which both makes the program more engaging for users but also shows more of the capabilities of machine learning.	Essential
4.	Win detection	There will be a system which detects when the car has completed a lap, and when the car has completed enough laps to win the game.	It is essential that the Ai is detected when it completes laps on the track as this is the entire goal which the Ai is training for. Additionally, it is conventional in video games for the "player" to be able to win the game, so having the Ai being able to win after completing a certain number of laps	Essential

			will make the game more engaging.	
5.	A main menu	There will be a main menu screen which will allow the user to begin watching the Ai race on the track or open the settings menu to modify any settings.	This feature is essential to implement as it is necessary that the user is given the option between beginning the game or modifying the settings. Additionally, it is conventional for a video game to have a main menu screen before beginning the game, otherwise the Ai may immediately begin the race before the user is ready to watch it.	Essential
6.	A settings menu	There will be a settings menu which will allow the user to modify various aspects of the game such as accessibility features, enabling certain additional features and modifying some essential features.	This is an essential feature as the settings menu is required so that the user can access many of the important/Optional features such as the game mode to play against the Ai. This is also important so that the user can toggle some accessibility features such as a colour blindness mode and modify essential features which means that the settings menu itself is essential.	Essential
7.	A neural network	There will be a neural network which will take various information about the car in the input layer and output probabilities of the car making various actions such as accelerating.	The neural network is a fundamental part of this program and will be used to control the car and show the user an example of machine learning being used in action.	Essential
8.	More advanced physics	Including more advanced physics	The implementation of more advanced physics	Important

		will include the ability for the car to drift on the race train and the effects of resistive forces on the car.	gives is rated as highly important to my stakeholders, so it is a desirable feature for my project. The Ai more options of how to traverse the course as well as making the game a more realistic simulation of real life which will be more engaging for the user to view.	
9.	Ability to play against the Ai	This will allow for the user to control their own car and try to beat the neural network in a race.	The implementation of this feature is rated as the most important to my stakeholders, so it is a very important feature to implement to make the project more engaging. This gives the user interactivity with the AI which will cause the users to spend more time on the program and have a better experience.	Important
10.	Multiple racetracks	This will allow the AI to perform and race on different tracks with different layouts which means that the AI will need an understanding of the game mechanics in general and not just a specific track.	The implementation of this feature is rated as moderately important to my stakeholders so it is a feature which will add benefit to the engagement of my users. As this causes the AI to have to perform well even on different track layouts, it will make the AI more robust and stronger which will aid in the teaching of machine learning.	Optional
11.	Select Ais with different amounts of training	This will allow the user to watch Ais with a different amount of time having been spent	This feature will aid in the education of the users as the ability to compare the neural network after different	Optional

		on its training and compare how they perform on the track.	amounts of training will demonstrate how the time spent training impacts the performance of the agent.	

Limitations:

Some of the requirements of this project have been labelled as “Important” or “optional”. The features labelled important are requested highly by my stakeholders but are not essential for the functionality of the project and the requirements labelled as optional are not as highly requested by my stakeholders and are also not essential for the functionality of my project and will be completed with lower priority.

More advanced physics:

This feature would make the racing game more realistic and engaging which would aid in the immersion and interest from my users. However, only basic racing options are essential for the project to be completed such as accelerating/ decelerating and turning. Additionally, the implementation of the neural network is a much larger task and is more impactful than the quality of the racing game, so I am prioritising the creation and training of that before implementing additional physics.

Ability to race against the AI:

This feature would allow the users to become more involved with the AI and adds a fun game to the project which would make it more memorable, engaging and educational. However, the AI only needs to be watched completing the tracks by itself for the completion of this project, so this feature is labelled as “important” as the creation and training of the neural network is more crucial than an additional game mode for this project. Additionally, creating this feature will require the creation of an entirely separate game mode with competitive racing elements and user inputs so due to the scope of this project it would be too time consuming to deem an essential feature.

Multiple racetracks:

This feature would cause the AI to have to perform well even on different track layouts, which would make the AI more capable to work in different scenarios which will aid in the teaching of machine learning as neural networks allow for the AI to work well even in new scenarios. However, only a single racetrack is essential for this project to be completed as this is sufficient to show most of the capabilities of machine learning and the creation of different racetracks will not only cost time to design them to test different skill sets of the neural network's racing but it will also significantly increase the amount of training time required for the neural network to consistently perform well . Additionally, this feature was only moderately requested by my stakeholders so it would not increase the engagement of my users as much as some other features.

Comparing different amounts of training:
This feature would require multiple neural networks to be stored as part of the application to be used and compared with each other which will likely use a large amount of memory and will be complicated to store. Also, it would be much simpler to only work with a single neural network instead of saving its state at multiple points during training to compare with and this was a lowly rated feature by my stakeholders meaning that it should not be of a high priority to include.

Hardware and software requirements:

I will be creating this project in the language python using pygame for my GUI and windows as my operating system. The implementation of a neural network increases the hardware requirements significantly as they are computationally expensive to train and implement.

Hardware:

- This project is just run in python and requires no additional hardware so a typical general-purpose computer can be used.
- A decently capable CPU is required to train the neural network (such as an intel i7/i9 or AMD ryzen 7/9) however the end user will only need to use the neural network and not train it themselves therefore they may be able to use a less effective CPU

- A strong GPU is required to train a neural network (such as an NVIDIA RTX 3080) however the end user will only need to use the neural network and not train it themselves therefore they are very likely to be able to use a less effective GPU
- 8GB of VRAM is also required for training the neural network but this may be less for the final user when the network is fully trained.
- 16GB of RAM is recommended for training a neural network but this also may be less for the final user.
- A large amount of storage may be required to contain the neural network (A terabyte is recommended for training a large neural network however this neural network is small) so this is likely to be required from the user.
- A mouse and keyboard are required for all the major inputs in the program.

Software:

- Both Python and Pygame are available on many operating systems, however as this project is being created in windows, a windows OS may be optimised for this project.
- A graphical user interface is required for the operating system but not much else.
- The user will need to have installed Python and Pygame independently, however these are both free and accessible.

Computational methods:

I believe that this project is suitable for a computational approach because the following methods can be used to complete it:

- Abstraction can be used to ensure that all key elements of the project are focused on and completed. This also means that more unnecessary features such as an improved racing game or image recognition with the neural network do not need to be implemented so that the core features can be created in the given timeframe. Abstraction is a tool widely and easily used for computational problem solving as it is straightforward to decide which elements of a program are more important than others which means the problem is suited to a computational approach.
- Visualisation can be used to effectively show the user's machine learning being displayed in action inside of a visual environment which is a much more natural and engaging way to get students interested in machine learning rather than describing it. Being able to visualise abstract concepts such as machine learning in engaging and interactive ways such as using a racing game is greatly strengthened using a computational approach as concepts can be graphically

demonstrated on a screen, so the problem is suitable for a computational approach.

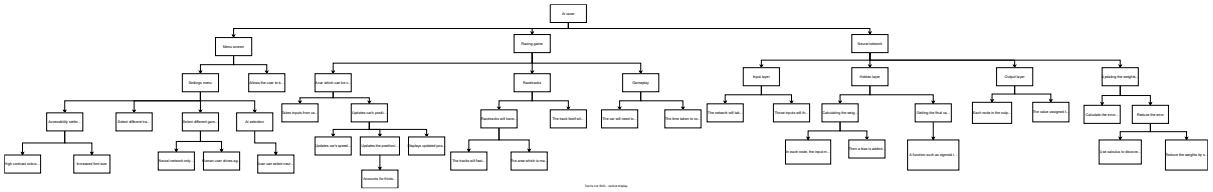
- Decomposition will be used to break down the sections of the project into smaller parts to be designed individually which will make the project easier to design as only a smaller problem needs to be solved at a time. For example, the racing game and the neural network elements can be designed separately and they themselves can be broken down into smaller features to be made. This can be done computationally by dividing processes in the program into modular subroutines which means that a computational approach works well with decomposition of the problem meaning that the solution to the problem is easier to plan and design.
- Some elements of the program will be executed concurrently. For example, the neural network will make the decision of which actions to take when driving in the same frame as the driving game is updated. This is suited to a computational approach as concurrent processing is regularly performed on multi-core processors to execute multiple processes inside of a program faster.
- Q-learning which is a reinforcement learning algorithm will be used to train the neural network to drive the car. This is a suitable algorithm to meet the needs for this project as this method allows the agent to begin training initially making completely random decisions however as the agent receives positive and negative feedback as the results of its actions, its decisions begin to shift towards ones with a higher probability of positive feedback. This means that for a racing game, the car will begin driving randomly and poorly but after further training, the agent will begin to drive in a way which allows it to reach the finish line. This algorithm is good to visually show how machine learning adapts and evolves and will be capable of performing well in the racing game context as positive reinforcement can be getting closer to the finish line and negative reinforcement can be driving off the track and staying still.

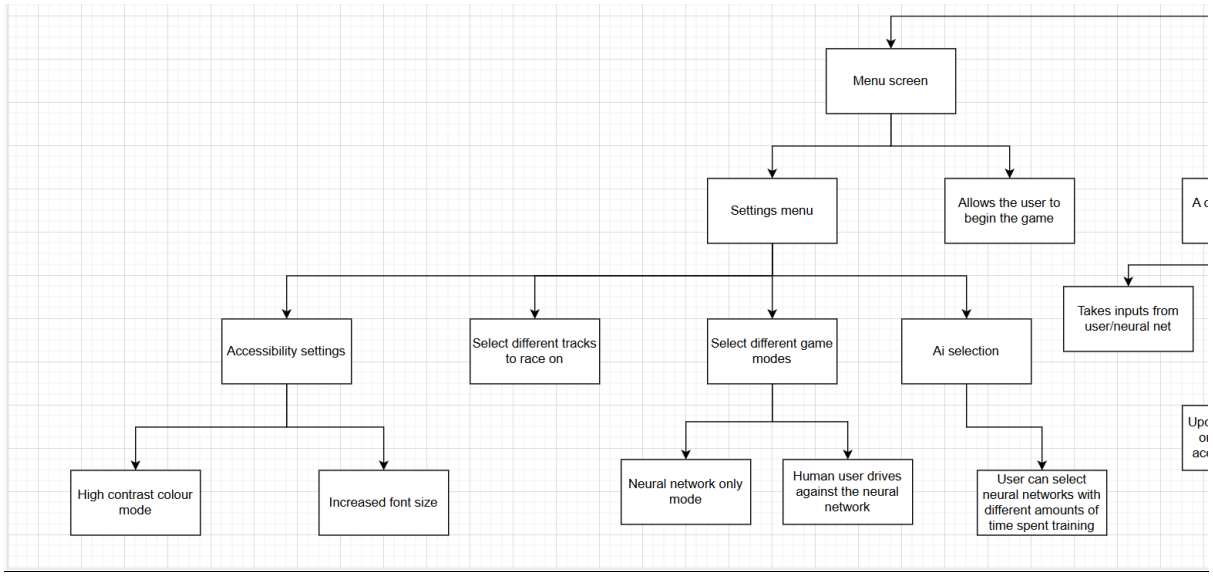
Design:

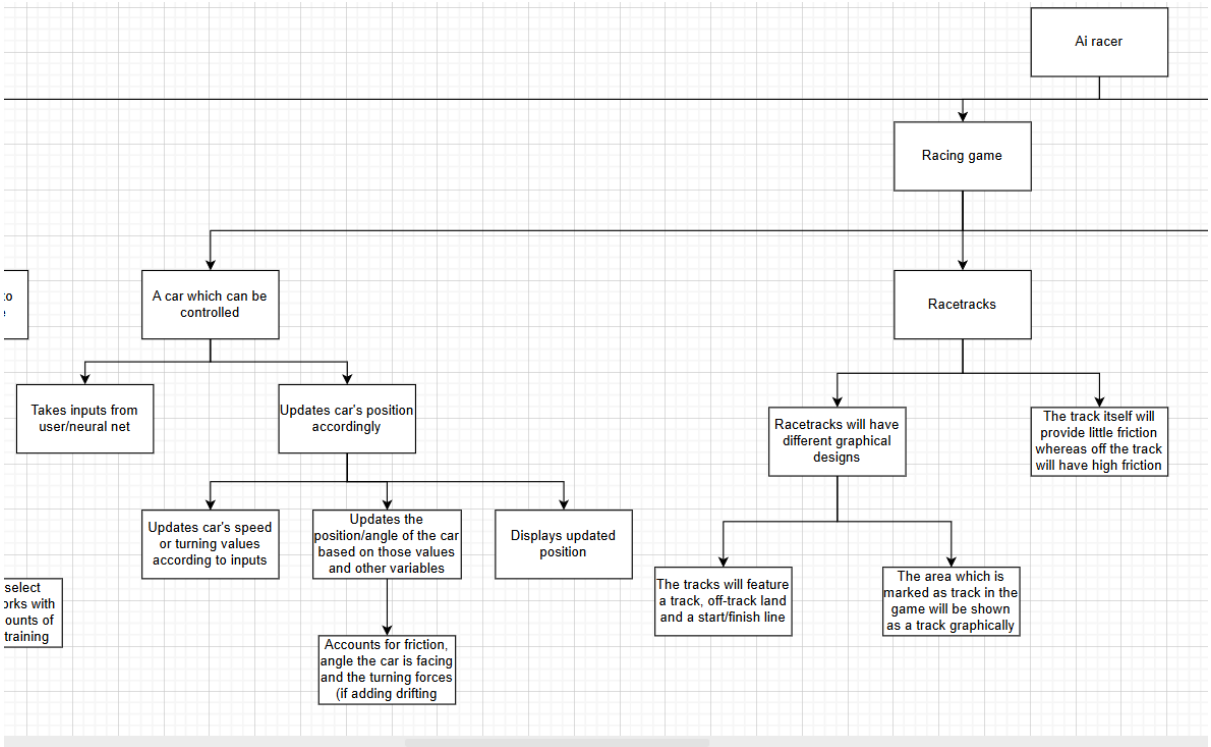
Structure diagram:

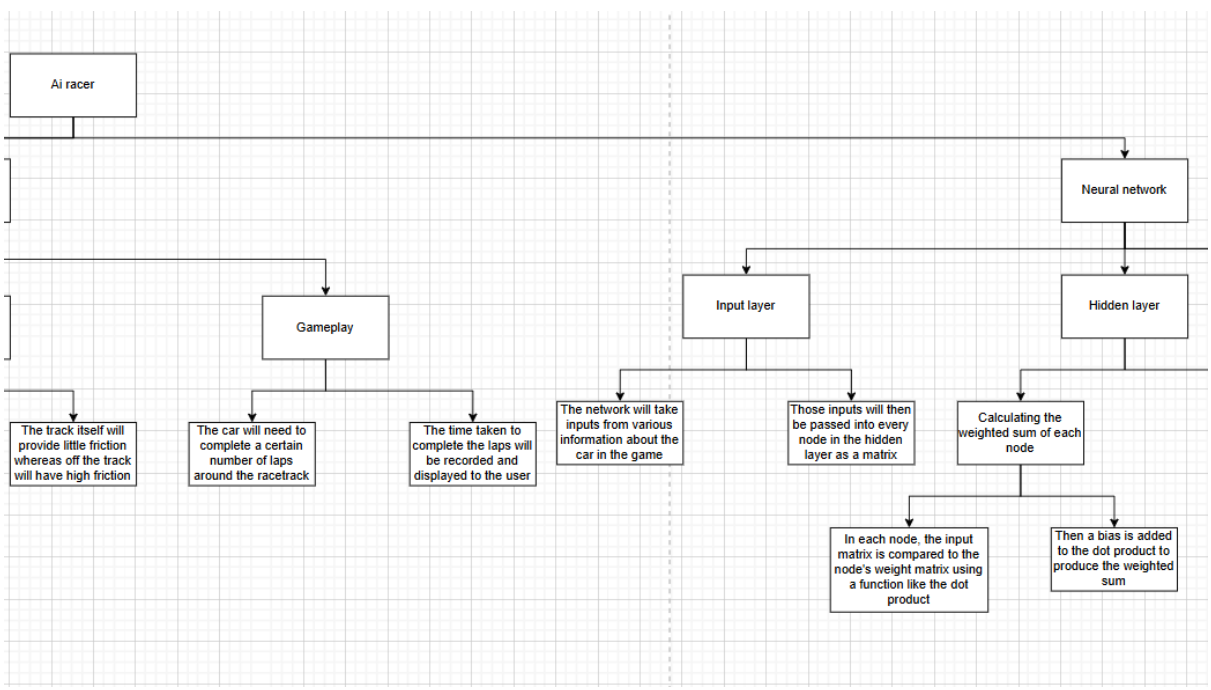
Here is my structure diagram. I decided to break the overall structure into three main sections: the main menu, the racing game and the neural network. This is because the

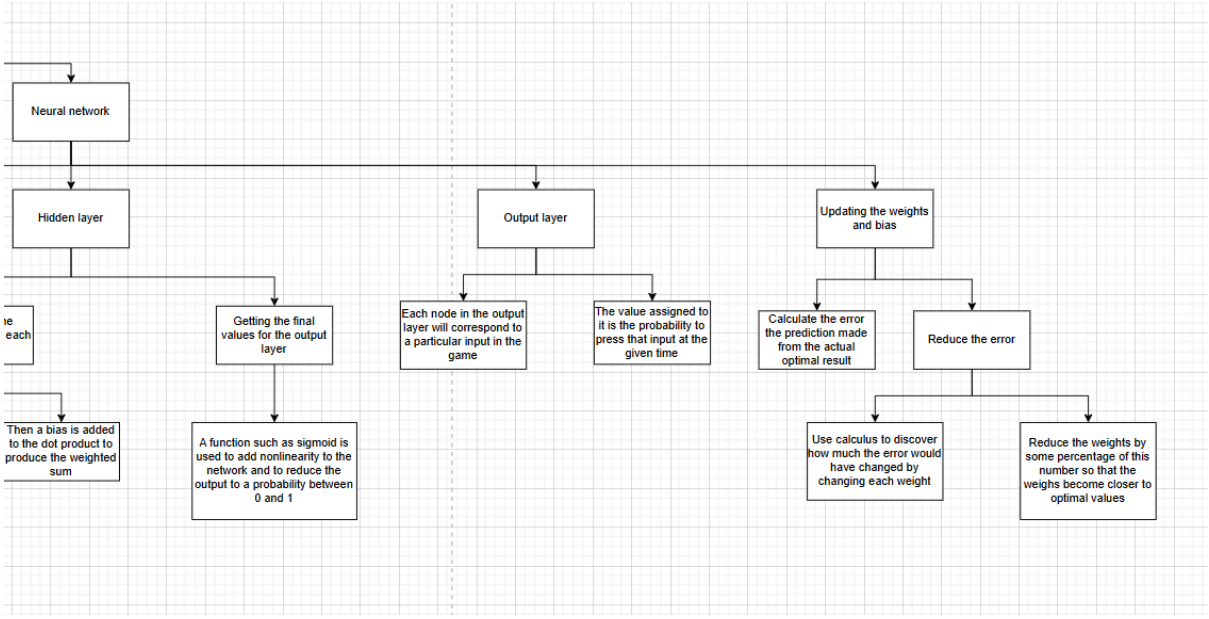
racing game is the game itself which is processed and displayed on screen and functions separately from the neural network which calculates how the car in the racing game should use. The neural network takes values as inputs from the racing game, processes them and then returns to the game what the car should do so although they communicate with each other, the racing game and the neural network are separate processes and can be designed separately. The main menu allows the user to customize the program to their preferences which will bring greater engagement from the user. Then the main menu allows the user to begin the racing game itself which uses the neural network so the main menu can also be designed as a separate process to the racing game as it is a separate process.






Ai racer
Racing game
A car which can be controlled
Racetracks
Takes inputs from user/neural net
Updates car's position accordingly
Racetracks will have different graphical designs
The track itself will provide little friction whereas off the track will have high friction
Updates car's speed or turning values according to inputs
Updates the position/angle of the car based on those values and other variables
Displays updated position
The tracks will feature a track, off-track land and a start/finish line
The area which is marked as track in the game will be shown as a track graphically
Accounts for friction, angle the car is facing and the turning forces (if adding drifting


Ai racer
Neural network
Gameplay
Input layer
Hidden layer
The track itself will provide little friction whereas off the track will have high friction
The car will need to complete a certain number of laps around the racetrack
The time taken to complete the laps will be recorded and displayed to the user
The network will take inputs from various information about the car in the game
Those inputs will then be passed into every node in the hidden layer as a matrix
Calculating the weighted sum of each node
In each node, the input matrix is compared to the node's weight matrix using a function like the dot product
Then a bias is added to the dot product to produce the weighted sum



Development plan:

Stage 1- creating a controllable car:

Features to implement:

- A display window to contain the game
- A car which speeds up when up is pressed and slows down when down is pressed
- The car can also turn right when right is pressed and turn left when left is pressed
- Create boundaries at the edge of the display window so that the car cannot leave the screen

In this stage, a display window will be created to display all of the events which occur inside the racing game. Next, a car will be displayed on screen which will accelerate and decelerate with user input and is also able to turn left and right. A frictional force will be applied as well which will slow down the car when it is moving. I will also implement boundaries at the edge of the display window so that the car cannot leave the screen.

I decided to create the controllable car before the gameplay and the track as both of those features require the existence of the car to some extent. For example, the racetrack's collision detection with the car cannot be implemented until the car is

created. The neural network which will be trained on the game cannot be trained until all of the racing game elements of the project is built (the car, racetrack and gameplay) so that must be done before the neural network.

Stakeholders voiced that it is important for the game to have accurate and advanced racing physics, so care should be taken to ensure that the car's movement is realistic and fun to use.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program.	When the program is run, see if a display window appears.	A display window will appear.	
2	Run the program.	When the program is run, see if a car is displayed on the display window.	A car will appear inside of the display window.	
3	Run the program and press W.	When W is pressed, see if the car moves forwards.	The car will move forward when W is pressed.	
4	Run the program and press S.	When S is pressed, see if the car moves backwards.	The car will move backwards when W is pressed.	
5	Run the program and press A.	When A is pressed, the car's image will rotate anticlockwise.	The car will turn anticlockwise when A is pressed.	
6	Run the program and press D.	When D is pressed, the car's image will rotate clockwise.	The car will turn clockwise when D is pressed.	
7	Run the program and press and hold A. Then press W.	Pressing A will cause the car to rotate anticlockwise and then pressing W will cause the car to move forwards. The movement of the car should be going forward in the same direction that the car is facing after it is turned.	The car moves in the same direction the car is facing when W is pressed.	
8	Run the program and press and hold A. Then press S.	Pressing A will cause the car to rotate	The car moves in the opposite direction the car	

		anticlockwise and then pressing S will cause the car to move backwards. The movement of the car should be going backwards in the opposite direction that the car is facing after it is turned.	is facing when S is pressed.	
9	Run the program and press W then let go.	When W is pressed, the car will move forward. After W is let go, the force of friction should be applied to it.	After W is let go, the car should gradually slow down and eventually stop.	
10	Run the program and hold W until the car reaches the edge of the display window.	The car should be stopped when it reaches the edge of the display window so it cannot leave.	The car will stop moving as it reaches the edge of the display window.	

Stage 2- creating the racetrack:

Features to implement:

- The background of the game should indicate where on the screen is the track, off-track sections and the start/finish line
- The car should store whether it is currently on or off track and update the friction acting on the car due to that

The racetrack must be created next. This is because the gameplay features require a finish line to be defined so that the car is able to win the race so the gameplay stage is dependent on the racetrack being made first.

In this stage, a graphical track will be displayed on the screen as the background. When the car is on the track, one of its attributes will be set to being on the track which will cause the car to have a lower friction value used, however when the car is detected as leaving the track, then its attributes will be set to being off the track and it will have a higher friction value used. Additionally, the start and finish line will also be created and displayed on the track which will allow for collisions with it to be later used.

Stakeholders unanimously agreed that the car should only be slowed down when leaving the track, so implementing that feature will cause the game to be more engaging for its users.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program.	When the program is run, there should be a background image displayed.	A background image is displayed.	
2	Run the program.	The background image should be of a racetrack which includes track, off-track sections and the start/finish line.	The background image is of a racetrack and displays the track, off track and start/finish line.	
3	Run the program and press W then let go on track. Then repeat the same off track.	There should be stronger friction applied to the car off track than on track.	When off the track, the car slows down faster than on the track.	
4	Before running the program, add a temporary line of code which prints the text "Test passed" when the car collides with the start/finish line. Then run the program and then drive the car over the start/finish line.	There should be a collision check implemented with the car and the finish line. To test this, the string "Test passed" will be printed when there is a successful collision between the car and the finish line.	"Test passed" will be printed only when the car collides with the finish line.	

Stage 3- creating the gameplay and menu:

Features to implement:

- The car will be tracked for how many laps it makes around the track and once it reaches a set number the game will end
- The car's time spent on each lap will be recorded and displayed when the game has finished.

- A main menu, settings screen and race finish screen will be created which will open when the program is run

Now that the car and the racetrack have been completed, the gameplay is able to be implemented. This has been chosen to be created before the neural network as the neural network needs to be rewarded when it performs well in the game and penalised when it performs poorly, however this cannot happen before the gameplay itself has been made.

The main menu, settings menu and race finish screen will also be made at this stage as some of the settings which will be available to change are directly involved with the gameplay element so should be developed at the same time and the menu is a relatively small feature to include.

Stakeholders should be consulted on which settings they would like to have in the program so that the application can be as customizable as they please and to increase the functionality of the program.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program.	The main menu screen should appear instead of the usual racetrack.	The main menu screen should appear instead of the usual racetrack.	
2	Run the program and click the “start racing” button with the mouse.	This button should cause the racetrack to appear and the gameplay to begin.	The racetrack then appears, and the gameplay begins.	
3	Run the program, press “start racing” on the main menu and drive the car around the track so that it passes over the finish line 3 times.	When the car drives over the finish line for the third time, the race finish screen should appear.	The race finish screen appears.	
4	Run the program, press “start racing” on the main menu.	A timer should appear at the top of the screen which counts the time passed after the race began.	The timer appears and counts down.	
5	Run the program, press “start racing” on the main menu.	A lap counter should appear at the top of the screen which counts how many	A lap counter should appear at the top of the screen which begins at 1/3	

		laps have occurred.		
6	Run the program, press “start racing” on the main menu and race a single lap.	The lap counter should increment by one when the finish line is reached.	A lap counter should appear at the top of the screen which begins at 1/3 and reaches 2/3 when the finish line is first reached	
7	Run the program, press “start racing” on the main menu and race two laps.	The lap counter should increment by one when the finish line is reached each time.	A lap counter should appear at the top of the screen which begins at 1/3 and reaches 2/3 when the finish line is first reached and 3/3 when it is reached a second time.	
8	Run the program, press “start racing” on the main menu and attempt to drive the car in the opposite direction through the finish line.	The game should not allow the car to simply drive back through the finish line resulting in a completed lap so this should not affect the lap counter.	The lap counter is unaffected	

Stage 4- creating the neural network:

Features to implement:

- An input layer for the neural network to take inputs from the game.
- A hidden layer for the neural network to process the given information and make predictions
- An output layer to return the probabilities that the neural network should use each input

After the game is fully completed, then the neural network can begin to be made. The neural network must be created before it is able to be trained therefore it is an earlier stage than training the neural network.

The input layer will take various inputs from the game such as the distance from the side of the track and the speed of the car. These values will then be stored in a vector which will be passed into every node in the hidden layer. Here, the vector's dot product is found with the weight for each node and a bias value is added to find the weighted sum of each node. Then a nonlinear function such as sigmoid is used which reduces the sums to a probability value between 0 and 1 for each output node. The output layer will then return the probabilities to the game which will execute the inputs and then repeat the process. At this stage, the weights and biases are random values, and the neural network is not trained however this will allow for communication between the game and the neural network which must be done before it can be trained.

The majority of stakeholders said that it was very important that the A.I. can reach the end of the track consistently so when choosing which inputs/outputs the neural network has access to in the game, inputs and outputs which would make the A.I. more likely to reach the finish line should be prioritized.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program, press "start racing" on the main menu.	The car should move randomly. This is as the neural network now controls the car's inputs however it has not been trained so it makes random decisions.	Once the gameplay begins, the car randomly moves.	
2	Before running the program, add some code which prints the input data to the neural network from the game. Then run the program, press "start racing" on the main menu.	Analyse each variable and measure if each value is correct as it is passed into the neural network.	Each variable is correct which is passed into the neural network.	
3	Before running the program, add some code which prints all of the outputs from the neural network to the game.	Ensure all values are between 0 and 1.	All probabilities are between 0 and 1.	
4	Before running the program, add some code which prints all of the outputs from the neural network to the game.	Ensure the probability values of each car input vary each frame.	The probability values vary.	

Stage 5- training the neural network:

Features to implement:

- The error that the neural network makes from what would be optimal is calculated
- Calculus and mathematics is used to determine how much the error would vary by changing each weight
- The weights are subtracted by a fraction of this amount to put them closer towards their optimal values
- The neural network is trained

After the neural network is completed, it can now be trained on the game which means this stage had to be the final essential stage. This will result in the neural net being able to control the car much more effectively which means that it will be able to race around the racetrack and complete laps instead of moving randomly.

Here, the neural network calculates the error it has made from the optimal prediction by using a cost function. Then, calculus and partial derivatives are used to determine the differential of the error with respect to the weight which means how much the error would change if the weight was increased by 1. This value multiplied by a learning rate (which is a constant value used to decide the speed the neural network trains at as well as the precision) is subtracted from the weight so that the weight is now slightly closer to the optimal value it can be. Then, this is repeated many times so that eventually the weights become very close to their optimal values, and the neural network can play the game effectively.

A large amount of stakeholders said that it is important that the A.I. can drive quickly so the speed of the A.I. will be considered when deciding the cost function so that the A.I. drives quickly and consistently.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program, press "start racing" on the main menu. Observe if the car approaches the finish line. Before running the program again, add some code which runs a training subroutine in the neural network to train it inside of 10000 racing games.	The car should not approach the finish line before training but after training the neural network it should.	After training, the neural network's ability to reach the finish line improves.	

	Run the program, press “start racing” on the main menu. Observe if the car approaches the finish line.			
2	Add some code to run the training subroutine in the neural network a single time and print the values of the error calculated by the neural network and then each of the derivatives calculated.	Compare the values calculated by the program with values calculated by hand to confirm that the neural network’s error calculations are correct.	The neural network correctly calculates how much each weight should be updated by due to the calculated error.	

Stage 6- implementing additional features:

Features to implement:

- More advanced physics
- The game mode to play against the Ai
- Multiple different racetracks to be selected
- The ability to select Ai’s with different amounts of training

After all the essential features have been completed, the features identified as important or optional can be implemented. As this stage only contains non-essential features, it would be acceptable for this stage to be only partially completed considering the scope of the project which is why this stage will be developed last.

In this stage, the car’s movement is updated to allow for it to “drift” which means that when the player turns the car when it is at a high speed, the back of the car will turn faster than the front of the car. Additionally, more resistive forces will be considered apart from friction such as the effects of wind on the car.

A game mode to play against the Ai will also be included inside of the settings menu. This mode will mean that two cars will be created and displayed on the screen when the gameplay begins. One of the cars will remain the same and will be controlled by the neural network however the other car will be controlled by the user and both cars will compete to reach the finish line first. This will also cause two separate lap values to be displayed during gameplay to show which lap each of the cars are on and the race finish screen will display a different time for each car.

In the settings menu, different racetracks will be able to be selected. Changing this setting will mean that when the gameplay starts, there will be a different background image which will display a different racetrack and the on track and off track sections of the screen that the car can drive on will be changed to correspond with the background image.

In the settings menu, there will be the option to select different Ais to race and control the car which will each have had different amounts of training which means that the ones with little training will likely perform worse than ones with high amounts of training.

A large amount of stakeholders said that it is important that the A.I. is able to drive quickly so the speed of the A.I. will be taken into account when deciding the cost function so that the A.I. drives quickly and consistently.

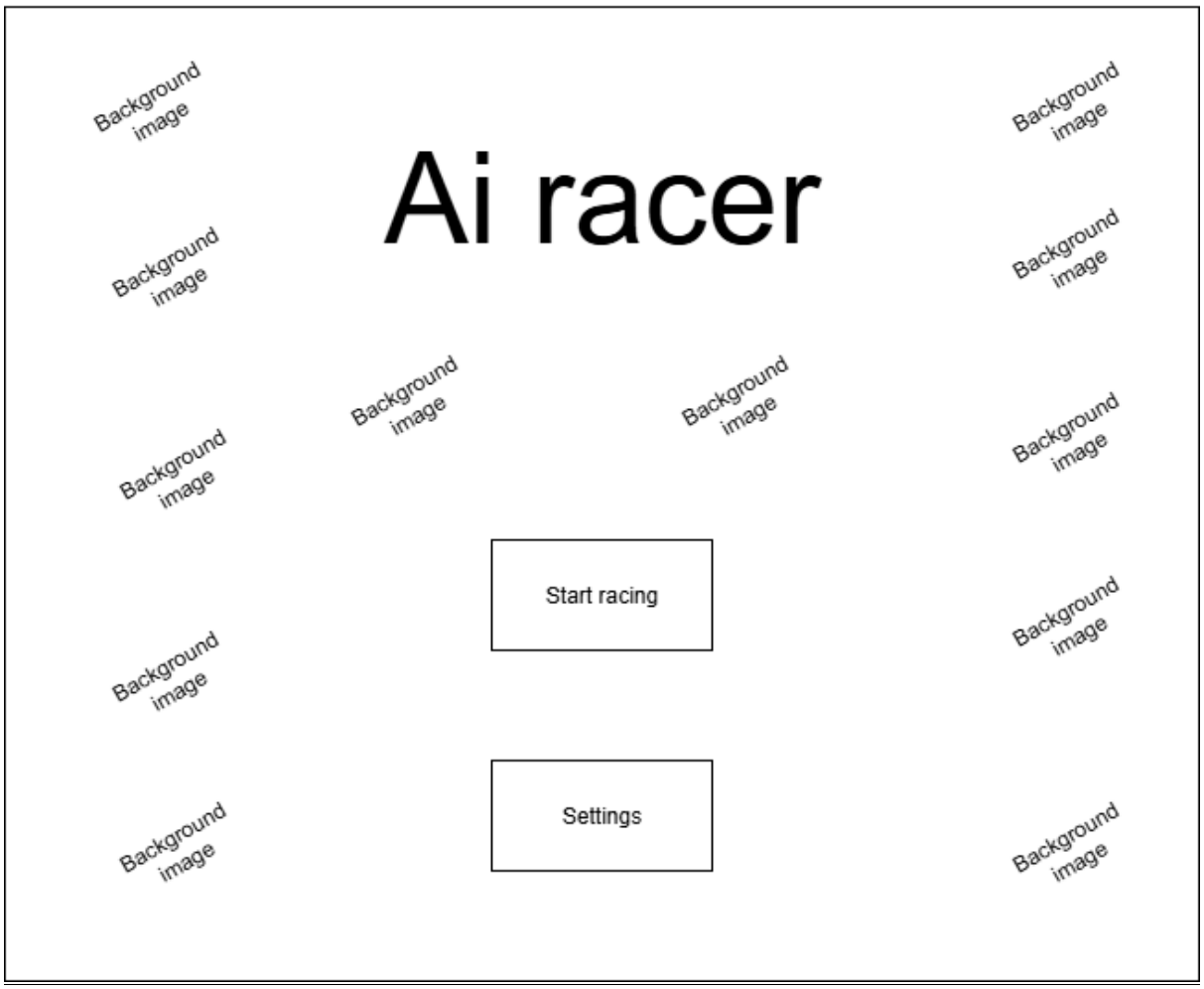
Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program, press “start racing” on the main menu.	The car should be observed until it reaches a high velocity and attempts to turn. The back of the car should turn faster at higher velocities.	The back of the car turns faster than the front at higher velocities.	
2	Before running the code, ensure that there is a high wind value. Run the program, press “start racing” on the main menu.	The wind should decrease the velocity of the car if the car heads against it and it should increase the velocity of the car if the car heads with it.	The car’s velocity decreases when driving towards the wind and the velocity increases when driving away from it.	
3	Before running the code, ensure that there is a high wind value. Run the program, press “start racing” on the main menu.	There should be a visual indication on the screen that there is wind.	There is a visual indication on screen that there is wind.	
4.	Before running the code, ensure that there is a high wind value. Run the program, press “start racing” on the main menu.	There should be an indication on screen of the direction where it is blowing	There is a visual indication on screen of the direction which the wind is blowing.	
5	Run the program, press “settings” on the main menu. Change the “game mode” setting to be “Vs Ai”. Return to the main menu and press start racing.	There should be two cars displayed on the screen when this mode is selected.	Two cars are displayed on screen.	

6	Run the program, press “settings” on the main menu. Change the “game mode” setting to be “Vs Ai”. Return to the main menu and press start racing.	One of the cars should be controlled by the neural network and should move on its own whereas the other should be controlled by the player.	One car moves on its own while the other car does not move.	
7	Run the program, press “settings” on the main menu. Change the “game mode” setting to be “Vs Ai”. Return to the main menu and press start racing. Hold W for 3 seconds.	The car which is controlled by the player should move forwards when W is pressed as it takes inputs from the user instead of the neural network.	The user’s car moves forward when W is pressed.	
8	Run the program, press “settings” on the main menu. Change the “Track” setting to be a different one than normal. Return to the main menu and press start racing.	The background image should be changed to a different one.	The background image will be different to the usual background.	
9	Run the program, press “settings” on the main menu. Change the “Track” setting to be a different one than normal. Return to the main menu and press start racing. Then drive the car over the off-track section displayed.	The car’s friction should be lower on the newly displayed track and higher on the newly displayed off-track.	The car’s friction is lower on the track and higher on off-track.	
10	Run the program, press “settings” on the main menu. Change the “Track” setting to be a different one than normal. Return to the main menu and press start racing. Then drive around the track and reach the finish line.	The lap counter should increase by 1. This is because the newly displayed finish line on screen should be the position which increases the lap counter in game.	The lap counter increases by 1.	
11	Run the program, press “settings” on the main menu. Change the “Ai selection” setting to be the Ai with the least amount of training. Then, return to the main menu and press start racing. Then watch the neural network drive around the track and record the time taken by the Ai once the race	The neural network with little training should perform worse than the neural network with lots of training so their times should be compared. This test should be	The Ai with little training should have a higher time taken than the Ai with lots of training.	

	is over. Then, return to the settings menu and change the Ai to the one with the highest amount of training and return to the main menu. Next, press start racing and record the time that it takes for this Ai to complete the race.	completed at least 5 times in case some results differ from the average time for each network.		

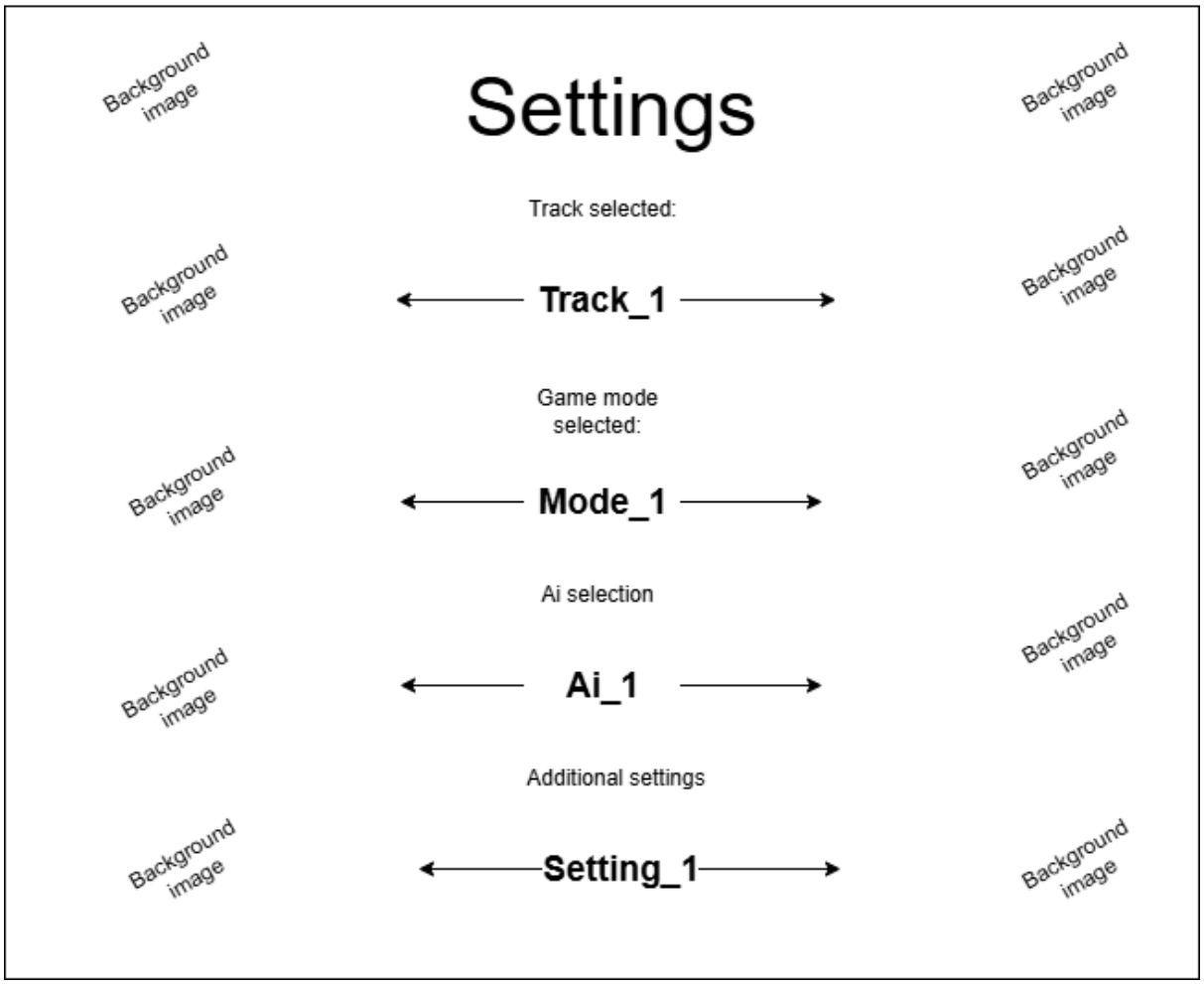
Gui design:

Main menu

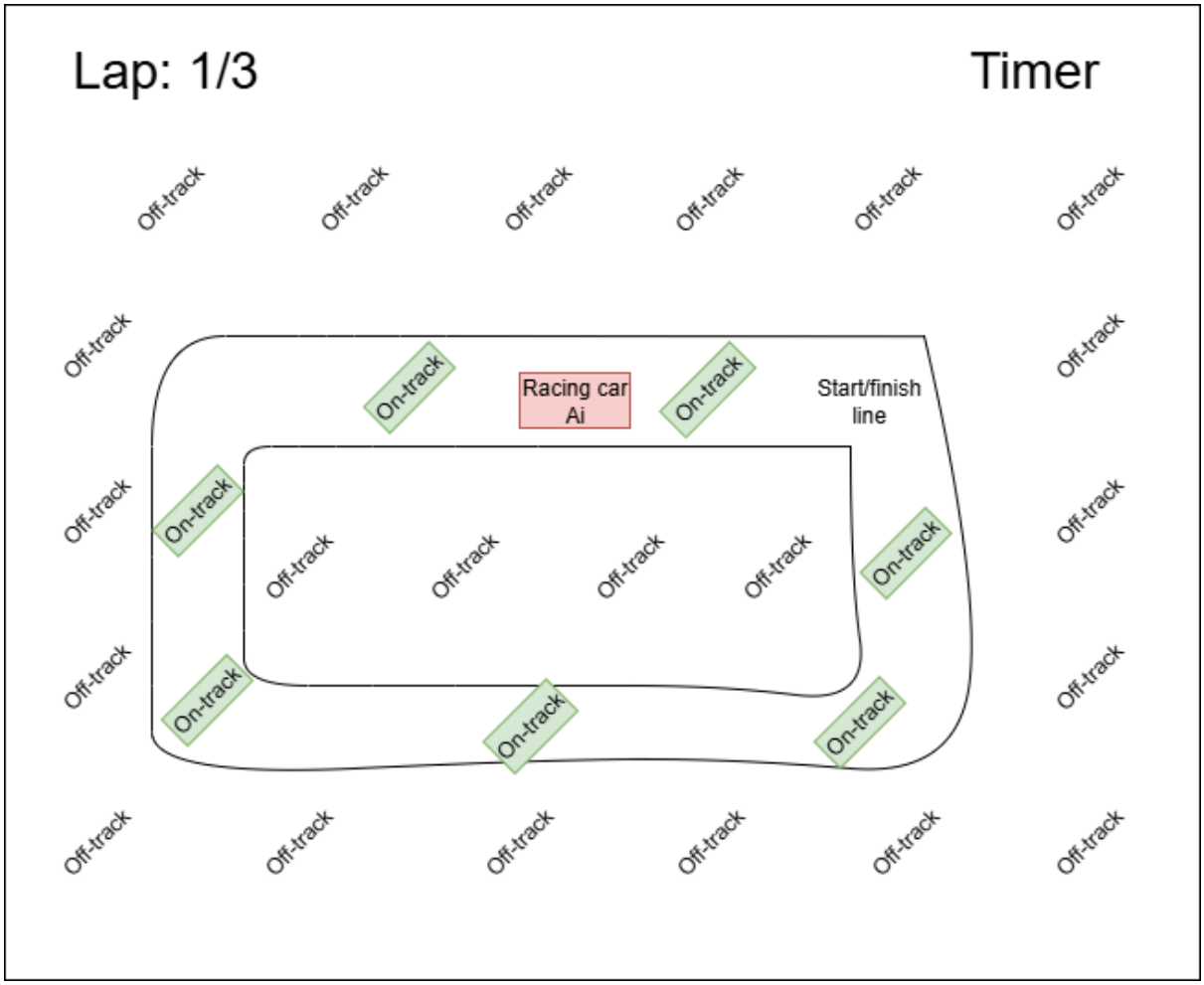


This is the main menu of the program which will be the first screen that the user is met with when running the program. The “Start Racing” button will change the screen to display the car and the track and the racing game itself will begin using the currently applied settings. The “Settings” button will change the screen to the settings menu which will change certain functionality and accessibility features in the game for the user’s requirements. The background image will be a sample image of the racing game.

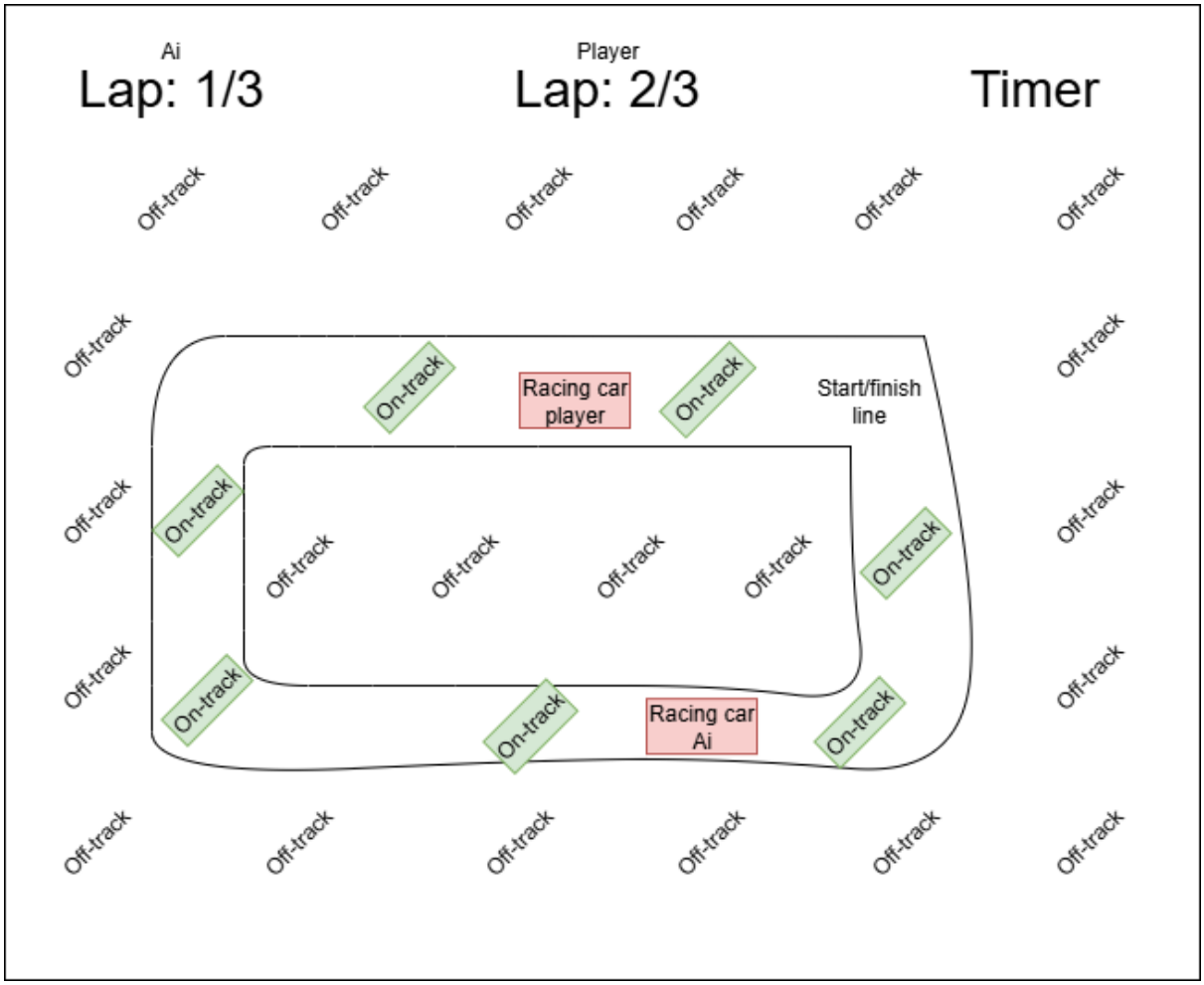
Settings



This is the settings menu which will be displayed after the user presses the settings button. When the arrows are clicked on each setting, the setting will switch to the next value it can be set as. The track selected setting will change the layout and image of the racetrack when the gameplay begins. The game mode setting will allow the user to choose between the neural network driving by itself or the player racing against the neural network. The Ai selection setting will allow the user to chose to use different neural networks with different amounts of time spent training. The additional settings will contain settings to modify other things such as a high contrast colour mode to assist with colour blind users and other accessibility settings.



This is the gameplay screen which is accessed by pressing the “Start racing” button. The timer begins from 0s and counts up as the game progresses. The lap counter displays the current lap. The “on-track” sections of the screen are which would be classed as the track in this example which are enclosed by the two track edges. The “off-track” sections of the screen are the areas which would be classed as off the track in this example as they are not between the two edges of the track. The start/finish line represents where the start/finish line could be which is where the race starts and ends. Instead of text, the off-track and on-track sections will be displayed graphically with the track likely having a colour such as black which is typical of roads and racetracks whereas the off-track sections will be made up of colours such as green and brown to represent grass and mud which will allow make the user understand which sections of the screen are on and off the track. Additionally, a setting will be added for colour blind users which will use a similar labelling system as used above to differentiate between the on and off-track sections. The racing car Ai also represents the neural network’s car which will drive on the track. This gameplay screen is only accessible if the game mode setting is set as Ai only. If the play against the AI mode is selected, the following screen will be displayed instead:



This is very similar to the other gameplay screen, however there is an additional racing car for the player to control on the track and there are separate lap counters for the player and the Ai as they are able to be on different laps at the same time.

The race finish screen will be displayed once all laps have been completed in a race. The “Time:” section will display the total time it took for the Ai to complete all of the laps. The new race button will begin a new race with the same settings already applied whereas the main menu button will display the main menu screen. This is the race

finish screen when the Ai races by itself. The player vs Ai race finish screen is here:

This screen is very similar to the Ai only screen however it displays a different time for the player and the Ai which displays the time it took for each of them to finish the race individually.

Post development test plan:

Finally, before development begins I have created a post-development test plan. This will cover tests that I will do myself to test the program in its entirety and its robustness as well as questions that I will ask a group of stakeholders who have used my program focused on usability. Please note that this is only a test plan and certain tests may be added if some of the optional features are implemented or certain features are not met.

Here is the plan for the tests I will do myself:

Test number	Required inputs	Test description	Success criteria	Pass/fail
1	Run the program on the lower spec pc, begin racing with the Ai and compare how well the program can run compared to a higher spec pc.	Run the program on a lower spec pc and record its frame rate and performance.	If the program is robust, then the frame rate on a lower spec pc should be similar to the frame rate on a higher spec pc and the program should be able to run on a lower spec pc.	
2	Run the program, navigate to the settings menu and attempt to change settings beyond their maximum values.	Attempt to select unintended values for settings in the settings menu	If the program is robust, it will prevent the user from being able to select unintended values for settings and it will instead loop back to the first value of the setting when attempting to select settings beyond the final one.	
4	Run the program, begin racing. Attempt to drive the car off all the edges of the screen.	Attempt to drive the car off the screen.	The car is prevented from crossing over the edge of the screen and is kept inside of the display window.	
5				
6				

After testing the functionality and robustness of the program, I will gather a group of stakeholders and ask them the following usability questions:

1. How easy was your experience navigating the in-game menus?

I chose to create this question as the navigation between menus takes place often during the program so this will reflect the usability of the program.

2. To what extent did the available settings improve your experience of the game?

I chose to create this question as the settings can greatly improve the usability of the game, so if the stakeholders respond well to this question, then this well reflect upon the usability of the program.

3. Are there any additional settings you would like to introduce to improve your experience of the game?

I chose to create this question as the settings can greatly improve the usability of the game, so if the stakeholders list many additional settings that they would like to have that are not present in my program, then this may suggest that the usability of the program was poor.

4. How realistic did you find the game's physics system compared to any other 2D games?

I chose to create this question as stakeholders have previously identified that the accuracy of the game's physics is important, so if they find the physics system unrealistic then the usability of the program may be poor.

5. How enjoyable was the game's physics system?

I chose to create this question as stakeholders have previously identified that the game's physics is important, so if they find the physics system enjoyable then this will increase their engagement with the program.

6. Are there any improvements that you would want to be implemented for the game's physics system?

I chose to create this question as stakeholders have previously identified that the accuracy of the game's physics is important, so if there are many improvements that they would like to add to it then it would suggest that the physics system does not meet their standards and that the usability of the game is poor.

7. How would you rate the game's graphics and GUI?

The graphics and GUI of a game represent how all the information in the game is displayed. If all the information is displayed clearly and informatively then this means that the usability of the game will be good.

8. Do you have any concerns with the accessibility of the game?

I chose to create this question as if many stakeholders have concerns with the accessibility of the game, then the usability will be poor for many users who require specific accessibility features to engage with programs.

9.How well was the neural network explained during the game?

I chose to create this question as the neural network will control the car in the game, however, if the stakeholders feel that the neural network was not explained well then they may have a difficult time understanding what is going on in the program which means that the usability will be poor.

10.How well do you feel the neural network performed on the track?

This question on its own relates to the functionality of the program and not the usability. However, the next question relates to usability directly and requires that this question was asked first.

11.How easy was it for you to evaluate the neural network's performance?

I chose to ask this question as if the stakeholder finds it very easy to evaluate the neural network's performance then this may mean that the information was displayed very clearly in the game and it was very easy to understand how well the neural net was doing which means that the usability of the program would be good.

Development:

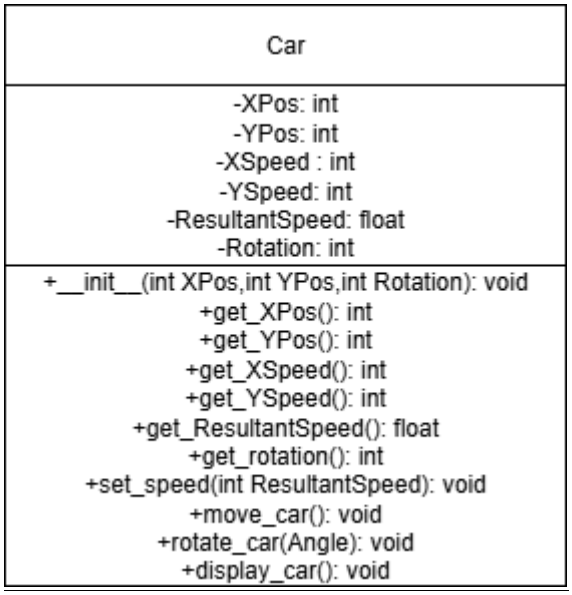
Stage 1- creating a controllable car:

Now that the program design is complete, the iterative development of the program can begin. Firstly, a display window will be created. To do this, I will be using pygame so, I watched and followed this pygame tutorial (Pygame tutorial) to familiarize myself with the basic functionality of pygame. After this, a basic car image will be displayed onto the screen. Then, user input will be implemented which will mean that if W is pressed, then the speed of the car will increase going forwards and if S is pressed then this will increase the speed of the car going backwards. Additionally, if A is pressed, then the car will rotate anticlockwise and if D is pressed then the car will rotate clockwise. It is important to ensure that the car's position on screen is updated every time it is moved and if the car rotates a certain angle, then it will move forwards and backwards parallel to the direction of the car. I will also need to ensure that the car is unable to continue driving beyond the edge of the screen so that it remains on the race track.

The features that will be implemented in this stage are:

- A display window to contain the game
- A car which speeds up when up is pressed and slows down when down is pressed
- The car can also turn right when right is pressed and turn left when left is pressed
- Create boundaries at the edge of the display window so that the car cannot leave the screen

Class diagram:



This is a class diagram for the "Car" class which will contain all the necessary methods and attributes for the car in the game to be displayed on screen and move. The __init__ method is the constructor for the class in python which I learnt how to implement in this python tutorial: Python for Beginners – Full Course [Programming Tutorial]. The class will store the x position and the y position of the car as attributes which will correspond to the display window in pygame. It will also store the x and y speeds of the car which will be used to move the x and y position attributes. Finally, it will store the rotation of the car which will be used to display the image of the car at an angle and will be used outside of this class to determine how much of the overall velocity will be used as x speed and y speed.

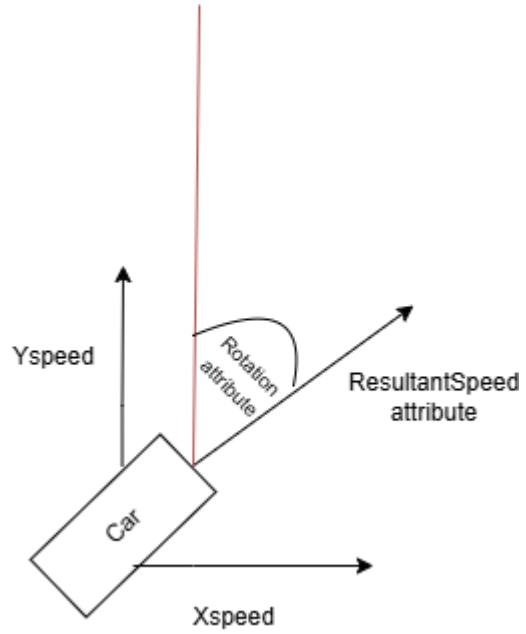
The __init__ method is the constructor of the class and it will initialise the x, y and resultant speeds as 0 and set the x and y positions as well as the rotation to arguments passed in. The get_ methods will return the corresponding attribute e.g. get_XPos() will return the value of the XPos attribute. The set_speed method will take in the resultant speed of the car and then calculate the X and Y speeds using the rotation of the car. The move_car method will add the XPos attribute to the XSpeed attribute and set this to the XPos and will do the same for the YPos and YSpeed which will happen every frame so that the car moves smoothly and naturally. If the XSpeed or YSpeed values are negative, then this will decrease the x and y positions of the car as well. The rotate_car method will take an angle in degrees as a parameter and will use the pygame "rotate()" function to rotate the image of the car by that amount. This value will also be added to the rotation attribute and can be negative.

I decided to use an object orientated class for the car as in the final stage of development a second car will be created to race against the first one and it is easy to instantiate two objects from the same class in python and both cars will require the exact same methods and attributes. The alternative option was to store all the attributes and methods from the car as variables and subroutines outside of a class and either create duplicate attributes for each car or store each attribute as an array of values which stores each value for each car however this would be more time consuming and difficult to work with and would introduce additional array handling or many repeated lines of code so creating an object orientated class was the best decision.

I decided to use snake case for the method names as this is the standard in python, and I decided to use Pascal case for the attributes so that the attributes were visually distinct from the methods meaning the code will be easier to read.

Algorithm design:

As the set_speed method involves a complicated calculation to calculate the X and Y speeds from the rotation and the resultant speed, I decided to design the method in detail before coding.



I created a diagram to visualize how some of the different attributes stored in the car class will act in the game.

The ResultantSpeed attribute will be directly affected by the inputs from the player or the neural network. This is because if there is a “W” input then the resultant speed attribute will increase and if there is an “S” input then it will decrease. The rotation attribute will store the angle that the car is rotated at clockwise which is shown as the angle from the red line on the diagram. As the car can only be moved pixel values in the X and Y directions, trigonometry will be used to calculate the XSpeed and YSpeed that will update the X and Y positions of the car which are stored as XPos and YPos.

I listed the calculations required to find the X and Y speeds in the table below which I deduced using my diagram and trigonometry:

Rotation	YSpeed calculation	XSpeed calculation
Between 0 and 90	sin(90-Rotation) * ResultantSpeed	cos(90-Rotation) * ResultantSpeed
Between 90 and 180	sin(Rotation - 90) * ResultantSpeed	cos(Rotation - 90) * ResultantSpeed
Between 180 and 270	sin(270-Rotation) * ResultantSpeed	cos(270-Rotation) * ResultantSpeed
Between 270 and 360	sin(Rotation-270) * ResultantSpeed	cos(Rotation-270) * ResultantSpeed

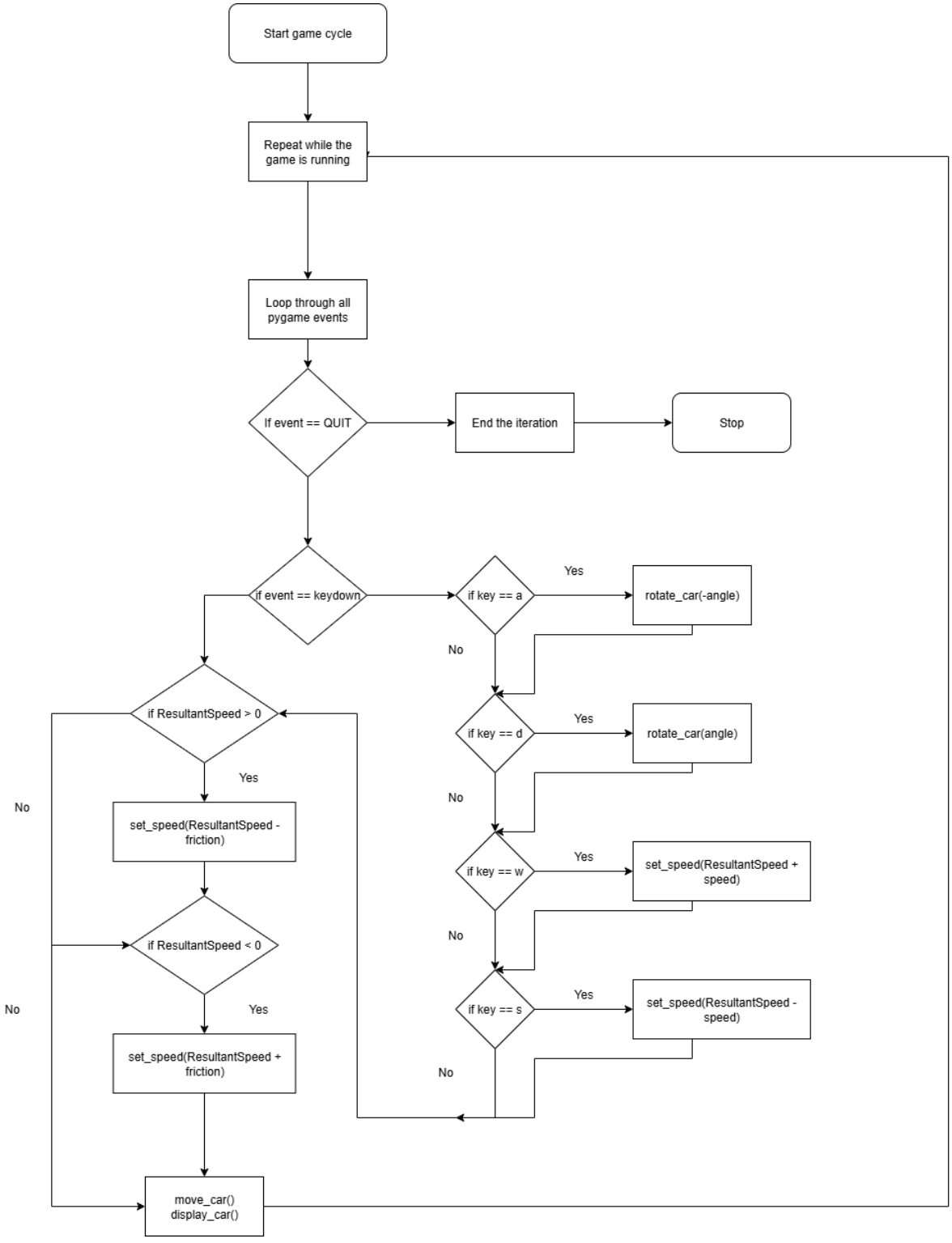
Here is the final pseudocode I created using this for the set_speed method:

```
FUNCTION set_speed(ResultantSpeed)
    this.ResultantSpeed = ResultantSpeed
    IF 0 ≤ Rotation AND Rotation < 90 THEN
        XSpeed = cos(90-Rotation) * ResultantSpeed
        YSpeed = sin(90-Rotation) * ResultantSpeed
    ELSE IF 90 ≤ Rotation AND Rotation < 180 THEN
        XSpeed = cos(Rotation - 90) * ResultantSpeed
        YSpeed = sin(Rotation - 90) * ResultantSpeed
    ELSE IF 180 ≤ Rotation AND Rotation < 270 THEN
        XSpeed = cos(270-Rotation) * ResultantSpeed
        YSpeed = sin(270-Rotation) * ResultantSpeed
    ELSE THEN
        XSpeed = cos(270-Rotation) * ResultantSpeed
```

```
            YSpeed = sin(270-Rotation) * ResultantSpeed
        ENDIF
END FUNCTION
```

In the beginning of the set_speed method, I update the ResultantSpeed attribute inside of the car class into the new resultant speed which is passed into the method as a parameter. I decided to use if statements instead of a switch statement as python does not have inbuilt switch case functionality so when I develop this method in python I will need to use if statements. Here, I used the table that I created to determine which trigonometric function multiplied by the resultant speed should be used for the X speeds and Y speeds for different angles.

The flowchart for the “game cycle” or “game loop”:



As this game will continuously run until the user decides to quit the program, a loop will repeatedly iterate so that the program cannot end which I learn how to do from the pygame tutorial. Inside of the loop contains the operations which must be performed every frame while the program is running for stage 1. Here, the program loops through and checks every event which occurred since the last frame in the program. If this event was the user attempting to quit, then the iteration will end and pygame will quit. If the event was one of either w,a,s or d being pressed then a certain operation occurs

because of it. If the user pressed A, then the rotate_car method is called from the car class which will rotate the car anticlockwise. If the user pressed D, then the rotate_car method will be called to rotate the car clockwise. If W was pressed then the set_speed method will be called to increase the resultant speed of the car. If S was pressed, then the set_speed method will be called to decrease the resultant speed of the car. Also each frame, if the resultant speed of the car is positive then a friction value will be taken away from it and if the resultant speed is negative then a friction value will be added to it to take the resultant speed closer to 0. Finally, the move_car and display_car methods are called to move the car depending on its speed and to display the car on screen.

Data dictionary:

Here is the data dictionary for variables which are not encapsulated inside of classes for this stage. The validation column refers to any validation which must be done inside of the code (mostly when user input is involved) and does not refer to if a particular constant must be kept in a certain range since a constant will never be modified regardless. Note that I modified the data dictionary throughout Stage 1 as new variables/attributes were needed.

Name	Data type	Description	Validation
screen	Surface (pygame)	This is the display window which will display the entire program to the user.	None
CarImage	Surface (pygame)	This is the image of the car which will be stored and used by the DisplayImage. The CarImage should never be modified in the program and should reflect the original car image.	Must store an image file
DisplayCarImage	Surface (pygame)	This is the image of the car which will be displayed to the user and will be modified by transformations such as rotations.	Must store an image file
running	Boolean	This is a boolean value which will be true when the program is running	None

		continuously and will be false when the user is quitting the program.	
IsGoingUp	Boolean	Stores whether the user is holding down the W key or not	None
IsGoingDown	Boolean	Stores whether the user is holding down the S key or not	None
IsTurningLeft	Boolean	Stores whether the user is holding down the A key or not	None
IsTurningRight	Boolean	Stores whether the user is holding down the D key or not	None

Here is the data dictionary for attributes which are encapsulated inside of classes for this stage.

Name	Data type	Description	Validation	In class
XPos	int	The x position of the car in pixels	Must be between 0 and the maximum horizontal pixel value of the screen	Car
YPos	int	The y position of the car in pixels	Must be between 0 and the maximum vertical pixel value of the screen	Car
XSpeed	int	The x speed of the car which is how much the XPos will be added to each frame.	None	Car
YSpeed	int	The y speed of the car which is how much the	None	Car

		YPos will be added to each frame.		
Rotation	int	The clockwise angle which the car is rotated at	Must be between 0 and 360	Car
ResultantSpeed	float	The overall speed of the car which is affected by user or neural network inputs. This speed is then resolved into x and y speeds.	None	Car

Programming:

Key for programming:

Black text will be used when creating new features.

Red text will be used when I have discovered an error.

Green text will be used when I am fixing an error.

After completing the iterative design for stage 1, the programming of the stage can now begin.

```
import pygame

pygame.init() #Initialises pygame so its functionality can be used

screen = pygame.display.set_mode((800, 600)) #Creates a display window with 800 horizontal pixels and 600 vertical pixels


running = True
while running: #Infinite loop to prevent the display window from closing until the user decides to
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

pygame.quit()

```

Initially, I set up a display window using pygame. The “screen” variable holds the display window created using pygame which currently has 800 horizontal pixels and 600 vertical pixels; however I will likely change this value after implementing the car and

determining which screen size is most suitable to work with the car. The while loop ensures that the program does not end without the user choosing to quit as the player would have a better experience if they can choose to quit after running the program for any amount of time. Line 10 loops through all the pygame events which occurred in the last frame such as player inputs and line 11 and 12 decide that if the event is the user pressing the quit button on the display window, then the program will end.

```
 8   class Car:
 9
10       def __init__(self,XPos,YPos,Rotation):
11           self.XPos = XPos
12           self.YPos = YPos
13           self.Rotation = Rotation
14           self.XSpeed = 0
15           self.YSpeed = 0
16           self.ResultantSpeed = 0
17           pass
18
19
20
```

Next, I began creating the car class. I chose to implement this early in the stage as most of the features of this stage depend heavily on the car class's methods and attributes. Here, I created the constructor for the car class where the initial XPos, YPos and Rotation of the car are set to passed in parameters and all of the speeds of the car are initialised to 0. I chose to initialise the attributes in this way as when the car(s) are initially placed on the racetrack, they may begin in different positions and at different rotations depending on which track was selected. However, at the beginning of each game the car's speeds will always be 0 so this can be determined.

```
    #Getters and setters
    def get_XPos(self): #Getter for the X position of the car
        return XPos

    def get_YPos(self): #Getter for the Y position of the car
        return YPos

    def get_XSpeed(self): #Getter for the X speed of the car
        return XSpeed

    def get_YSpeed(self): #Getter for the Y speed of the car
        return YSpeed

    def get_ResultantSpeed(self): #Getter for the resultant speed of the car
        return ResultantSpeed

    def get_Rotation(self): #Getter for the rotation of the car
        return Rotation

```

Next, I created all of the getters for the class which were explained in the class diagram.

```
    def set_speed(self,ResultantSpeed): #This method takes in a new resultant speed as a parameter and updates the ResultantSpeed attribute and then calulates the correct
        self.ResultantSpeed = ResultantSpeed
        if self.Rotation < 90:
            self.XSpeed = np.cos(90-self.Rotation) * ResultantSpeed
            self.YSpeed = np.sin(90-self.Rotation) * ResultantSpeed

        elif self.Rotation < 180:
            self.XSpeed = np.cos(self.Rotation-90) * ResultantSpeed
            self.YSpeed = np.sin(self.Rotation-90) * ResultantSpeed

        elif self.Rotation < 270:
            self.XSpeed = np.cos(270-self.Rotation) * ResultantSpeed
            self.YSpeed = np.sin(270-self.Rotation) * ResultantSpeed

        else:
            self.XSpeed = np.cos(self.Rotation-270) * ResultantSpeed
            self.YSpeed = np.sin(self.Rotation-270) * ResultantSpeed
```

Here, I created the set_speed method which follows the previous design. The if statements from the pseudocode could be simplified by using elifs because for the range 0≤Rotation<90, The rotation will be validated to always be positive so the 0 is not necessary and for the range 90≤Rotation<180, as the range 0 to 90 has already been checked only the rotation being less than 180 needed to be specified. Inside each if statement follows the pseudocode for the method where the correct trigonometric functions are applied to calculate the X and Y speeds from the rotation and the resultant speed.

```
57
58      def move_car(self):
59          self.XPos += self.XSpeed
60          self.YPos += self.YSpeed
61
62      def rotate_car(self,angle):
63          self.Rotation += angle
64          if Rotation >360:
65              Rotation -= 360
66          pygame.transform.rotate("Car temp",angle * -1)
67
68      def display_car(self):
69          screen.blit("Car temp",(self.XPos,self.YPos))
70
```

Next I created the move_car, rotate_car and display_car methods. The move_car method will be called once every frame and it will add the X and Y speeds to the X and Y positions of the car to move them. The rotate_car method will add an angle which is passed in as a parameter to the Rotation attribute. It will then rotate the image of the car by that angle multiplied by -1 as the pygame rotate function rotates the image anticlockwise whereas my rotation attribute stores the clockwise rotation. Finally, I created the display_car procedure will which display the image of the car in the current X and Y positions. I created a temporary image of the car named car temp which is a pink rectangle which will be used to test that the code involving the car works correctly and then more accurate car images will be used in the future when I implement the final images to be used in the game. This ensures that in the early stages of development, all of my focus can be spent on ensuring that the program functions correctly which will speed up the development overall instead of splitting my focus over both functionality and aesthetics.

```
screen = pygame.display.set_mode((800, 600)) #Creates a display window with 800 horizontal pixels and 600 vertical pixels
Car1 = Car(100,100,0)


running = True
while running: #Infinite loop to prevent the display window from closing until the user decides to

    for event in pygame.event.get(): #event handling
        if event.type == pygame.QUIT:
            running = False
        if event.type == pygame.KEYDOWN: # This means any key has been pressed
            if event.key == pygame.K_a: #This means it was the a key
                Car1.rotate_car(-2)

            if event.key == pygame.K_d: #This means it was the d key
                Car1.rotate_car(2)

            if event.key == pygame.K_w: #This means it was the w key
                Car1.set_speed(Car1.get_ResultantSpeed() + 2)

            if event.key == pygame.K_s: #This means it was the s key
                Car1.set_speed(Car1.get_ResultantSpeed() - 2)


        #end event handling
    if Car1.get_ResultantSpeed() > 0:
        Car1.set_speed(Car1.get_ResultantSpeed() - 1)
    elif Car1.get_ResultantSpeed() < 0:
        Car1.set_speed(Car1.get_ResultantSpeed() + 1)


    screen.fill((0,0,0))
    Car1.move_car()
    Car1.display_car()
    pygame.display.update()
```

Next, I followed the flowchart for this stage to get user input which was included inside the while loop which repeats until the user quits. Firstly, a new car object called Car1 is instantiated at 100 pixels horizontal and 100 pixels vertical. Next, if the player presses the a key, then the car will rotate anticlockwise by 2 and if the player presses the d key then the car will rotate clockwise by 2. I chose the value 2 temporarily but when running the code, I will determine the correct value to rotate the car by. If the player presses the w key, the car's set_speed method will set the resultant speed 2 greater than it currently is. If the player presses the s key, the car's set_speed method will set the resultant speed 2 less than it currently is. Again, I picked the 2 value randomly and this will be updated after testing to ensure that the movement of the car feels as natural as possible as this is important to stakeholders. After the events are handled, a frictional force will be applied to the car where if the car's resultant speed is positive then it will be reduced by 1, and if it is negative then it will increase by 1 which ensures that the car will slow down when the player stops pressing any keys.

After implementing the code, I attempted to run the program, however I was met with this error message:

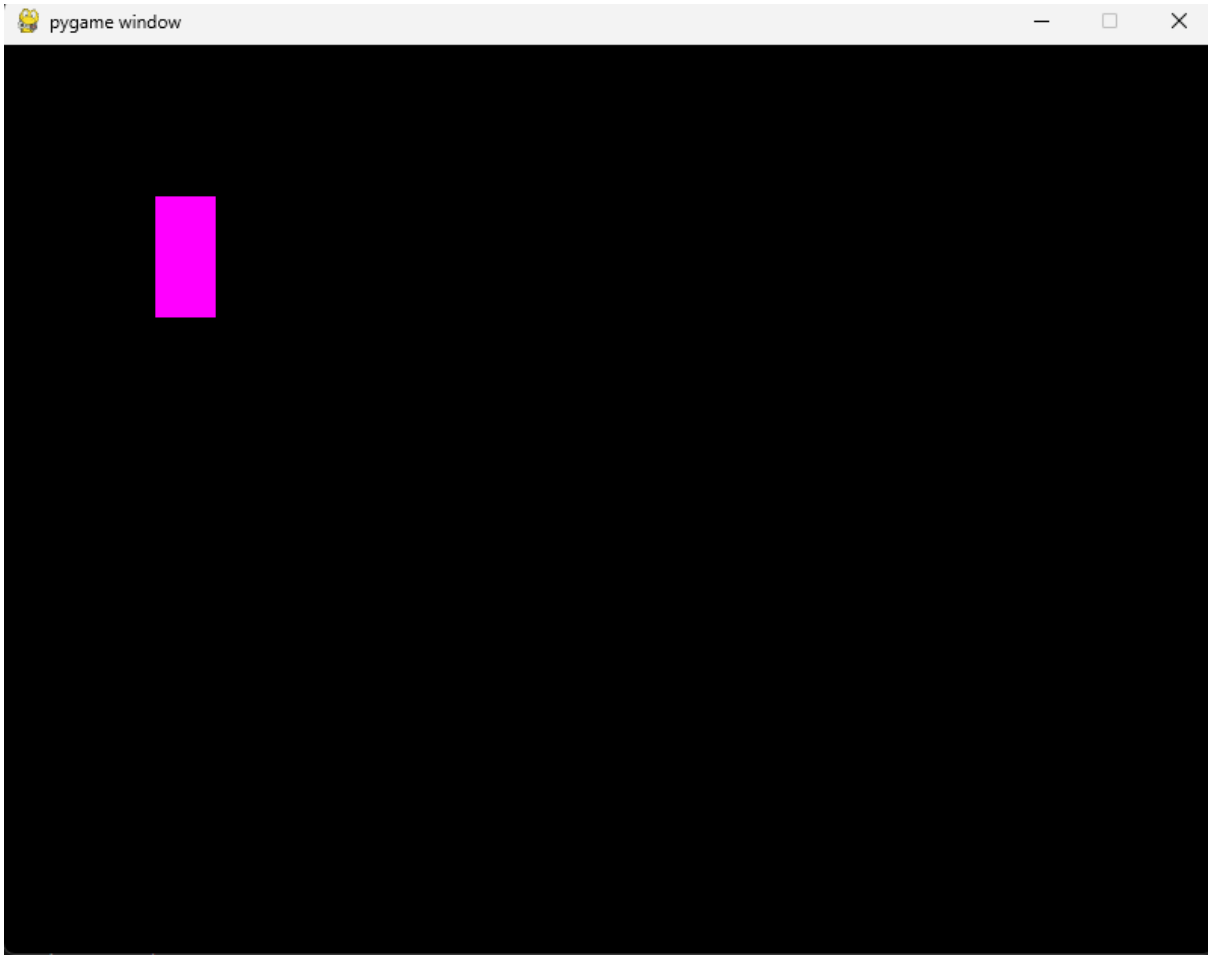
```
    screen.blit("Car temp",(self.XPos,self.YPos))
TypeError: argument 1 must be pygame.surface.Surface, not str
```

I then realised than “Car temp” is the name of the image file I created for the car however I should have specified that the file name is “Car temp.png” instead. To fix this I created a variable to store the image using pygame’s surface data type and I referenced the variable name instead of the image name as follows:

```
    CarImage = pygame.image.load('Car temp.png')
```

```
def display_car(self):
    screen.blit(self.CarImage1,(self.XPos,self.YPos))
```

Next, the code executed, and a display window was created which contained the correct car image at the correct place on the screen:



However, I noticed two errors with the handling of player inputs. Firstly, when pressing the a or d keys, the image of the car did not rotate. Secondly, when w or s is pressed, the car did move correctly however it did not move continuously as the key is held down as I planned instead it moved the instant the key is pressed and then no more as the key is held down.

To fix the first issue, I looked again at the pygame documentation for rotating images (pygame.transform — pygame v2.6.0 documentation). I then realised that the rotate function does not apply the rotation to the image itself but instead returns a rotated image. To fix this, I modified the rotate_car method as follows:

```
def rotate_car(self,angle):
    self.Rotation += angle
    if self.Rotation >360:
        self.Rotation -= 360
    CarImage = pygame.transform.rotate(CarImage,angle * -1)
```

Where I set the CarImage equal to the rotation. However, when I ran this code and pressed a to rotate the car I was met with this error:

```
    CarImage = pygame.transform.rotate(CarImage,angle * -1)
                                       ^^^^^^^^
UnboundLocalError: cannot access local variable 'CarImage' where it is not associated with a value
```

I noticed that CarImage was described as a local variable to the rotate_car method when it is a global variable. This meant that instead of the method accessing the real CarImage, it instead attempted to work with its own version. To fix this I passed the CarImage into the rotate_car method as a parameter.

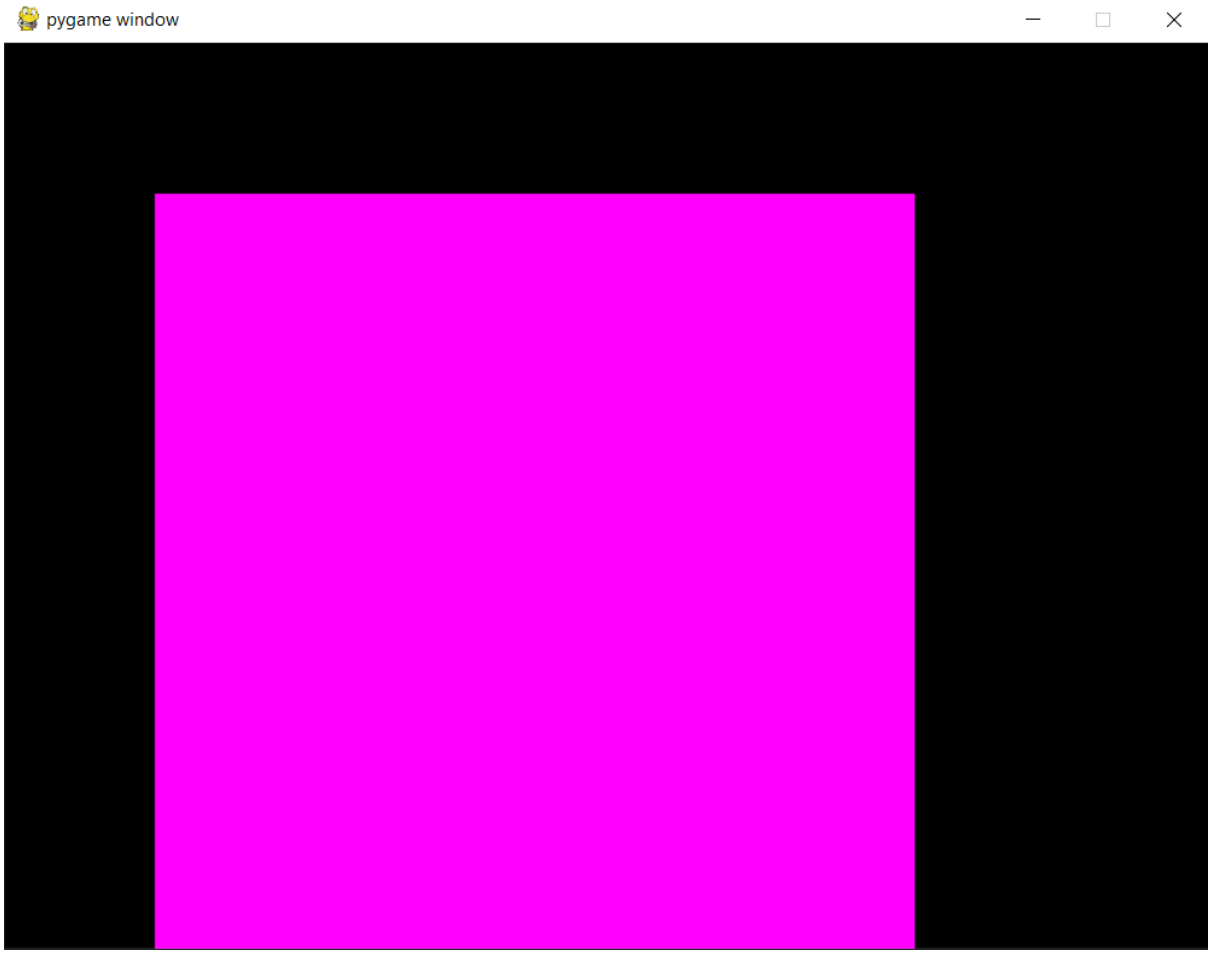
```
62      def rotate_car(self,angle, theCarImage):
63          self.Rotation += angle
64          if self.Rotation >360:
65              self.Rotation -= 360
66          theCarImage = pygame.transform.rotate(theCarImage,angle * -1)
67          return theCarImage
68
```

As the passed in car image is now local to the method, it has to be returned as well.

```
if event.type == pygame.KEYDOWN: # This means any key has been pressed
    if event.key == pygame.K_a: #This means it was the a key
        CarImage = Car1.rotate_car(-2, CarImage)

    if event.key == pygame.K_d: #This means it was the d key
        CarImage = Car1.rotate_car(2, CarImage)
```

I also changed the player input handling so that the result of the rotate_car method was set to the CarImage. However, instead of the image rotating it grew every time either a or d was pressed. After reading the pygame documentation further, I realised that what was happening was that when the image was being rotated, the image was padded larger to hold the new size however the new pixels which are introduced are unable to be transparent as the image does not have pixel alphas, so the new pixels matched the colour of the top-left pixel which was pink. This meant that instead of the image rotating, it was padded larger and then given additional pink pixels to fill the space. As

this is done to the same image, every time a or d was pressed it grew even larger. Here is the result of pressing a and d many times:



To fix this, Firstly I decided to fix the issue where each time the image is padded, the padded image is reused and padded again.

```
5   CarImage = pygame.image.load('Car temp.png')
6   DisplayCarImage = CarImage
```

```
89              if event.key == pygame.K_a: #This means it was the a key
90                  DisplayCarImage = Car1.rotate_car(-2, CarImage)
91
92              if event.key == pygame.K_d: #This means it was the d key
93                  DisplayCarImage = Car1.rotate_car(2, CarImage)
94
```

```
70      def display_car(self):
71          screen.blit(DisplayCarImage,(self.XPos,self.YPos))
72
```

```
62      def rotate_car(self,angle, theCarImage):
63          self.Rotation += angle
64          if self.Rotation >360:
65              self.Rotation -= 360
66          theCarImage = pygame.transform.rotate(theCarImage,(self.Rotation) * -1)
67          return theCarImage
```

I created the variable DisplayCarImage which will be the image which is displayed. Initially DisplayCarImage is the same as the CarImage however if the car is rotated then the original CarImage is rotated and set equal to DisplayCarImage. This means that the original car image is never modified so the image padding which occurs will only happen a single time before the image is displayed. In the rotate_car method, I changed the angle the image is rotated by from the angle passed in as a parameter to the entire rotation variable. This is because the image which is passed into this method is CarImage which is not rotated at all which means the entire rotation must be applied to get the new image.

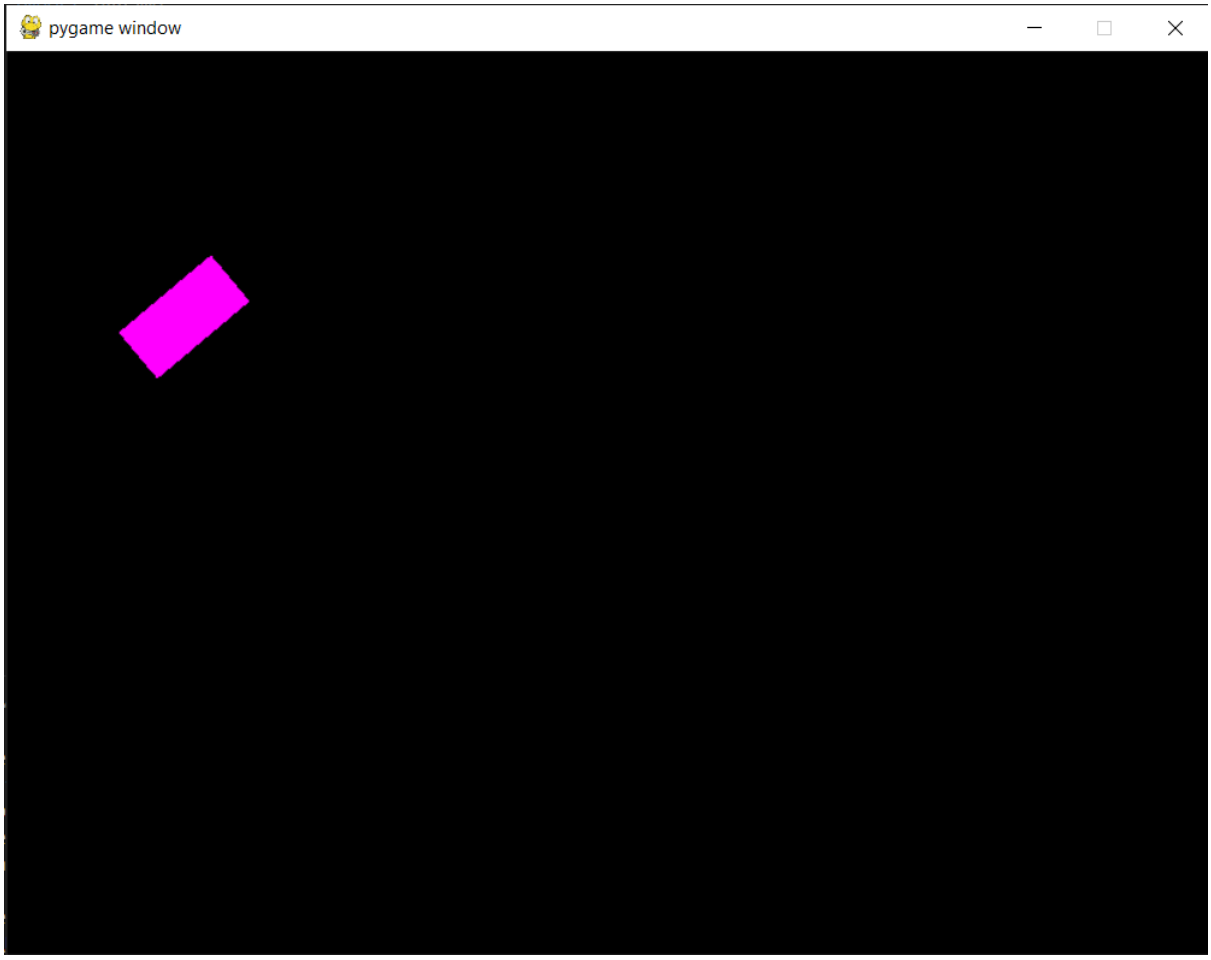
Next, I read the pygame documentation (pygame.Surface — pygame v2.6.0 documentation) to search for a way to add pixel alphas to an image so the padded pixels would be transparent. To do this, the method pygame.Surface.convert_alpha() can be used. To implement this, I wrote the following code:

```
4   pygame.init() #Initialises pygame so its functionality can be used
5   CarImage = pygame.image.load('Car temp.png') #Sets the Surface object CarImage equal to Car temp.png
6   CarImage = pygame.Surface.convert_alpha(CarImage) #Converts that image so that it can contain pixel alphas
7   DisplayCarImage = CarImage
8   """ Class definitions"""
9
```

This meant that the CarImage and DisplayCarImage now supports pixel alphas meaning that the padded pixels would be transparent. However when running the code I encountered this error message:

```
    CarIMAGE = pygame.Surface.convert_alpha(CarImage)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
pygame.error: No video mode has been set
```

I realised that no video mode has been set because I was executing this before the display window had been created, so I moved the line to create the display window before the CarImage is set. After doing this and running the program, the car image rotated correctly with no visible padded pixels:



Next, I fixed the issue where the Car only moved immediately as the key is pressed and not continuously. To do this I created the variables “IsGoingUp”, “IsGoingDown”, “IsTurningLeft” and “IsTurningRight” which will store whether the car is moving up, moving down, turning left or turning right. Then when the corresponding key is pressed for each movement, the variable will be set to true and when the key is let go of (which can be checked using the KEYUP event in pygame) the variable will be set to false. Then every time the while running loop (the game loop) iterates, it will do the corresponding car method if the variable is set to true. Here is the code I wrote to do this:

```
84  running = True
85  IsGoingUp = False
86  IsGoingDown = False
87  IsTurningLeft = False
88  IsTurningRight = False
89  while running: #Infinite loop to prevent the display window from closing until the user decides to
```

```
for event in pygame.event.get(): #event handling
    if event.type == pygame.QUIT:
        running = False
    if event.type == pygame.KEYDOWN: # This means any key has been PRESSED
        if event.key == pygame.K_a: #This means it was the a key
            IsTurningLeft = True

        if event.key == pygame.K_d: #This means it was the d key
            IsTurningRight = True

        if event.key == pygame.K_w: #This means it was the w key
            IsGoingUp = True

        if event.key == pygame.K_s: #This means it was the s key
            IsGoingDown = True
```

```
if event.type == pygame.KEYUP: # This means any key has been LET GO OF
    if event.key == pygame.K_a: #This means it was the a key
        IsTurningLeft = False

    if event.key == pygame.K_d: #This means it was the d key
        IsTurningRight = False

    if event.key == pygame.K_w: #This means it was the w key
        IsGoingUp = False

    if event.key == pygame.K_s: #This means it was the s key
        IsGoingDown = False
```

Running the code now, meant that the car rotated and moved correctly when each button was pressed and it stopped moving when the key was let go of. However, the car moved and rotated much too fast with the current values. After experimenting with different values, I decided upon 0.001 for the amount the speed is increased while pressing the W or S keys, 0.0005 for the friction and 0.2 for the angle the image is rotated by while pressing the A or D keys.

However, when rotating the car, the direction that the car travelled in did not match the direction which the car was facing. To identify the issue, I printed the value stored in the rotation attribute every time the car is displayed:

```
-2.0
-1.5
-1.0
-0.5
0.0
0.5
1.0
1.5
2.0
```

Here, I noticed that the rotation attribute could go into negative values. This was because when I created the rotate_car method I accidently left out the validation for when the rotation is less than 0. To fix this I corrected the code as follows:

```
64      def rotate_car(self,angle, theCarImage):
65          self.Rotation += angle
66          if self.Rotation >360:
67              self.Rotation -= 360
68          if self.Rotation <0:
69              self.Rotation += 360
70          theCarImage = pygame.transform.rotate(theCarImage,(self.Rotation) * -1)
71          return theCarImage
```

Now the validation for the Rotation is correct. However, the movement of the car still did not match the car image. After reading the numpy documentation for its sin and cos functions (numpy.sin — NumPy v2.3 Manual), (numpy.cos — NumPy v2.3 Manual), I realised that those functions treat the argument passed into them as if they were in radians however, I was using degrees in the set_speed method. To fix this, I used the numpy documentation again to find the radians function which converts a degree values into radians (numpy.radians — NumPy v2.3 Manual).

```
41      def set_speed(self,ResultantSpeed): #This method takes in a new resultant speed as a parameter and updates 
42          self.ResultantSpeed = ResultantSpeed
43          if self.Rotation < 90:
44              self.XSpeed = np.cos(np.radians(90-self.Rotation)) * ResultantSpeed
45              self.YSpeed = np.sin(np.radians(90-self.Rotation)) * ResultantSpeed
46
47          elif self.Rotation < 180:
48              self.XSpeed = np.cos(np.radians(self.Rotation-90)) * ResultantSpeed
49              self.YSpeed = np.sin(np.radians(self.Rotation-90)) * ResultantSpeed
50
51          elif self.Rotation < 270:
52              self.XSpeed = np.cos(np.radians(270-self.Rotation)) * ResultantSpeed
53              self.YSpeed = np.sin(np.radians(270-self.Rotation)) * ResultantSpeed
54
55          else:
56              self.XSpeed = np.cos(np.radians(self.Rotation-270)) * ResultantSpeed
57              self.YSpeed = np.sin(np.radians(self.Rotation-270)) * ResultantSpeed
58
```

This is the updated set_speed function with the added conversion to radians inside each sin or cos function. However, when attempting to drive the car with this function, some of the x and y directions seemed to be backwards from what they should be. I

realised that this was because my calculations were finding the positive magnitude of the X and Y speeds, however for the car to be able to turn left or downwards, some of the speeds would be negative. To fix this I changed my code as shown below:

```
def set_speed(self,ResultantSpeed): #This method takes in a new resultant speed as a parameter and updates the ResultantSpeed attribu
    self.ResultantSpeed = ResultantSpeed
    if self.Rotation < 90:
        self.XSpeed = np.cos(np.radians(90-self.Rotation)) * ResultantSpeed
        self.YSpeed = np.sin(np.radians(90-self.Rotation)) * ResultantSpeed

    elif self.Rotation < 180:
        self.XSpeed = np.cos(np.radians(self.Rotation-90)) * ResultantSpeed
        self.YSpeed = -1 * np.sin(np.radians(self.Rotation-90)) * ResultantSpeed

    elif self.Rotation < 270:
        self.XSpeed = -1 * np.cos(np.radians(270-self.Rotation)) * ResultantSpeed
        self.YSpeed = -1 * np.sin(np.radians(270-self.Rotation)) * ResultantSpeed

    else:
        self.XSpeed = -1 * np.cos(np.radians(self.Rotation-270)) * ResultantSpeed
        self.YSpeed = np.sin(np.radians(self.Rotation-270)) * ResultantSpeed
```

This means that for the YSpeed, whenever the angle is between 90 and 270 degrees, the result should be multiplied by -1 as this means the car is facing downwards. For the XSpeed, whenever the angle is between 180 and 360 then the result should be multiplied by -1 as the car is now facing left and the XSpeed is positive in the right direction.

```
def move_car(self):
    self.XPos += self.XSpeed
    self.YPos -= self.YSpeed
```

I updated the move_car method as well. I made it so the Y speed is subtracted from the Yposition of the car. This is because in pygame the y values increase as you go down however I am used to the y values increasing as you go up. Working with YSpeed values which treated YSpeed as if positive Y speed was directed upwards made developing the program easier as I personally am used to positive Y values being directed upwards. This meant that I could develop the trigonometry in the set_speed method without being potentially confused by the Y values being the opposite of what I would expect and simply subtracting this in the move_car method is a very simple thing to include.

Now when I attempted to drive the car, it travelled correctly in the direction which the car was facing.

The final feature to implement as part of this stage is making it so that the car cannot go past the edges of the display window. The XPos and YPos attributes which are passed into pygame's blit function refer to the middle of the car image. This means that if I

restricted the values of XPos and YPos to the dimensions of the screen then half of the car image would still be outside of the screen. This means that when the XPos and YPos are updated in the move_car function, they must be kept between 0 and the edge of the display window – half of the image's size.

```
61      def move_car(self):
62          self.XPos += self.XSpeed #update X and Y values
63          self.YPos -= self.YSpeed
64
65          #Adding boundaries to the screen
66          if self.XPos > screen.get_width() - CarImage.get_width():
67              self.XPos = screen.get_width() - CarImage.get_width()
68          if self.YPos > screen.get_height() - CarImage.get_height():
69              self.YPos = screen.get_height() - CarImage.get_height()
70
71          if self.XPos<0:
72              self.XPos = 0
73          if self.YPos<0:
74              self.YPos = 0
```

Here is the validation of the X and Y positions. When I ran the program, The top and left edges of the screen correctly prevented the car from passing past them and the car displayed correctly. However, with the right and bottom edges of the screen, the car was also prevented from moving past them but at certain rotations of the car, the car appears to be either beyond or behind the edge of the screen when it is prevented from moving. The functionality itself of this feature has been successfully implemented and as this is a small graphical bug, I have decided to not fix this issue until stage 6 of development where the GUI and graphical design of the program will be enhanced in greater detail. This is because it is highly important that the functionality of the program is developed as soon as possible in the development cycle as that is the main focus of the project.

Now that stage 1 has been developed, I will work through the test plan for the stage which I created in the design section. This will allow me to evaluate if this stage has met the success criteria which has been set out for it. **SEE EVIDENCE OF TESTS BELOW**.

Test number	Required inputs	Test description	Success criteria	Pass/fail
1	Run the program.	When the program is run, see if a display window appears.	A display window will appear.	Pass
2	Run the program.	When the program is run, see if a car is displayed on the display window.	A car will appear inside of the display window.	Pass

3	Run the program and press W.	When W is pressed, see if the car moves forwards.	The car will move forward when W is pressed.	Pass
4	Run the program and press S.	When S is pressed, see if the car moves backwards.	The car will move backwards when W is pressed.	Pass
5	Run the program and press A.	When A is pressed, the car's image will rotate anticlockwise.	The car will turn anticlockwise when A is pressed.	Pass
6	Run the program and press D.	When D is pressed, the car's image will rotate clockwise.	The car will turn clockwise when D is pressed.	Pass
7	Run the program and press and hold A. Then press W.	Pressing A will cause the car to rotate anticlockwise and then pressing W will cause the car to move forwards. The movement of the car should be going forward in the same direction that the car is facing after it is turned.	The car moves in the same direction the car is facing when W is pressed.	Pass
8	Run the program and press and hold A. Then press S.	Pressing A will cause the car to rotate anticlockwise and then pressing S will cause the car to move backwards. The movement of the car should be going backwards in the opposite direction that the car is facing after it is turned.	The car moves in the opposite direction the car is facing when S is pressed.	Pass
9	Run the program and press W then let go.	When W is pressed, the car will move forward. After W is let go, the force of friction should be applied to it.	After W is let go, the car should gradually slow down and eventually stop.	Pass
10	Run the program and hold W until the car reaches the edge of the display window.	The car should be stopped when it reaches the edge	The car will stop moving as it reaches the edge	Pass (functionality wise however it

		of the display window so it cannot leave.	of the display window.	is not perfect graphically)

Test 1:

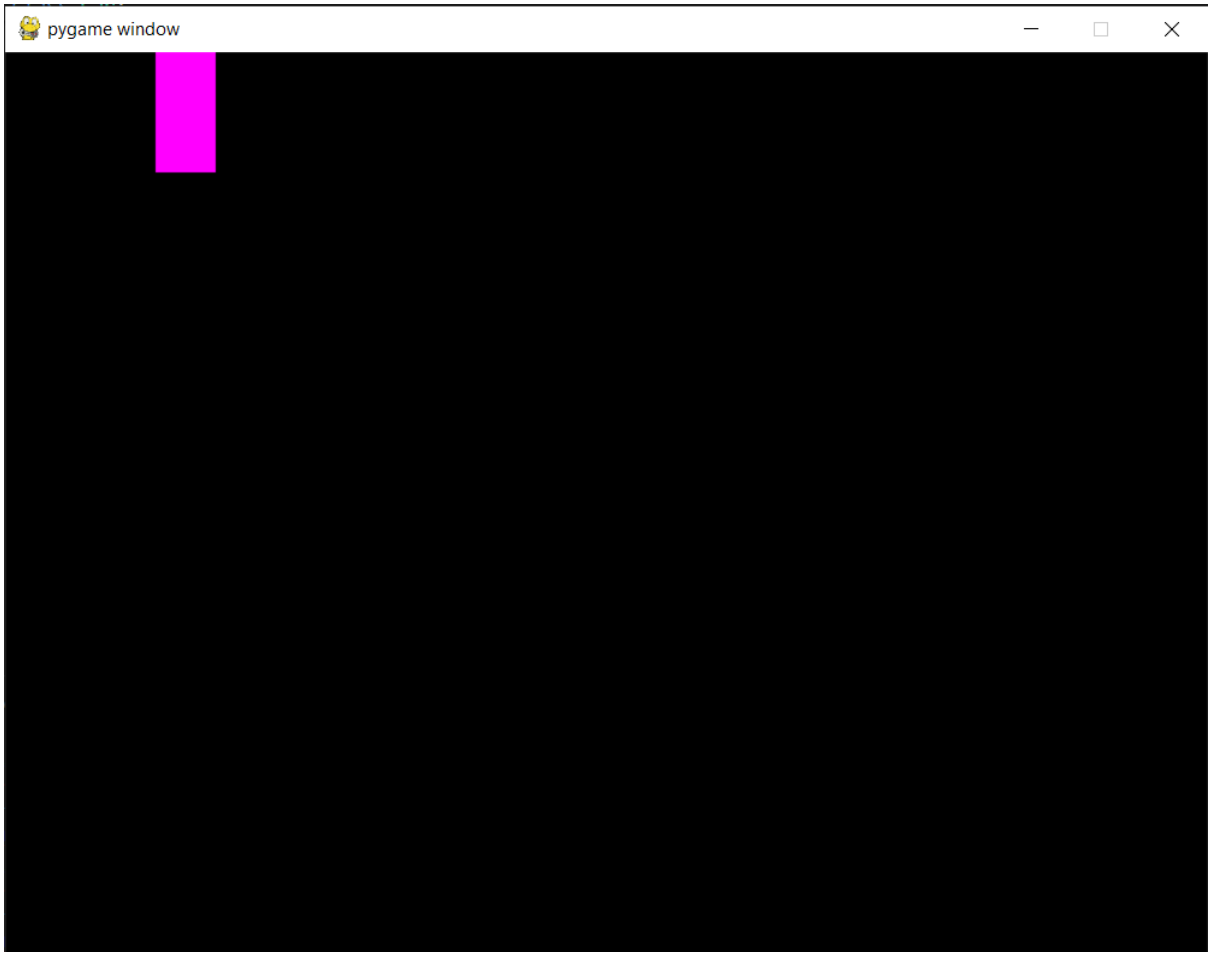
Here I need to run the program to see if a display window appears. After running the program, a display window did appear so this test is a pass.

Test 2:

Here, I need to run the program to see if the image of a car is displayed. After running the program, the image of the car was displayed so this test is a pass.

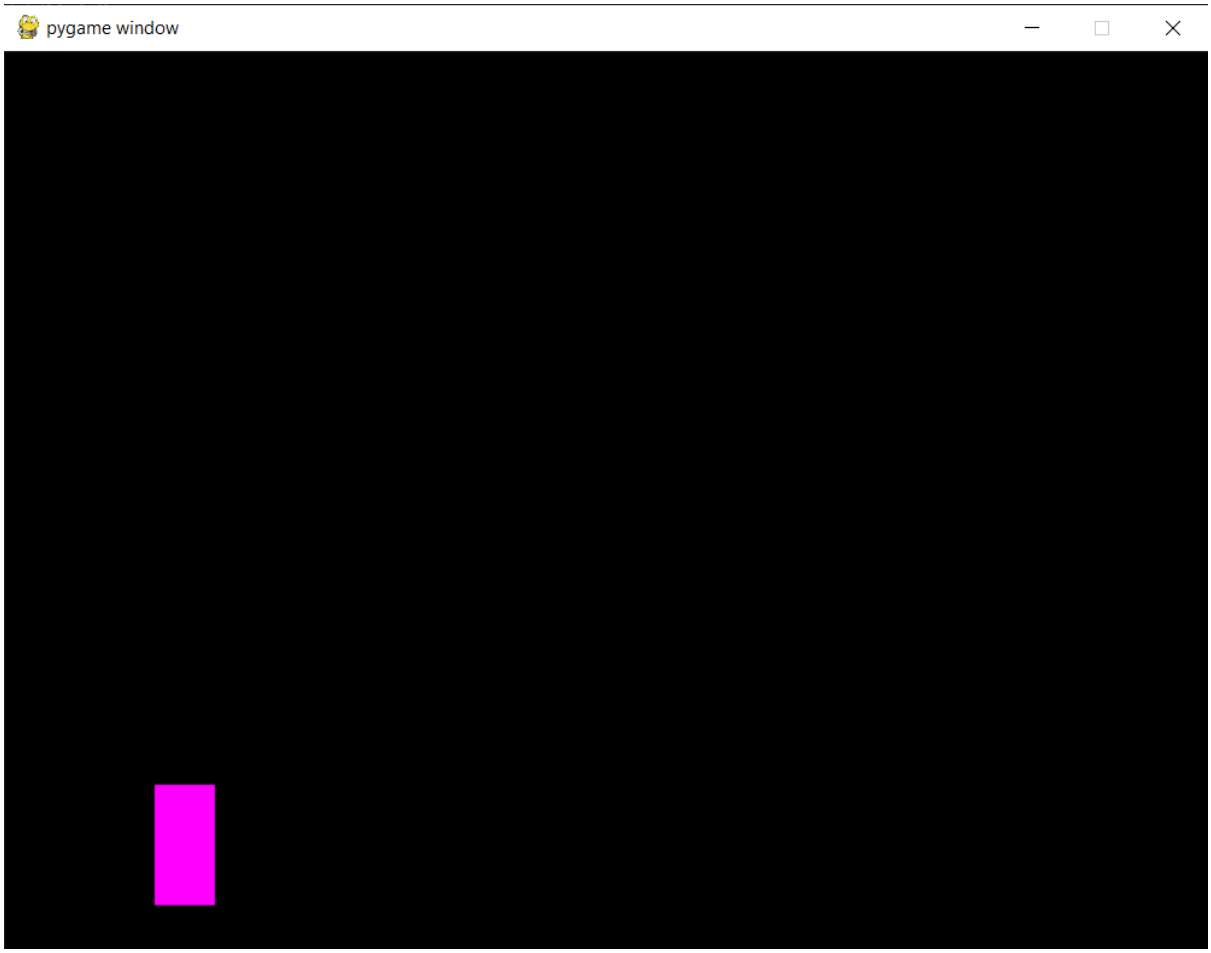
Test 3:

Here, I need to run the program and press W to see if the car moves forwards. After running the program and pressing W, the car did move forwards after I pressed W so this test is a pass.



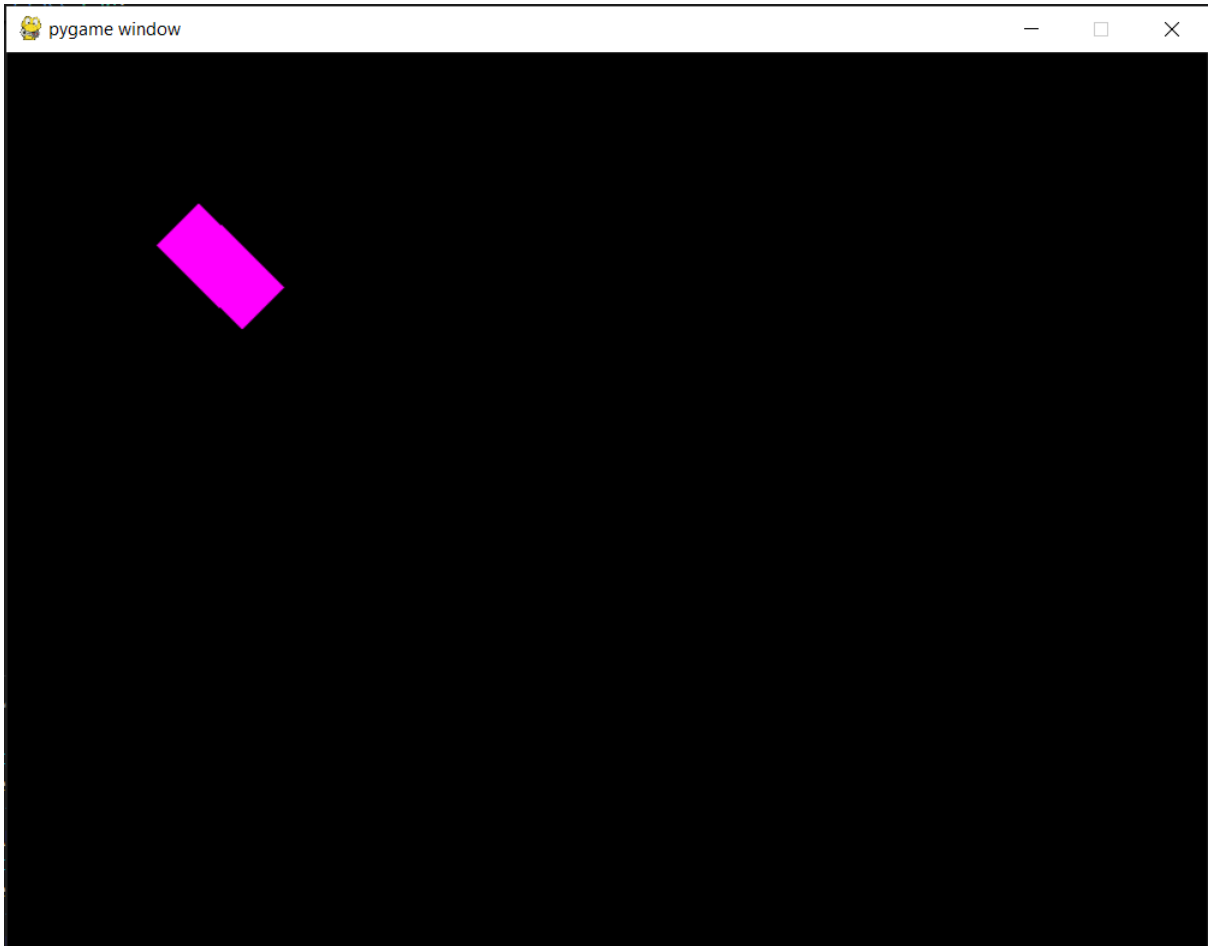
Test 4:

Here, I need to run the program and press S to see if the car moves backwards. After running the program and pressing S, the car did move backwards after I pressed S so this test is a pass.



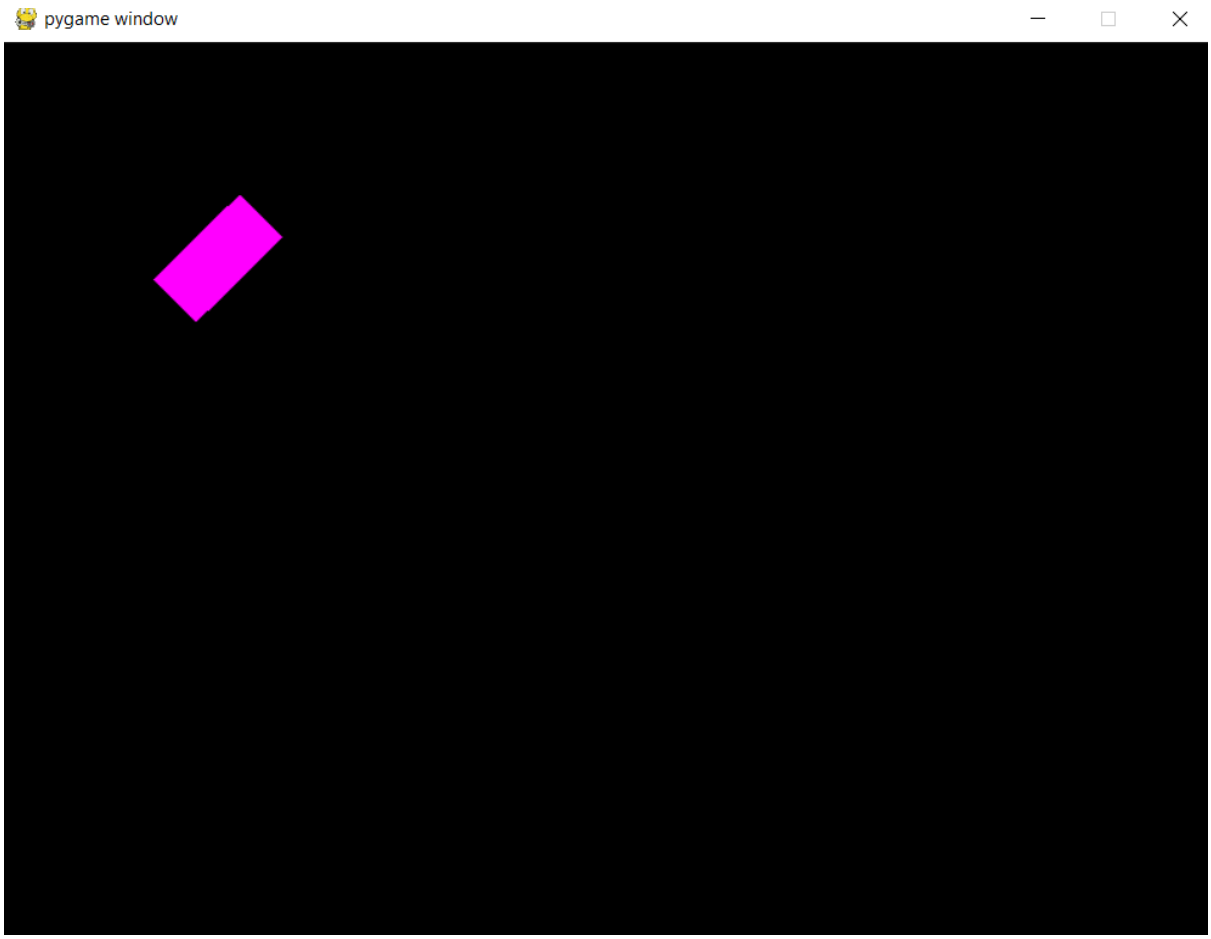
Test 5:

Here, I need to run the program and press A to see if the car turns left. After running the program and pressing A, the car did turn left only after I pressed A so this test is a pass.



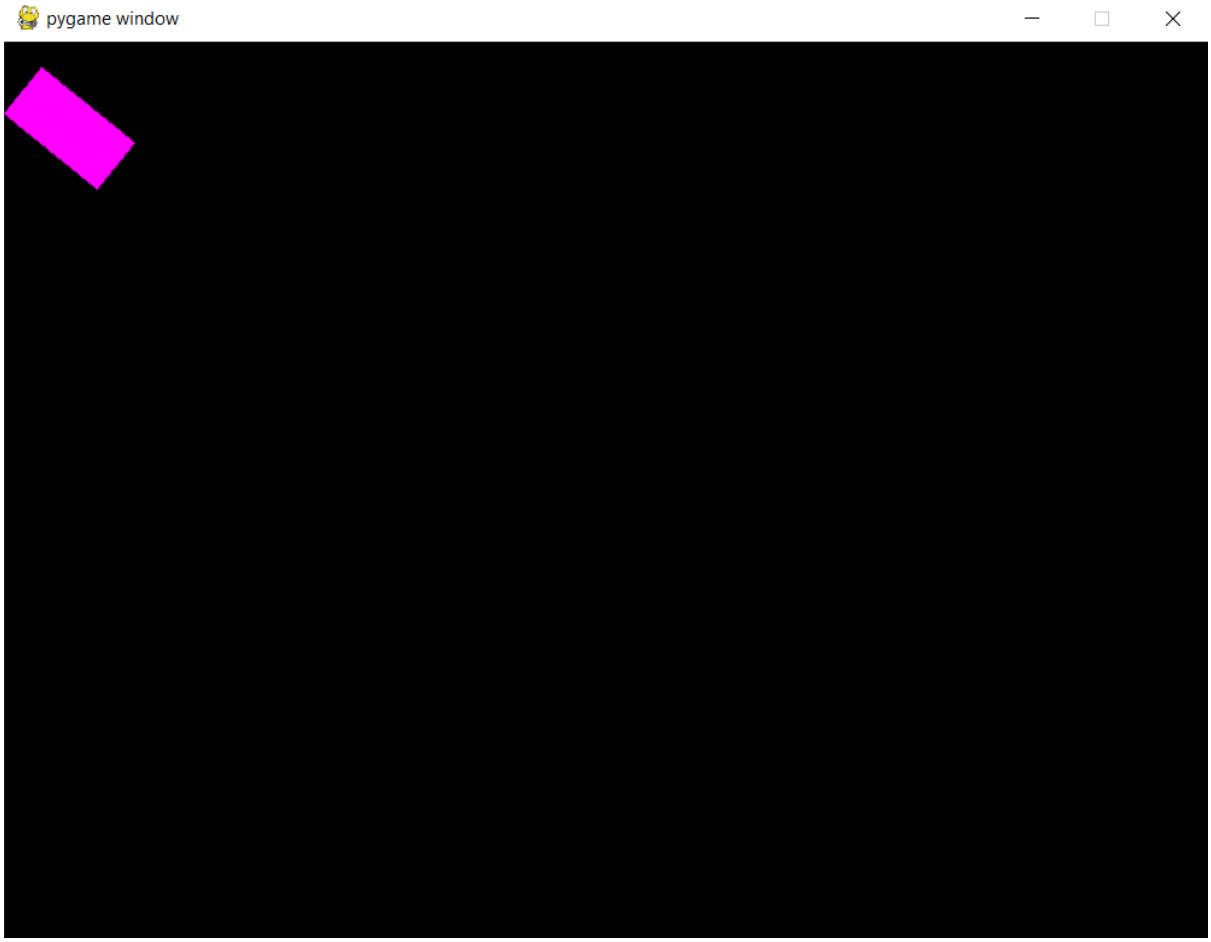
Test 6:

Here, I need to run the program and press D to see if the car turns right. After running the program and pressing D, the car did turn right only after I pressed D so this test is a pass.



Test 7:

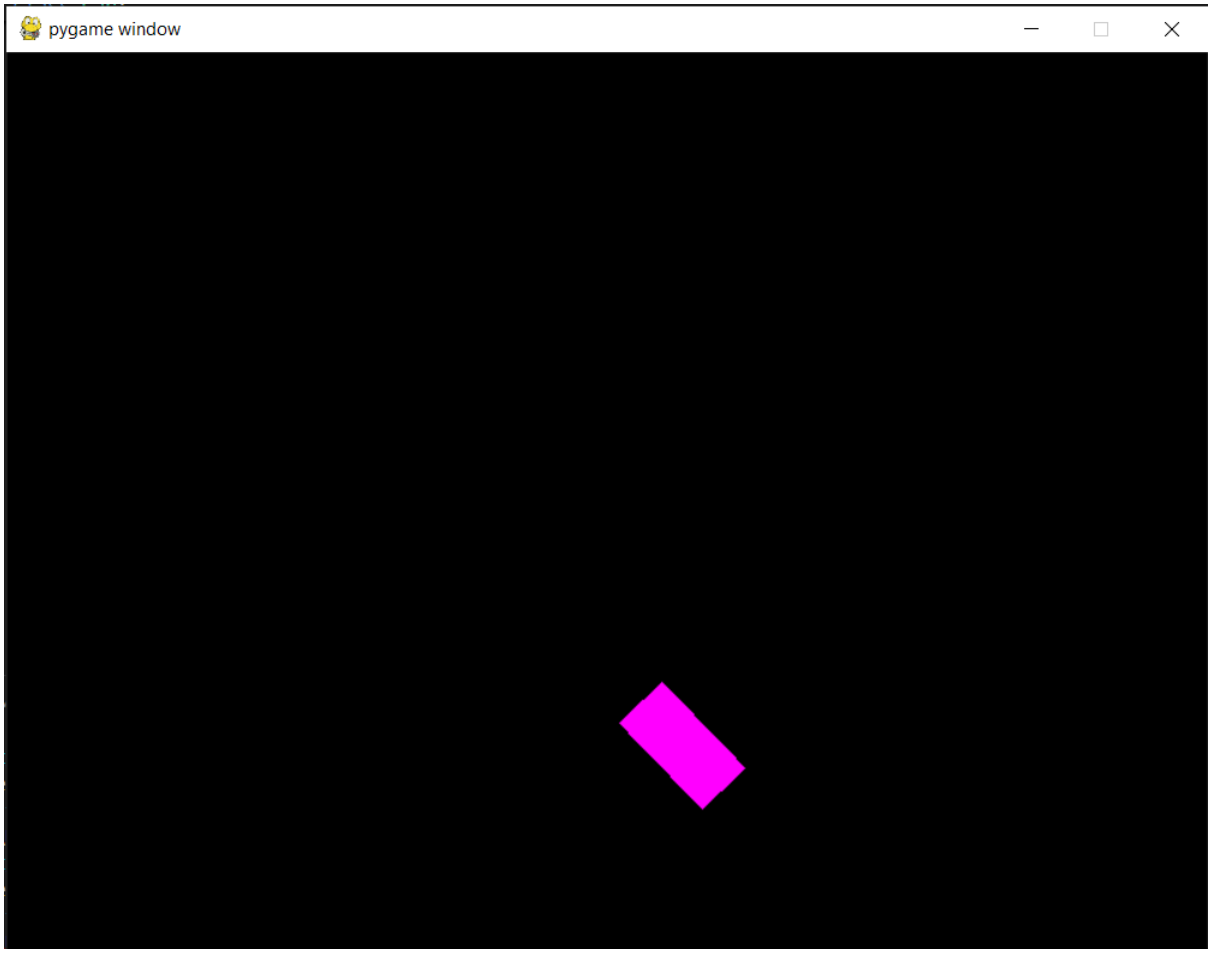
Here, I need to run the program and press and hold A. Then press W to see if the car moves in the direction that the car is facing. After running the program and doing this test, the car did move in the direction it was facing so this test is a pass.



Test 8:

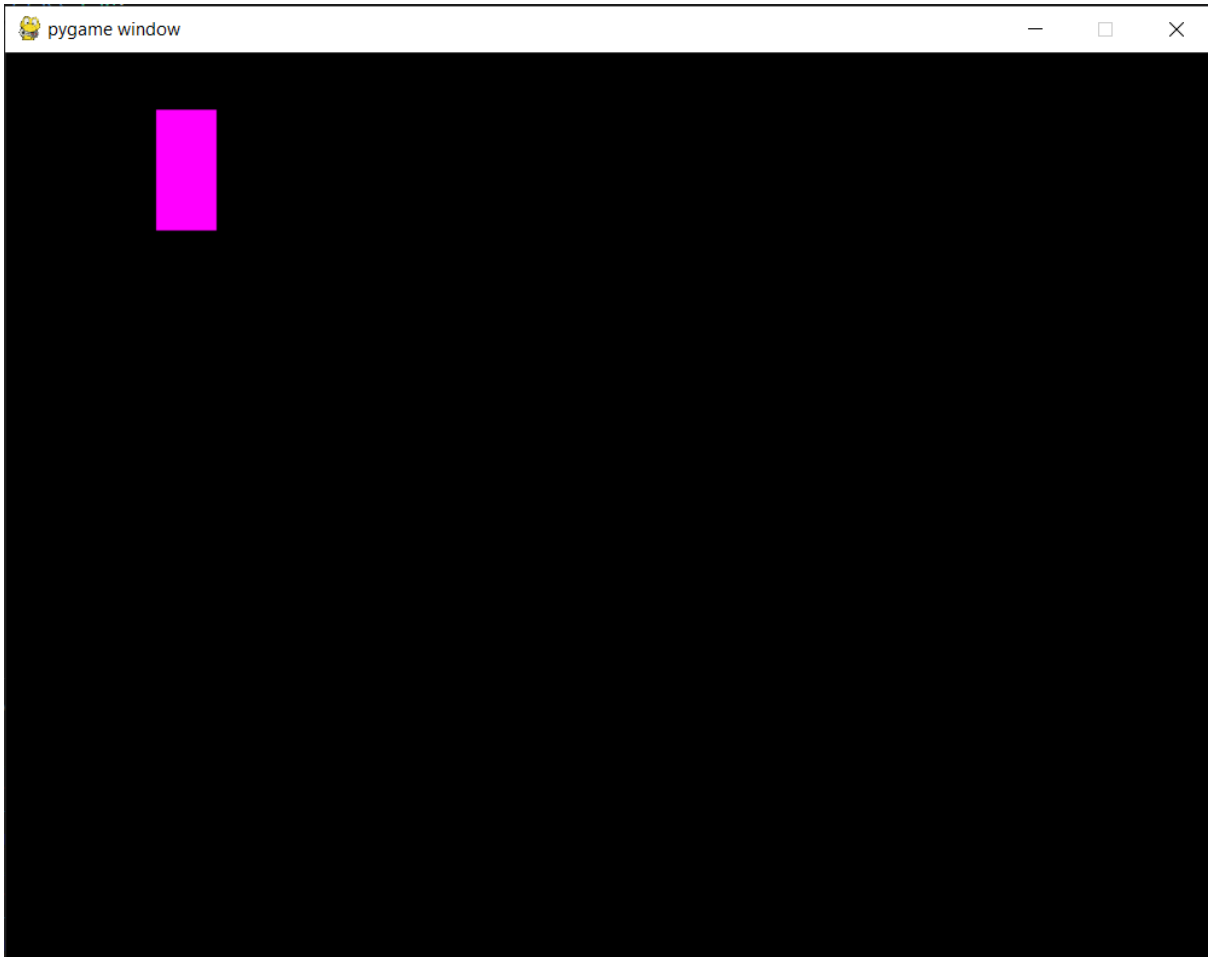
Here, I need to run the program and press and hold A. Then press S to see if the car moves in the opposite direction that the car is facing. After running the program and doing this test, the car did move in the opposite direction than it was facing so this test

is a pass.



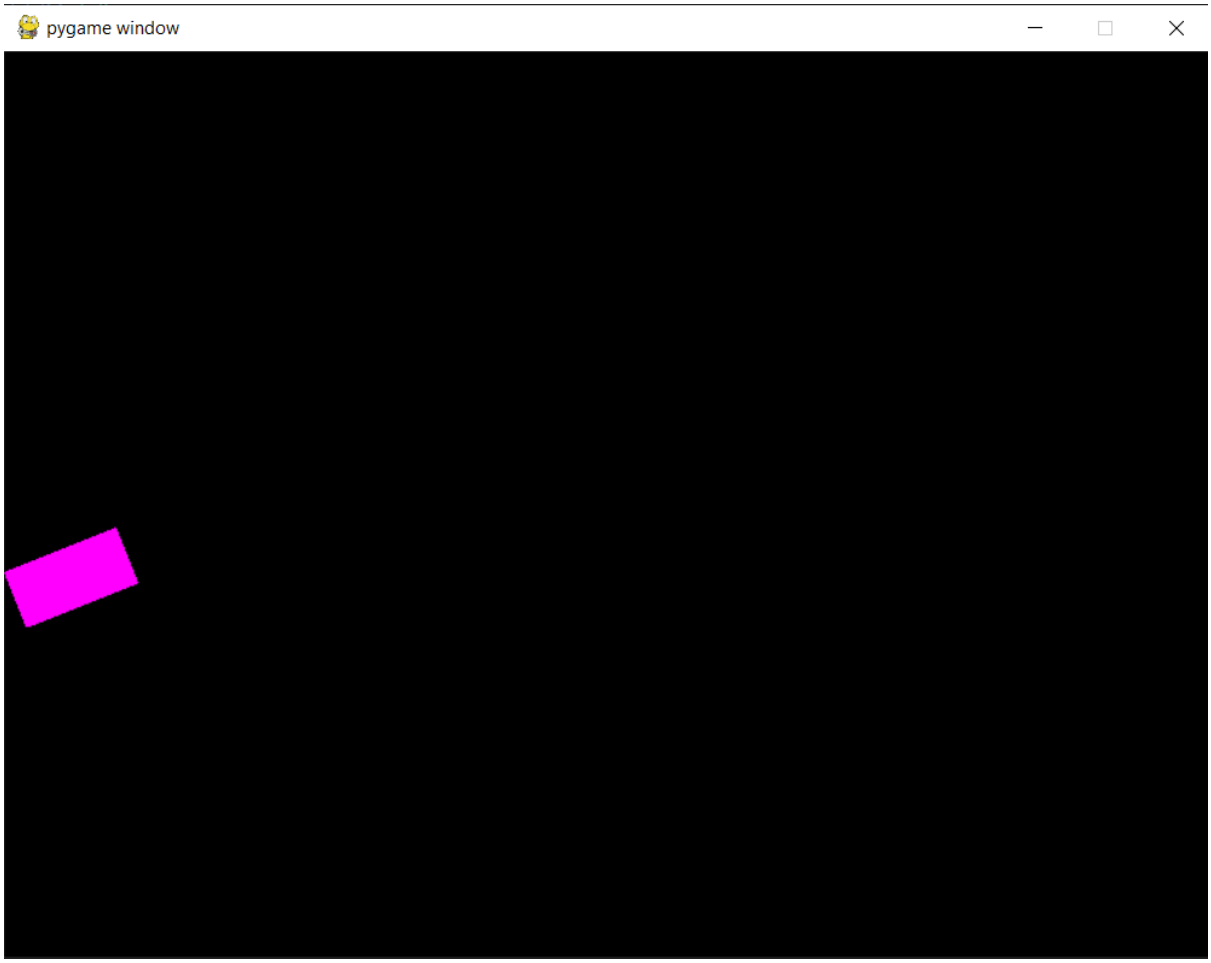
Test 9:

Here, I need to press and hold W and then let go to see if the frictional force slows down the car and then stops it from moving. After running the program and doing this test, after W was let go of the car slowed down and eventually stopped so this test is a pass.

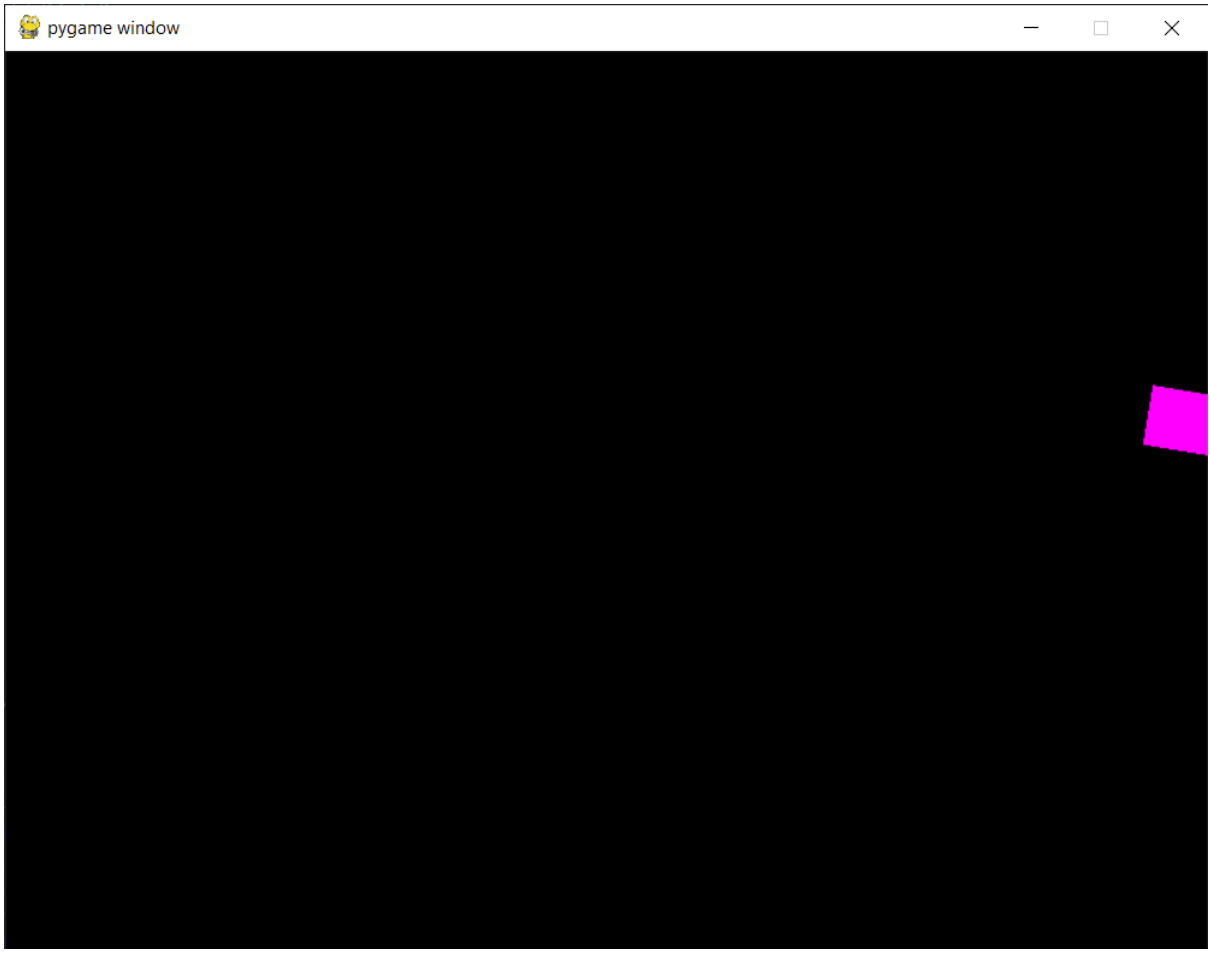


Test 10:

Here I need to press and hold W until the car reaches the edge of the display window. After running the program and completing this test, the car could not move past the edge of the display window, so this test is a pass. However, at the end of this stage, this only displays properly all the time on the top and left edges of the screen whereas the bottom and right edges of the screen display the car as stopping too far or too short of the edge of the window at certain rotations of the car. This test is still a pass as the functionality of this feature is correct to be used in the rest of the project, however fixing the issue in a later stage would make the program more appealing to users as the engagement of the program is highly important to ensure that this project is a success.



The image above is where the car cannot pass the edge of the screen and it is displayed correctly.



The image above is where the car still cannot pass the edge of the screen however it is not displayed correctly.

Stage 2- creating the racetrack:

Next, I will implement the racetrack which the car will race on. Firstly, the display window will contain a background image of an environment, and this will include a racetrack and a finish line which will all be displayed on screen. It must also be clear whether the car is driving on the track or off the track as this will affect the frictional forces. Next, the collisions between the car, the racetrack and finish line must be detected. This will mean that if the car is colliding with the racetrack, then it will have a lower frictional value and if the car is not colliding with the racetrack, it will have a higher frictional value. The collisions with the finish line will not be utilised in this stage, however it will be used heavily in the next stage where the racing gameplay is created which means that the collision detection must be developed first.

Features to implement in this stage:

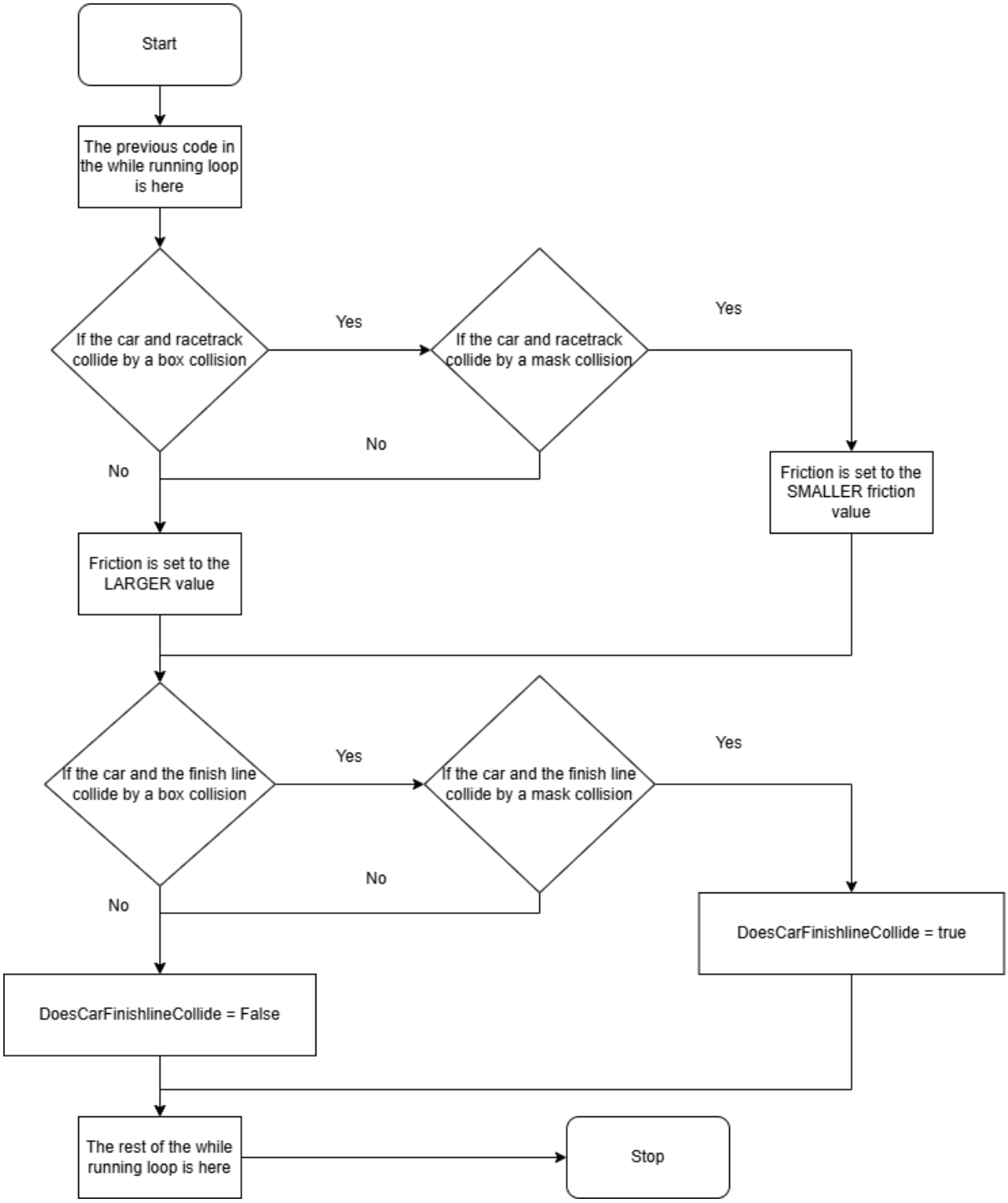
- The background of the game should indicate where on the screen is the track, off-track sections and the start/finish line
- The car should store whether it is currently on or off track and update the friction acting on the car due to that

Algorithm design:

The use of collisions between different objects is a very important element of this stage as collisions need to be checked between the car and the racetrack and the car and the finish line. After researching how to do this in pygame here: Collision Detection in PyGame - GeeksforGeeks and here: How to Use Pygame Masks for Pixel Perfect Collision - YouTube. I learnt here that there are two methods that can be used for collisions. Box collisions check if there is a collision between two surfaces by seeing if there is an overlap between the entire box containing the image. This is fast to do; however, box collisions still count collisions between transparent pixels as collisions which means that it will report some events as collisions when they do not appear as a collision to the user. This is very important for this project, as the image of the racetrack loops around, it will encapsulate an area of non-racetrack inside of it which will be transparent, but this area should not be reported as a collision with the racetrack and the car as it is an off-track area. The other method to check collisions between objects in pygame is by using masks. Here, masks of the images are created which are a special type of surface which only stores the information of which pixels are transparent and which ones are not. Next, the overlap method is used to determine whether any of the opaque pixels in the images overlap. This method of collision checking is slower to use than box collisions, however it is pixel perfect which means that is suitable to use when the collision checking needs to be completely accurate. Because of this, I have decided for my project that I will attempt to use box collisions where possible due to their speed which will help ensure that the program runs quickly, however I will use mask collisions where they are needed such as checking for collisions between the car and the racetrack.

This is the flowchart for the collision detection and updating of the friction value. I created this flowchart to visualise the general structure that the program will follow,

however it does not provide detail as to how the collisions systems will be implemented so I have also written pseudocode to plan how the different collision systems will work after it. The flowchart shows that inside of the “while running loop” inside of the main code which iterates repeatedly during the gameplay, the game will check if the car and the racetrack collide by a box collision check which is a less accurate than a mask collision check, but much faster method of collision detection. If they do not collide by a box collision, then this means that the car and the racetrack are not colliding so the friction value will be larger (as the car is now off the track). If they do collide by a box collision, then a more accurate mask collision check is used. If they do not collide by a mask collision, then the car is also off track. If they do collide by a mask collision however, then the car is colliding with the track so the friction value will be reduced as it is on the track. The same method is also used for the collision between the car and the finish line. The result of this is stored in the “DoesCarFinishlineCollide” Boolean variable which will be used later during later stages of development.



Here is the pseudocode for the collisions, I once again viewed the tutorial [How to Use Pygame Masks for Pixel Perfect Collision] to ensure that I used the correct pygame functions for collision detection.

```
class Car:

    PROCEDURE __init__(self,XPos,YPos,Rotation):
```

```
        self.CarImage = load('Car temp.png')
        self.DisplayCarImage = self.CarImage
        self.rect = self.DisplayCarImage.get_rect()
        self.mask = mask.from_surface(self.DisplayCarImage)

        END PROCEDURE
        … #the rest of the car class will continue
```

Firstly, the car class will be modified so that its initialiser contains the additional attributes “rect” and “mask” where rect will hold the rectangle of the image which will be used to determine if the car has had a box collision with another object and the mask will hold the mask of the image which will be used to determine if the car has had a mask collision with another object. The CarImage and DisplayCarImage variables which were previously not encapsulated in any classes and were global to the program will be changed so that they are encapsulated inside of the car class. This means that when multiple cars are created each one will have its own image so that each car can be displayed separately. Similar attributes will also be created for the Track class and the FinishLine class.

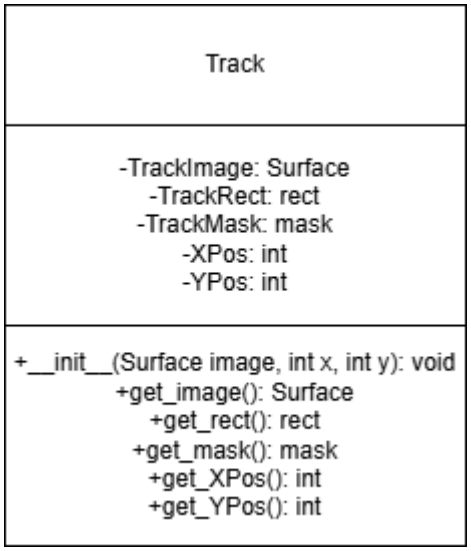
Here is pseudocode for the box and mask collisions between the car and the racetrack:

```
if sprite_collide(Track, Car, False):
     If sprite_collide(Track,Car,False,collide_mask):
          Friction = largerValue
else:
     Friction = smallerValue
end if
```

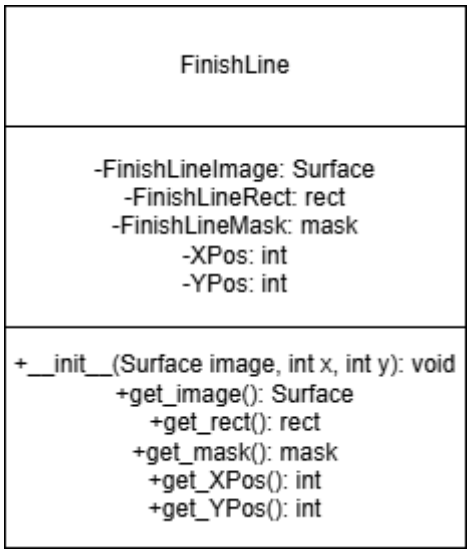
The sprite_collide function checks if the pixels of two sprites have overlapped using different collision methods. The first and second arguments are the objects which sprites are being compared, the third argument is whether the sprites are destroyed when they collide and the final argument is the method of collision detection used which has to be specified for mask collision, however if no final argument is given then it will default to box collision. In the psedocode above, firstly a box collision check is done between the car and the racetrack which is fast to do if there is no collision however if

there is a collision then a full mask check will ensure that the images are fully colliding. Similar code will also be created for the collision between the car and the finish line.

Class diagrams:



Here is the class diagram for the track class. The class will hold the image of the track, the rect object of the image, the mask object of the image and the x and y positions of the top left of the track. The __init__ method will assign the TrackImage, XPos and YPos attributes to passed in parameters. This will be especially useful when allowing the user to switch between multiple tracks as a different object can be created with a different image in a different position. The remaining getters return all of the corresponding attributes.



The class diagram for the FinishLine class is entirely the same however some of the attributes have been named differently for the finish line and when this class is used an image of a finish line will be passed into this class instead of one of a race track.

Here is the data dictionary for the variables which are not encapsulated in classes for this stage:

Name	Data type	Description	Validation
Friction	float	This is a float which will be subtracted from the car's resultant speed each iteration of the while running loop	None
Acceleration	float	This is a float which will be added to/subtracted from the car's resultant speed when W/S is pressed	None
RotationAmount	float	This is a float which will be added to/subtracted from the car's Rotation attribute when A/D is pressed	None
DoesCarFinishlineCollide	Boolean	This is a Boolean which will record if the car is currently colliding with the finish line or not	None
StartTime	float	This is the time read from the computer's clock taken when the last game loop started	None

NewTime	float	This is the time read from the computer's clock taken repeatedly at the start of the current game loop to compare with the StartTime	None
FrameRate	float	This is a constant value which is the time which must be waited between the last and the current game loop. A value of 1 is 1 second.	None (however this value should be constant)
CarGroup	Sprite Group	This is a container which holds the "spites" for all the instantiated car objects. This is so that certain pygame functions such as collision detection can be done easier and the sprite for each class includes its rect and mask objects.	None
TotalLaps	int	A player inputted value which stores the total number of laps which must be completed when playing to win.	Must be greater than 0 and less than or equal to 10
NormalFriction	float	A constant, frictional value which will be applied to the car's speed when driving on the track.	None (however must be kept constant)

OffTrackFriction	float	A constant, frictional value which will be applied to the car's speed when driving off the track.	None (however must be kept constant)

Here is the data dictionary for attributes which are encapsulated inside of classes for this stage.

Name	Data type	Description	Validation	In class
TrackImage	Surface	The image of the racetrack which will be displayed on screen.	Must store an image file	Track
rect	rect	A rectangle with the same dimensions as the track image which will be used for box collisions	None	Track
topleft	tuple	Contains the Xpos and YPos of the top left pixel of the track in a tuple so that it can be used by pygame functions	None	Track, rect
mask	mask	A mask with the same opaque pixels as the track image which will be used for mask collisions	None	Track
XPos	int	The X position of the top left pixel of the track	None	Track

YPos	int	The Y position of the top left pixel of the track	None	Track
FinishLineImage	Surface	The image of the FinishLine which will be displayed on screen.	Must store an image file	FinishLine
rect	rect	A rectangle with the same dimensions as the FinishLine image which will be used for box collisions	None	FinishLine
topleft	tuple	Contains the Xpos and YPos of the top left pixel of the finish line in a tuple so that it can be used by pygame functions	None	FinishLine, rect
mask	mask	A mask with the same opaque pixels as the FinishLine image which will be used for mask collisions	None	FinishLine
XPos	int	The X position of the top left pixel of the FinishLine	None	FinishLine
YPos	int	The Y position of the top left pixel of the FinishLine	None	FinishLine
CarImage	Surface	This is the image of the car which will be stored and used by the DisplayImage. The CarImage should never be modified in the program and should reflect	Must store an image file	Car

		the original car image.		
DisplayCarImage	Surface	This is the image of the car which will be displayed to the user and will be modified by transformations such as rotations.	Must store an image file	Car
rect	rect	A rectangle with the same dimensions as the Car image which will be used for box collisions	None	Car
topleft	tuple	Contains the Xpos and YPos of the top left pixel of the car in a tuple so that it can be used by pygame functions	None	Car, rect
mask	mask	A mask with the same opaque pixels as the car image which will be used for mask collisions	None	Car
LapCount	int	A variable which is used to store the number of laps which the car has completed	None	Car

Programming:

To begin stage 2, I decided to edit some of the values which I defined in stage 1 to make the existing program work better when introducing a racetrack on screen. Firstly, I

noticed that the car was able to rotate left and right even when the car was not moving and that the rotation of the car looked unnatural at different speeds. To fix this, when the rotate car function is called, I modified the angle parameter to be 0.2 or -0.2 multiplied by the resultant speed of the car. This means that the car will be unable to rotate when it is not moving which is accurate to cars in real life and the car will rotate more at higher speeds of the car which is also accurate to cars in real life. This will make the cars' movement appear more natural on screen which will support the engagement from the user.

```
112     if not IsTurningLeft or not IsTurningRight:
113         if IsTurningLeft:
114             DisplayCarImage = Car1.rotate_car(-0.2 *Car1.get_ResultantSpeed(), CarImage)
115         elif IsTurningRight:
116             DisplayCarImage = Car1.rotate_car(0.2* Car1.get_ResultantSpeed(), CarImage)
117 
118 
```

Next, I noticed that the amount of friction which is subtracted from the car's speed each frame is not assigned to a variable or constant. This makes the code less maintainable as instead of being able to modify a variable to update the friction, the place in code where the friction is used needs to be found. Additionally, in stage 2, the friction of the car will be different depending on if the car is on the track or not so having the amount of friction stored as a variable means that it can be easily updated.

```
99  #Definitions of global variables used in the game
100 running = True
101 IsGoingUp = False
102 IsGoingDown = False
103 IsTurningLeft = False
104 IsTurningRight = False
105 Friction = 0.0007
```

```
157     #Adding frictional forces to the car's speed
158     if Car1.get_ResultantSpeed() > 0:
159         Car1.set_speed(Car1.get_ResultantSpeed() - Friction)
160     elif Car1.get_ResultantSpeed() < 0:
161         Car1.set_speed(Car1.get_ResultantSpeed() + Friction)
162 
```

I repeated this process with the amount which the speed is increased by when the player presses W or S by creating the Acceleration variable and with the amount the car is rotated when the player presses A or D to increase the maintainability of the program further.

```
106 Acceleration = 0.001
107 RotationAmount = 0.2
108 while running: #Infinite loop to prevent the display window from closing until the user decides to
109
110     if not IsGoingUp or not IsGoingDown: #If both IsGoingUp and IsGoingDown are true, then the speed remains the same
111         if IsGoingUp:
112             Car1.set_speed(Car1.get_ResultantSpeed() + Acceleration)
113         elif IsGoingDown:
114             Car1.set_speed(Car1.get_ResultantSpeed() - Acceleration)
115
116     if not IsTurningLeft or not IsTurningRight:#If both IsTurningLeft and IsTurningRight are true, then the angle remains the same
117         if IsTurningLeft:
118             DisplayCarImage = Car1.rotate_car(-RotationAmount *Car1.get_ResultantSpeed(), CarImage)
119         elif IsTurningRight:
120             DisplayCarImage = Car1.rotate_car(RotationAmount *Car1.get_ResultantSpeed(), CarImage)
121
```

Next, I made it so that the friction which the speed of the car is decreased by is proportional to the resultant speed of the car. This made it so at higher speeds of the car, it accelerated less until the car's resultant speed no longer increased when pressing W or S. This is because eventually, the friction multiplied by the resultant speed became equal to the acceleration which means that there is now a maximum resultant speed. I chose to do this as in real life, an object moving at a higher velocity will be resisted by a stronger force which means that the car's acceleration will appear more natural which will make the game more engaging to its users. This also meant that when the car's resultant speed is less than 0, its friction is also subtracted instead of added as the friction is now multiplied by a negative number so there no longer needs to be an elif as shown in the image below:

```
159     #Adding frictional forces to the car's speed
160     if Car1.get_ResultantSpeed() != 0:
161         Car1.set_speed(Car1.get_ResultantSpeed() - Friction * Car1.get_ResultantSpeed())
162
```

Finally, I updated the code which updates the speed if the player presses W or S to increase/decrease the speed by a further value which is multiplied by the resultant speed. I did this so that the car would accelerate slower when it is at a lower speed and accelerate faster when it was at a higher speed. This meant that the car's movement also appeared more realistic since cars in real life accelerate slower when they are accelerating from being stationary than at higher speeds. I reached the value "0.00003" after testing the car's movement multiple times and deciding this was the most suitable value to use.

```
108 while running: #Infinite loop to prevent the display window from closing until the user decides to
109
110     if not IsGoingUp or not IsGoingDown: #If both IsGoingUp and IsGoingDown are true, then the speed remains the same
111         if IsGoingUp:
112             Car1.set_speed(Car1.get_ResultantSpeed() + Acceleration + 0.00003 * Car1.get_ResultantSpeed())
113         elif IsGoingDown:
114             Car1.set_speed(Car1.get_ResultantSpeed() - Acceleration - 0.00003 * Car1.get_ResultantSpeed())
115
```

Overall, this means that when holding W after the car is not moving, the car begins to accelerate slowly before this increases and then eventually, the car reached a maximum speed which it remains at even while holding W. This ensures that the car's movement is realistic to watch as well as enjoyable to control, as the car having a fixed maximum speed means that it will be consistent to control. Additionally, that means

that the neural network will be able to learn how to play the game better, as it will mostly only need to learn how to control the car at a consistent, maximum speed instead of many various speeds.

I then quickly created this temporary design of a racetrack to test the ability for it to be displayed and its collision detection. I also included some areas with thicker track and some areas with thinner track to compare which are the most enjoyable to drive in with the car.

Next, I updated the constructor for the class to match the new design.

```
10  class Car:
11
12      def __init__(self,XPos,YPos,Rotation,CarImage):
13          self.XPos = XPos
14          self.YPos = YPos
15          self.Rotation = Rotation
16          self.XSpeed = 0
17          self.YSpeed = 0
18          self.ResultantSpeed = 0
19          self.CarImage = CarImage
20          self.DisplayCarImage = self.CarImage
21          self.rect = self.DisplayCarImage.get_rect()
22          self.mask = pygame.mask.from_surface(self.DisplayCarImage)
23
24
25          pass
```

I also added the get_image, get_rect and get_mask info the car class:

```
45      def get_rect(self): #Getter for the rect of the car
46          return self.rect
47
48      def get_mask(self): #Getter for the mask of the car
49          return self.mask
50
51      def get_image(self): #Getter for the image of the car
52          return self.CarImage
```

I then created the Track and FinishLine classes. I followed the class diagrams here and created all of the attributes and methods which were previously defined. I also included a display_car method which was not included in the class diagrams as I realised that there needed to be a method to display the car on the screen.

```
106  class Track:
107      def __init__(self,image,x,y):
108          self.TrackImage = image
109          self.XPos = x
110          self.YPos = y
111          self.TrackRect = self.TrackImage.get_rect()
112          self.TrackMask = pygame.mask.from_surface(self.TrackImage)
113
114      def get_rect(self): #Getter for the rect of the track
115          return self.Trackrect
116
117      def get_mask(self): #Getter for the mask of the track
118          return self.Trackmask
119
120      def get_image(self): #Getter for the image of the track
121          return self.TrackImage
122
123      def get_XPos(self): #Getter for the mask of the track
124          return self.XPos
125
126      def get_YPos(self): #Getter for the image of the track
127          return self.YPos
128
129      def display_track(self): #Method to display the track to the screen
130          screen.blit(self.TrackImage,(self.XPos,self.YPos))
131
```

```
132  class FinishLine:
133      def __init__(self,image,x,y):
134          self.FinishLineImage = image
135          self.XPos = x
136          self.YPos = y
137          self.FinishLineRect = self.FinishLineImage.get_rect()
138          self.FinishLineMask = pygame.mask.from_surface(self.TFinishLineImage)
139
140      def get_rect(self): #Getter for the rect of the finish line
141          return self.FinishLinerect
142
143      def get_mask(self): #Getter for the mask of the finish line
144          return self.FinishLinemask
145
146      def get_image(self): #Getter for the image of the finish line
147          return self.FinishLineImage
148
149      def get_XPos(self): #Getter for the mask of the finish line
150          return self.XPos
151
152      def get_YPos(self): #Getter for the image of the finish line
153          return self.YPos
154
155      def display_FinishLine(self): #Method to display the finish line to the screen
156          screen.blit(self.FinishLineImage,(self.XPos,self.YPos))
```

I then instantiated the track and the car again before the while running loop:

```
163   """ End class definitions"""
164
165   Car1 = Car(500,350,0,CarImage)
166   Track1 = Track("TEMP racetrack.png", 100,100)
```

I then attempted to run the program; however, I was met with this error message:

```
    self.TrackRect = self.TrackImage.get_rect()
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'str' object has no attribute 'get_rect'
```

I then realised I had made a mistake in both the constructors for the Track and FinishLine classes. This is because I set the TrackImage attribute equal to the passed in argument however I passed in the string of the name of the image file instead of the pygame Surface object which holds the image. To fix this, I chose to convert the string of the name of the image file into a surface inside of the constructor. I chose to do this inside the constructor instead of inside the main body of code as creating another unnecessary image variable would make it harder to find the correct variable I want to find, and when creating multiple tracks later many variables would have to be created to temporarily hold the image of the racetracks before they are passed into the constructor.

```
def __init__(self,image,x,y):
    self.TrackImage = pygame.image.load(image)
```

```
def __init__(self,image,x,y):
    self.FinishLineImage = pygame.image.load(image)
```

When attempting to run the program again, I was met with this error message:

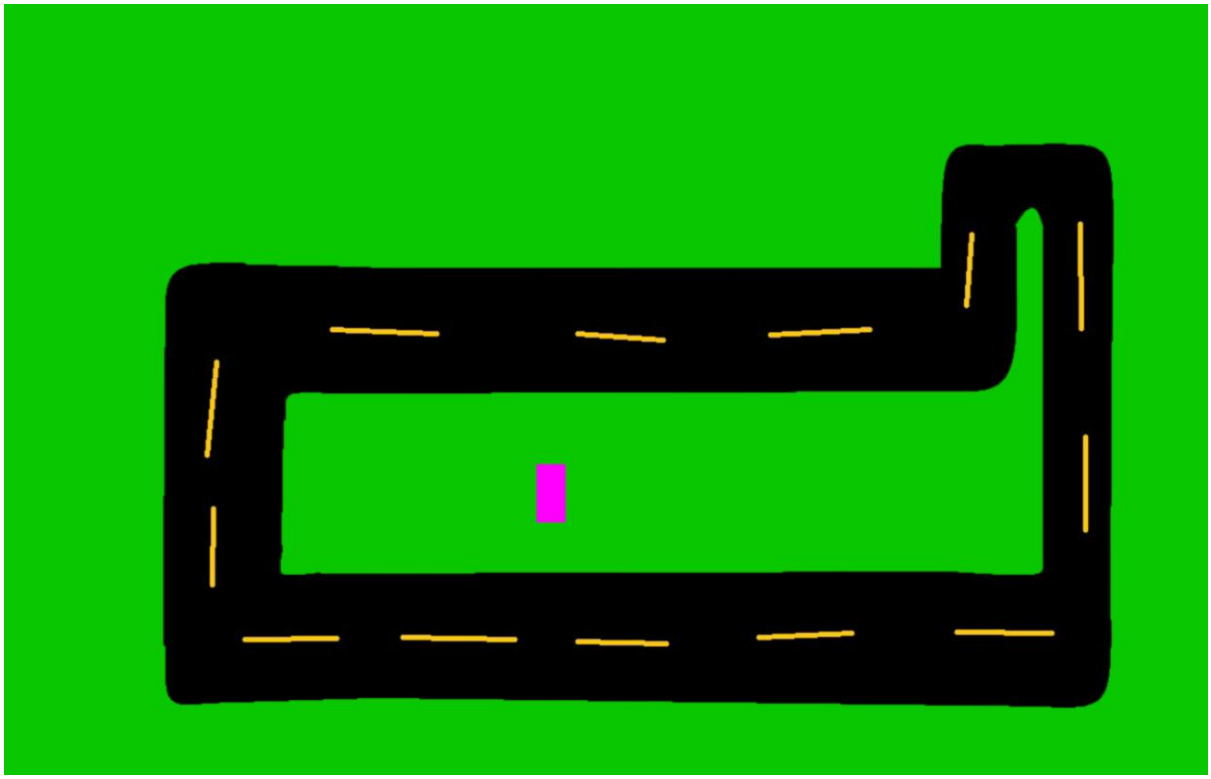
```
    screen.blit(DisplayCarImage,(self.XPos,self.YPos))
                ^^^^^^^^^^^^^^^
NameError: name 'DisplayCarImage' is not defined. Did you mean: 'self.DisplayCarImage'?
```

I realised that, when making the DisplayCarImage and CarImage attributes encapsulated inside of the Car class, I then needed to change every instance where I had used those variables in the main program to call the Car class instead. After fixing this I ran the code again. The code did run correctly, however the image of the track was being displayed above the car and it was not displaying transparent pixels. To fix this I displayed the car image after the track image and I converted the track image to have pixel alphas so it can have transparency.

```
236     Car1.move_car()
237     Track1.display_track()
238     Car1.display_car()
```

```
def __init__(self,image,x,y):
    self.TrackImage = pygame.Surface.convert_alpha(pygame.image.load(image))
```

Now, after changing the background colour to green so that the track is visible, it is correctly displayed and transparent.



However, after running the program again, I noticed that the speed of the car was extremely slow. I realized that this was because every time the game loop (the while running loop) iterated, after my changes there were more processes which had to occur and as the game loop iterated again immediately after, this decreased the number of times that the car's position would be updated in a second.

To fix this, I decided that the game loop should only iterate again after a set amount of time has passed since the previous iteration which would be done using the operating system's clock. time — Time access and conversions — Python 3.14.0 documentation.

I chose to fix the issue this way instead of increasing the speed values to compensate for the slower updating of the car's position as it would mean that the general game loop would repeat at a consistent rate every second.

This is beneficial as it means that I now only need to set a final speed value once and I will no longer have to keep modifying it each time features are added to the game and the game loop occurring a fixed amount of times per second means that the movement of the car will be more consistent as it will not be updated more times if the game loop takes less time to process and updated less times if the game loop takes longer.

However, this does mean that the time that the game loop will have to wait until will have to be quite high. Otherwise, as I add more features the game loop may end up taking more time than I have set out and the time taken by the game loop will just be how long it takes to process which will not be consistent. This is a disadvantage as setting the waiting period between iterations of the game loop to be long will mean that less processes of the game loop can occur in a certain period than what is possible which means that the games "frame rate" may appear slower which may make the game less engaging and enjoyable for some users.

This also means that I will have to decide upon a sensible value to be the frame rate which should be a higher amount of time which is possible for the game loop to take. This is required as if the game loop took longer than the frame rate then the time taken will no longer be limited by the frame rate and it will no longer be consistent.

```
189     StartTime = time.perf_counter()
190     FrameRate = 0.0165000000
```

Here I use the time library to set the StartTime variable equal to the current time on the computer's clock. Then the FrameRate variable is set to the number 0.0165 which means that 16.5 milliseconds must pass between each time the game loop is run. I explain later how I chose the number 0.0165 later

```
195 '''Game loop'''
196 while running: #Infinite loop to prevent the display window from closing until the user decides to
197 
198     NewTime = time.perf_counter() #newTime is set equal to the current time using the system's clock
199 
200     #Here is a loop which makes the game wait until the time defined in FrameRate has passed since the last iteration of the game loop
201 
202     while True:
203         if NewTime > StartTime + FrameRate:
204             break
205         NewTime = time.perf_counter()
206 
207 
208     StartTime = NewTime
209 
```

Here is the beginning of the game loop. I created a variable called NewTime which will be updated at the beginning of the loop and compared to the StartTime. In the loop, if the NewTime is greater than the StartTime plus the FrameRate, then 16.5 milliseconds will have passed since the last time the game loop was run, however if this is false then not enough time will have passed so the loop will repeat and continuously check the new time until it is great enough.

In order to analyse the amount of time passed between frames **without** the enforced framerate I had just implemented, I created the following temporary code:

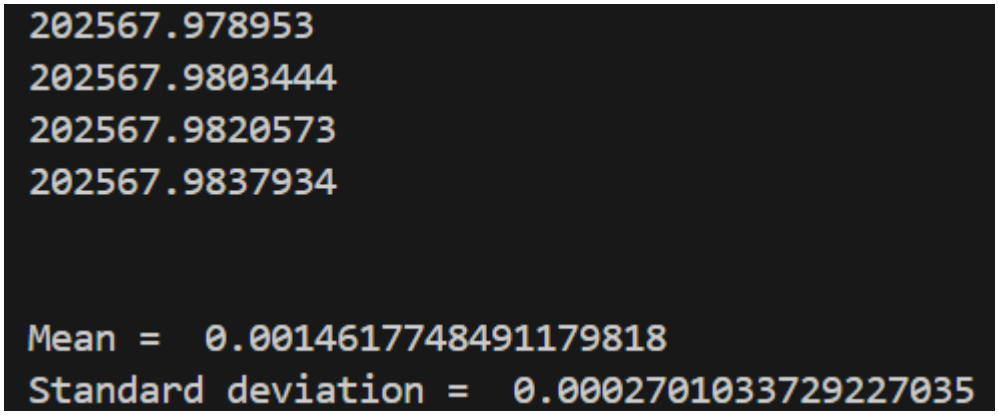
```
count = 0
total = 0
sigmaXSquared = 0

'''Game loop'''
while running: #Infinite loop to prevent the display window from closing until the user decides to

    NewTime = time.perf_counter() #newTime is set equal to the current time using the system's clock

    #Here is a loop which makes the game wait until the time defined in FrameRate has passed since the last iteration of the game loop
    '''
    while True:
        if NewTime > StartTime + FrameRate:
            break
        NewTime = time.perf_counter()
        '''




    count += 1
    total += NewTime - StartTime
    sigmaXSquared += (NewTime - StartTime) **2

    StartTime = NewTime

    print(StartTime)

```

Here, I temporarily removed the code which waits for the FrameRate to have passed and created other variables. The count variable will be used to count every single time that the game loop has been run, the total variable will be used to count the total difference in time taken for all of the iterations of the game loop and sigmaXSquared will store the sum of all the times taken squared. Additionally, the time at the beginning of the game loop will also be printed to be monitored.

```

    '''End game loop'''

    print("\n\nMean = ", total/count)
    print("Standard deviation = ",np.sqrt(sigmaXSquared/count - (total/count)**2))

    pygame.quit()
```

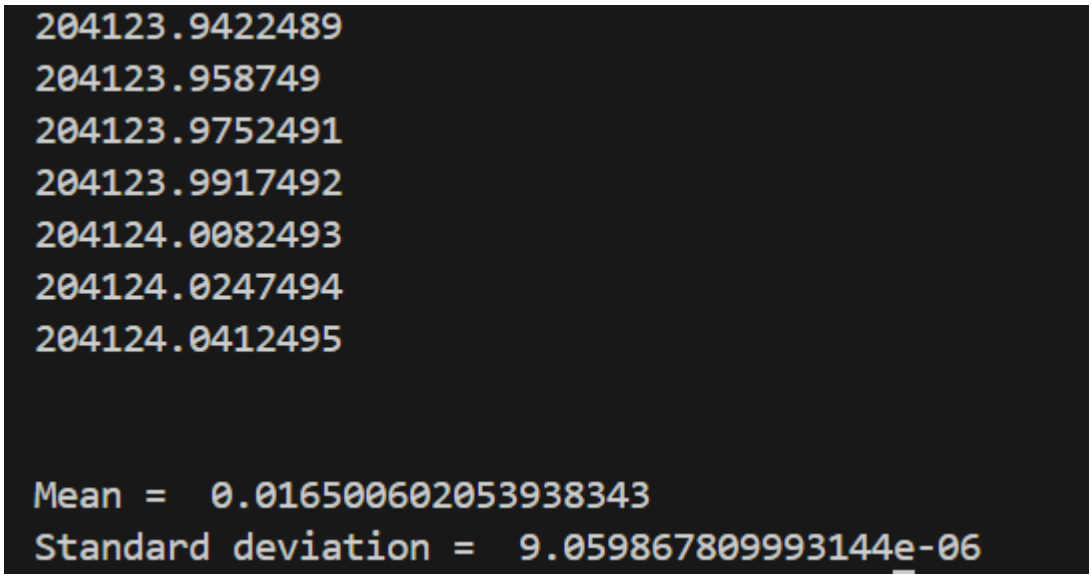
I then created this code at the very end of the program, after the game loop had finished. Here, I used statistical formulas to calculate the mean amount of time which the game loop takes to complete, and the standard deviation of those times. The mean is the average of all the times and the standard deviation is the expected amount of time which the actual times will vary from the average.

After running the program for 20 seconds, I received these values:



This meant that the average amount of time that the game loop takes to process is 1.46 milliseconds and this had a standard deviation of 0.27 milliseconds which is the average amount it varied by. The standard deviation was quite low which is beneficial as if it was high, then I would have to set the FrameRate much higher than the mean time as the time would vary by a large amount. I chose to set the frame rate as 16.5 milliseconds which is 11 times larger than the average frame rate currently, as when I add other features, such as the neural network, these are likely to increase the average frame rate by a very large proportion so I set the FrameRate to a value much higher than the current average which still did not appear "laggy" when running the program.

Additionally, I ran the program after including the code which limits the time between frames to 16.5 milliseconds to test its mean and standard deviation and got the following results:



The mean was very close to 0.0165 and the standard deviation was very close to 0 which suggests that the time between frames was now very consistently close to 0.0165. After re-adjusting all of the speed values of the car so that the gameplay was engaging and realistic, I was now confident that the frame rate would remain at 16.5 milliseconds, and I would no longer need to update the values again after each change.

After fixing that issue, I continued again with the development plan for this stage. The next step was to implement the collision detection between the car and the track. To do this I followed the pseudocode created in my iterative design and the flowchart I created to write this code:

```
196    NormalFriction = 0.0095
197    OffTrackFriction = 0.0120
```

```
280
281     screen.fill((10,200,0))
282     Car1.move_car()
283
284
285     if pygame.sprite.spritecollide(Track, Car, False):
286
287         if pygame.sprite.spritecollide(Track,Car,False,pygame.sprite.collide_mask):
288             Friction = NormalFriction
289     else:
290         Friction = OffTrackFriction
291
```

After running this code, I was met with the following error:

```
    default_sprite_collide_func = sprite.rect.colliderect
                                  ^^^^^^^^^^^
AttributeError: type object 'Track' has no attribute 'rect'
```

Initially, I attempted to rename the attribute TrackRect to rect however this did not fix the issue. I then realised that I had named the classes Track and Car instead of the names of the objects Car1 and Track1.

```
285     if pygame.sprite.spritecollide(Track1, Car1, False):
286
287         if pygame.sprite.spritecollide(Track1,Car1,False,pygame.sprite.collide_mask):
288             Friction = NormalFriction
289     else:
290         Friction = OffTrackFriction
291
```

After I fixed this the error persisted however, but after renaming the attributes TrackRect and TrackMask in the Track class to rect and mask, the issue was resolved. I then updated these names in the data dictionary. I also updated the names of the corresponding attributes in the car and finish line classes to prevent the same issue in the future.

However, after running the code again, I was met with the following error:

```
  File "c:\Users\oscar.turnbull24\Github\Coursework\Ai racer.py", line 285, in <module>
    if pygame.sprite.spritecollide(Track1, Car1, False):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\oscar.turnbull24\AppData\Roaming\Python\Python312\site-packages\pygame\sprite.py", line 1737, in spritecollide
    return [
           ^
TypeError: 'Car' object is not iterable
```

As this was an issue with the spritecollide function in pygame, I consulted the tutorial I had used for this function again How to Use Pygame Masks for Pixel Perfect Collision. I realised that the spritecollide function only worked on sprite groups which is something I had not created. However, creating a pygame sprite group would be beneficial for my project as it would allow me to store both car objects in the same group if multiple cars are being used and then perform collision detections with both car objects in the group.

```
174  #Instantiating the car and racetrack objects
175  Car1 = Car(500,350,0,CarImage)
176  Track1 = Track("TEMP racetrack.png", 100,100)
177
178  #Creating Sprite groups
179  CarGroup = pygame.sprite.Group()
180  CarGroup.add(Car1)
```

After this, I encountered another error message:

```
  File "c:\Users\oscar.turnbull24\Github\Coursework\Ai racer.py", line 181, in <module>
    CarGroup.add(Car1)
  File "C:\Users\oscar.turnbull24\AppData\Roaming\Python\Python312\site-packages\pygame\sprite.py", line 479, in add
    sprite.add_internal(self)
    ^^^^^^^^^^^^^^^^^^^
AttributeError: 'Car' object has no attribute 'add_internal'
```

I was unsure what the sprite.add_internal(self) function was so I read the pygame documentation on sprites Pygame Tutorials - Sprite Module Introduction — pygame v2.6.0 documentation. I then realised by reading this that the reason that my classes could not use pygame sprite functions was because my classes were not child classes inherited from the parent sprite class, so I changed this for the Car class:

```
11  class Car(pygame.sprite.Sprite):
12
13      def __init__(self,XPos,YPos,Rotation,CarImage):
14          pygame.sprite.Sprite.__init__(self)
```

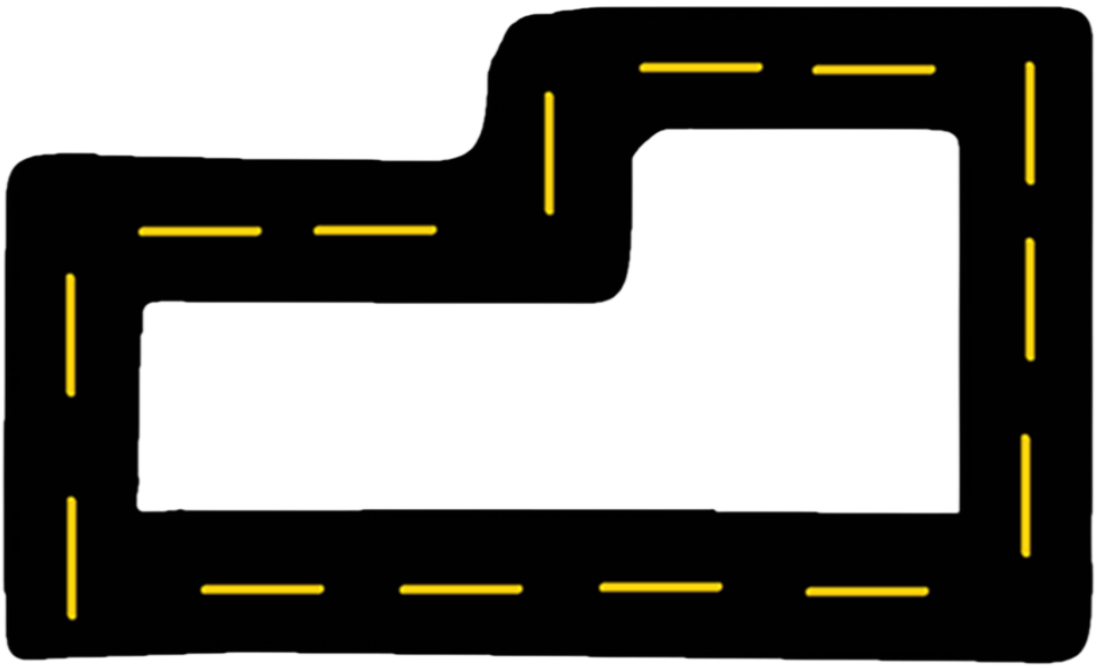
```
292     #Rectangle collision detection between the track and the car
293     if pygame.sprite.spritecollide(Track1, CarGroup, False):
294         #Mask collision detection between the track and the car
295         if pygame.sprite.spritecollide(Track1,CarGroup,False,pygame.sprite.collide_mask):
296             Friction = NormalFriction
297     else:
298         Friction = OffTrackFriction
299
```

After making these changes, the game was able to run again. However, when driving around the entire screen, it seemed that only the rectangle collision detection was

working. I realised this was because the else statement is only reached if the rectangle collision is false and is unaffected by the mask collision. I fixed that as follows:

```
300     #Rectangle collision detection between the track and the car
301     if pygame.sprite.spritecollide(Track1, CarGroup, False):
302         #Mask collision detection between the track and the car
303         if pygame.sprite.spritecollide(Track1, CarGroup,False, pygame.sprite.collide_mask):
304             Friction = NormalFriction
305         else:
306             Friction = OffTrackFriction
307 
308     else:
309         Friction = OffTrackFriction
```

After running the code again, both the mask collision and the rectangle collision worked correctly. Now that I could test out driving the car with the different friction values in place, I decided that the wider track was more enjoyable to drive around so I updated the image of the track as follows:



Now that the car and track correctly collide, I moved on to implementing the finish line.

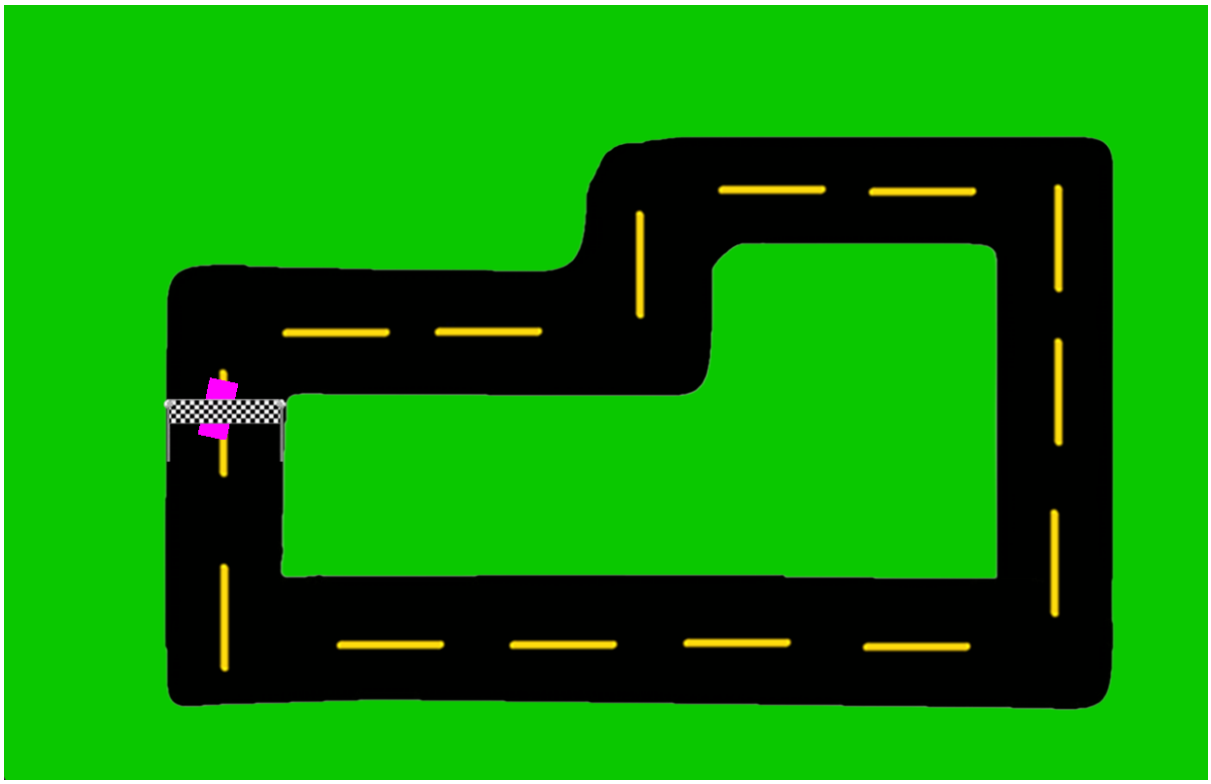
I downloaded an image of a racetrack from the following link:

https://tse3.mm.bing.net/th/id/OIP.V279wK8hzEyPjgwf0TxufwHaET?rs=1&pid=ImgDetMain&o=7&rm=3

Then, I inserted the following line of code:

```
189    Finishline1 = FinishLine("finishline.png", 159,400)
```

And, after adjusting the size of the image and the position of it in the game, it was correctly shown on the racetrack:



To make it so that the car and the finish line's collisions were detected, I reused the code from the previous collision detection:

```
117    def new_lap(self, TotalLaps): #Method which increases the number of laps.
118        self.LapCount += 1
119        if self.LapCount >= TotalLaps:
120            return False #Returns False if the max number of laps is reached so the car has won.
121        return True #Returns True if the lapCount is less than the max and the game continues.
```

I created the new_lap method which will be run every time the car reaches the finish line and is inside of the car class. As the car has completed a lap, the counter for the number of completed laps will increase by 1. If the counter has reached the total number of laps, then the method returns False which will mean that this car has won. If the counter has not reached the total number of laps yet, then the method will return True, and the game will continue.

```
14  class Car(pygame.sprite.Sprite):
15
16      def __init__(self,XPos,YPos,Rotation,CarImage):
17          pygame.sprite.Sprite.__init__(self)
18          self.XPos = XPos
19          self.YPos = YPos
20          self.Rotation = Rotation
21          self.XSpeed = 0
22          self.YSpeed = 0
23          self.ResultantSpeed = 0
24          self.CarImage = pygame.Surface.convert_alpha(CarImage)
25          self.DisplayCarImage = self.CarImage
26          self.rect = self.DisplayCarImage.get_rect()
27          self.rect.topleft = (self.XPos,self.YPos)
28          self.mask = pygame.mask.from_surface(self.DisplayCarImage)
29          self.LapCount = 0
```

```
218 TotalLaps = 3 #This will become a player input later
```

I also created the LapCount attribute inside of the car class and the TotalLaps variable which will be given a value decided by the user. I have not implemented the user input in this stage as that would involve modifying the in-game settings menu which has not yet been created.

```
325     #Rectangle collision detection between the finish line and the car
326     if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
327         #Mask collision detection between the finish line and the car
328         if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
329             if not Car1.new_lap(TotalLaps):
330                 #Car1 has won
331                 print("The car is colliding with the finish line")
```

Once again, a rectangle collision check is used between the car and the finish line as this is very fast to do, and if there is a rectangle collision is detected, then the more accurate mask collision is done. If the car has collided with the finish line, then the new_lap method will increase the number of laps in the car's counter. If the method returns true, then the game will continue, however if the method returns false then a section of code will handle the car winning the game. The final code which will handle the game winning will be created in the next development stage, but currently there is a placeholder print statement to test if the collision detection works. When I ran the program and drove the car into the finish line, the following output was produced:

```
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
The car is colliding with the finish line
```

This output did not occur when the car was not colliding with the finish line which means the collision system functions correctly. However, this output occurs constantly while the car is touching the finish line, however this is only useful if the new lap method is run the first time that the collision is detected. This issue however ties in more with the final gameplay of the racing game so I will address it during the next stage of development.

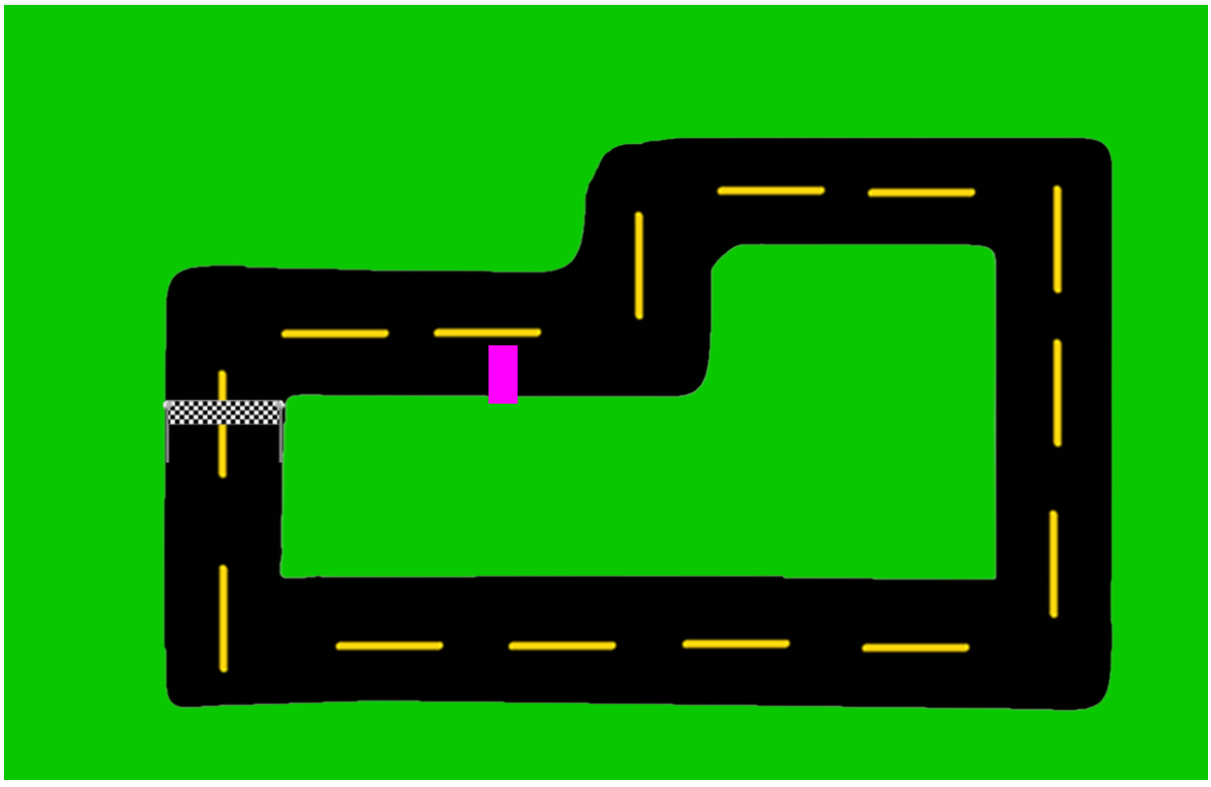
Now that the development for stage 2 has been completed, I will work through the test plan for the stage which I created in the design section. This will allow me to evaluate if this stage has met the success criteria which has been set out for it. **SEE EVIDENCE OF TESTS BELOW**.

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program.	When the program is run, there should be a background image displayed.	A background image is displayed.	Pass
2	Run the program.	The background image should be of a racetrack which includes track, off-track sections and the start/finish line.	The background image is of a racetrack and displays the track, off track and start/finish line.	Pass
3	Run the program and press W then let go on track. Then repeat the same off track.	There should be stronger friction applied to the car off track than on track.	When off the track, the car slows down faster than on the track.	Pass, see “Test recording 1”
4	Before running the program, add a temporary line of code which prints the text “Test	There should be a collision check implemented with the car and	“Test passed” will be printed only when the	Pass, see “Test recording 2”

	passed" when the car collides with the start/finish line. Then run the program and then drive the car over the start/finish line.	the finish line. To test this, the string "Test passed" will be printed when there is a successful collision between the car and the finish line.	car collides with the finish line.	

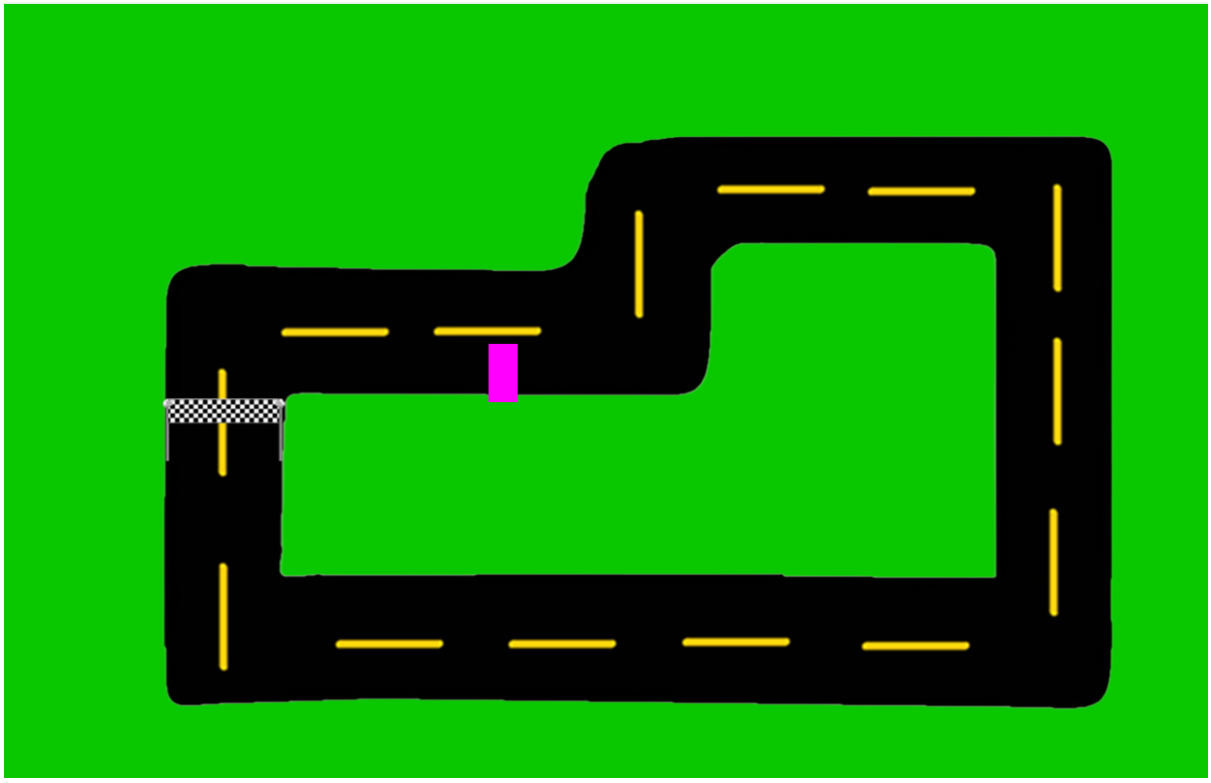
Test 1:

I ran the program, and a background image (of the racetrack and finish line) was displayed on screen, therefore this test is a pass.



Test 2:

I ran the program, and the background included the racetrack, the off-track sections (the green areas) and the finish line. Therefore, this test is a pass.



Test 3:

See “Test recording 1”. As I held W when the car was on the track, it accelerated quickly as there was less friction. As I held W when the car was off the track, it accelerated slowly as there was higher friction applied. Therefore, this test is a pass.

Test 4:

See “Test recording 2”. I added the temporary line of code which printed “Test passed” when the car collides with the finish line.

```
327         if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
328             if not Car1.new_lap(TotalLaps):
329                 #Car1 has won
330                 print("Test passed")
```

During the recording initially, the car was not colliding with the finish line and nothing was being printed to the terminal. However, when the car did collide with the finish line, “Test passed” was printed continuously. Additionally, “Test passed" was only printed when a mask collision took place. This was shown in the recording as “Test passed” was not outputted when the car was inside of the finish line’s entire image size (rectangle) but it was outputted when visible pixels from the car collided with visible pixels from the finish line. Therefore, this test was a pass.

FIX DIS

pygame.transform — pygame v2.6.0 documentation

Collision Detection in PyGame - GeeksforGeeks

pygame.mask — pygame v2.6.0 documentation

How to Use Pygame Masks for Pixel Perfect Collision - YouTube

numpy.absolute — NumPy v2.3 Manual

standard deviation formula - Search Images

pygame.sprite — pygame v2.6.0 documentation

Stage 3- creating the gameplay and menu:

Now that development for stage 2 is complete, stage 3 can begin. In this stage, I will create the "gameplay" of the game which will mean that the number of laps which the car has completed will be tracked, and once the number of laps has reached a certain amount, the game will end. Additionally, I will implement a main menu for the game which will allow the user to decide whether to begin playing the game or access the settings, I will implement a settings menu so that the user is able to customise the experience to their preferences and finally I will create a screen which will be displayed to the user once the game is over.

Features to implement in this stage:

- The car will be tracked for how many laps it makes around the track and once it reaches a set number the game will end
- The car's time spent on each lap will be recorded and displayed when the game has finished.

- A main menu, settings screen and race finish screen will be created which will open when the program is run

Algorithm design:

The various menus or screens which will be created in this stage do not involve many complicated processes (for example, the main menu screen will simply wait until the user presses a button on screen and then it will run the corresponding subroutine such as opening the settings menu when the settings button is pressed) so there will be little algorithm design needed for them. However, much of the planning for the functionality of these screens and their design has been outlines in the GUI design section of the predevelopment stage, and I will refer heavily to this.

Previously in stage 2, I discovered that the collision detection with the finish line would occur many times when the car was touching the finish line, however I only want the collision detection to be registered immediately once when the car collides with the finish line so that each full lap of the track is detected once. To fix this, I decided that I will create multiple checkpoints along the track and for a lap to be counted as completed, every checkpoint must have been reached in the order in which they are placed on the track. This approach will have many benefits.

An alternative approach I could have used would be to have only counted a collision with the finish line as a "full lap" as soon as the car begins colliding with it and not counting further collision detections as full laps until the car has finished colliding with it and moved away. However, this approach would mean that the car could touch the finish line to gain a lap complete, then drive slightly away from the finish line so that the car is not colliding with it, then the car could reverse and touch the finish line again to count another lap being complete despite the car not having actually driven across the entire track. This means that the alternative approach could be easily cheated and therefore is not suitable.

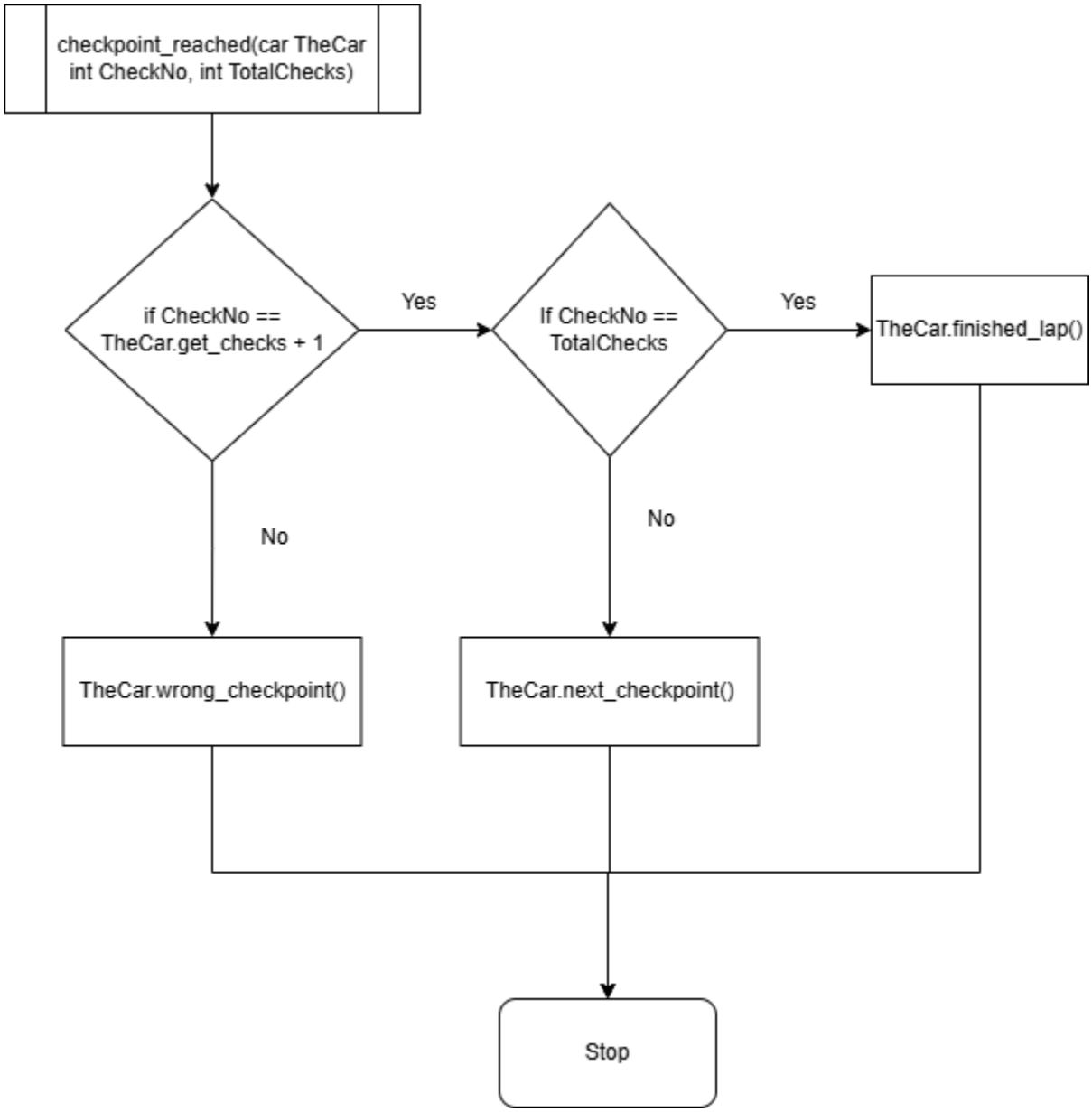
Using checkpoints across the track, however, ensures that the car must drive around the entire track and reach every checkpoint before it is able to claim a lap being completed by touching the finish line which means that this method is much harder to cheat.

Additionally, once the car reaches the finish line, the first collision detection will count as a lap complete however, the car's internal "last checkpoint" attribute will be reset to

0, which will mean that the next checkpoint the car will need to reach will be the first one on the track which means that further collisions with the finish line will not count as a lap being completed.

Finally, the method of having checkpoints around the track will be beneficial when training the neural network. This is because the neural network can be rewarded for reaching checkpoints across the track which will be an easy way to train and reward it for making progress towards the finish line.

Here is the flowchart for the “checkpoint_reached” subroutine. This subroutine will be run every time a checkpoint is reached including the finish line. The subroutine will receive the parameters of the car object which collided with the checkpoint, the number of the checkpoint (with 1 being the first checkpoint and the largest number being the finish line) and the total number of checkpoints which are on this track (which will equal the CheckNo for the finish line):



If the CheckNo for this checkpoint is 1 more than the last checkpoint attribute stored in the car, then this means that this checkpoint is the correct next one for the car to reach. If the CheckNo for this checkpoint is equal to the total number of checkpoints on the track, then this must be the finish line which means that the car's finished_lap() method will be called which will increase the number of laps which the car has completed by 1, reset the LastCheckpoint attribute to 0 (so that the next checkpoint to be reached will be the first one on the track) and it will check if the car has won the game. If the checkpoint is not the finish line, then the car's next_checkpoint() method will be called which will simply increase the car's LastCheckpoint attribute by 1. If the checkpoint reached is the incorrect one in the order, then the car's wrong_checkpoint() method will be called. This method does not necessarily need to do anything as it simply will just not increase the LastCheckpoint attribute which will mean that the next checkpoint which

will need to be reached will remain the same, however the method could be used to punish the neural network for reaching the wrong checkpoint if necessary.

Here is the pseudocode I created for the next_checkpoint() method:

```
PROCEDURE next_checkpoint():
    this.LastCheckpoint += 1
END PROCEDURE
```

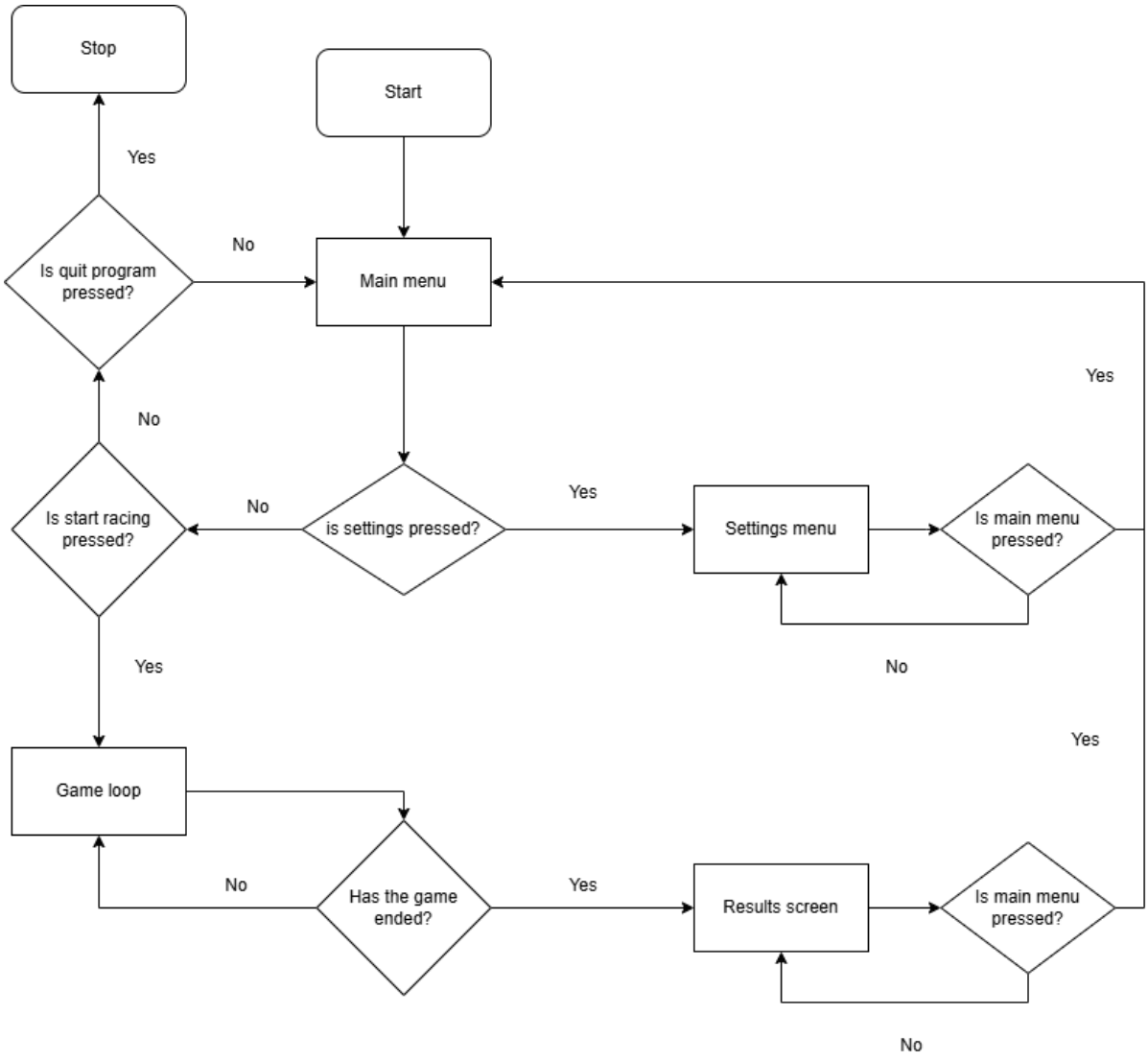
In this procedure the LastCheckpoint attribute inside the car is incremented by 1.

Here is the pseudocode I created for the finished_lap method:

```
PROCEDURE finished_lap():
    this.LapCount += 1
    this.LastCheckpoint = 0
    if this.LapCount == TotalLaps Then
        end_game(this)
    end if
END PROCEDURE
```

In this procedure, the car's LapCount attribute is incremented by 1 which means that the total number of laps the car has completed is increased by 1. The LastCheckpoint attribute is set to 0 which means that when the car reaches another checkpoint, it will now be required to reach checkpoint 1 after reaching the finish line and starting a new lap. If the LapCount is now equal to the total number of laps which are required to win, then this means that the car has won the race. If the car wins the race, then the global end_game subroutine will be called. This subroutine will end the current game loop and transition to the results screen. The end game subroutine will also take the car object as a parameter so it can receive information such as the car's total amount of time taken to win and the number of laps completed.

I created this flowchart to represent the general structure of all the elements of the game:



When the program is launched, the main menu is immediately launched. Inside of the main menu, if the settings menu button is pressed, then the settings menu is launched. If the start racing button is pressed, then the game loop is launched. If the quit program button is pressed, then the game closes.

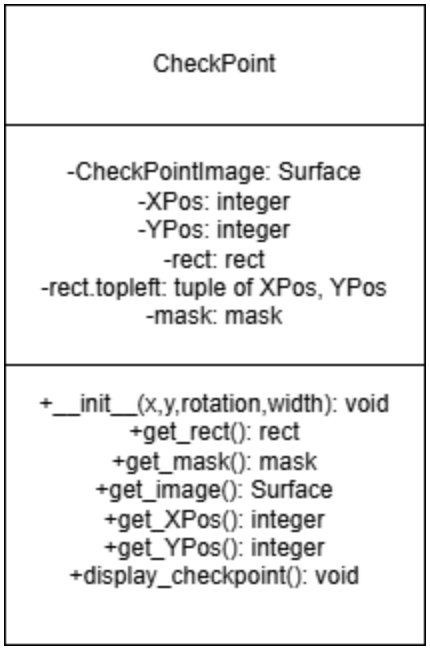
Inside of the settings menu, various settings can be adjusted. If the main menu button is pressed, then the main menu is launched.

In the game loop, the game continues until the car finishes all the laps.

Once this happens, the results screen is launched which displays information to the user. In the results screen, if the user presses the main menu button, then the user is

sent to the main menu. Additionally, if the user ever presses the X button on the game window, the game will close.

Class diagrams:



Here is a class diagram for the checkpoint class. The CheckPointImage attribute will store the image of the checkpoint as a surface. The checkpoints need to be wide enough to cover the entire width of the track so that the car is detected at any point on the track, however the checkpoint cannot be wider than the track, otherwise the car could be detected as having crossed a checkpoint when it is not on the track which is not intended to happen. As the width of the track varies at places, each CheckPointImage will initially have a width of 1, and the width will be updated for each checkpoint individually.

The XPos and YPos attributes store the X and Y positions of the top left pixel of the checkpoint as integers. The rect attribute stores the pygame rect for the CheckPointImage and the mask attribute stores the pygame mask for the CheckPointImage. The rect.topleft attribute stores the XPos and YPos attributes in a tuple to be used for collision detection with the checkpoints.

The __init__ method is the constructor for the class and this will set the XPos and YPos attributes to passed in parameters, so that each checkpoint can be in a different position on the track. It also takes in the rotation and width parameters. I leant from pygame.transform — pygame v2.6.0 documentation that the pygame.transform.scale()

function can be used to change the height and width of a surface and the pygane.transform.rotate() function can be used to rotate an image (which has been used previously with the car) so I will use these in the checkpoint's constructor to rotate and scale the CheckPointImage surface so that each checkpoint can be placed on any point on the track.

The rest of the methods in this class are getters, apart from the display_checkpoint() method which uses the pygame blit function (pygame.Surface — pygame v2.6.0 documentation) to display the image of the checkpoint on the screen. The display_checkpoint() method will only be needed for development and testing purposes so that I can see where checkpoints have been placed.

Here is the data dictionary for the variables which are not encapsulated in classes for this stage:

Name	Data type	Description	Validation
TotalChecks	int	The total number of checkpoints which are on the track, including the finish line	None (will be constant for each track)
CheckArray	array	An array which stores all of the checkpoint objects for a particular track	None
icon	Surface	An image of the car which is used as the icon in the top left of the display window	None
title_font	font	A very large pygame font object which is used to write text onto the screen	None
body_font	font	A small pygame font object which is used to write text onto the screen	None
medium_font	font	A medium sized pygame font	None

		object which is used to write text onto the screen	
LastButtonPress	float	The time on the system's clock when the last button was pressed	None
DisplayCheckpoints	boolean	This value will be True if checkpoints are being displayed and False if they are not	None
IsSinglePlayer	boolean	This value will be True if only 1 player is in the game and False if there are 2 players	None
LastResult	array	This array holds the time that the last winning player took to complete the game and the player number of the winning player	None
NextProcess	char	This is a character which holds which game screen will be opened next	None
TimerStart	float	This holds the time on the system's clock when the game loop began	None
TimerDisplay	float	This holds how much time has passed since the game loop began	None

Here is the data dictionary for attributes which are encapsulated inside of classes for this stage.

Name	Data type	Description	Validation	In class
LapCount	int	The number of laps which have been completed by the car	Must be between 0 and the TotalLaps variable	Car
LastCheckpoint	int	The number of the previous checkpoint which was reached on the track	Must be between 0 and the TotalChecks variable	Car
PlayerNo	int	The player number of this car	None	Car
CheckPointImage	Surface	The image of the checkpoint to be displayed on screen	None	Checkpoint
XPos	int	The X position of the checkpoint	None	Checkpoint
YPos	int	The Y position of the checkpoint	None	Checkpoint
rect	rect	The rect object for the checkpoint	None	Checkpoint
rect.topleft	tuple	A tuple holding the X and Y positions of the checkpoint	None	Checkpoint
mask	mask	The mask of the checkpoint	None	Checkpoint
CheckNo	int	The number of the checkpoint	None	Checkpoint
Width	int	The width of the button	None	Button
Height	int	The height of the button	None	Button
XPos	int	The X position of the button	None	Button
YPos	int	The Y position of the button	None	Button
XOffset	int	The offset of the X position of the button's text	None	Button

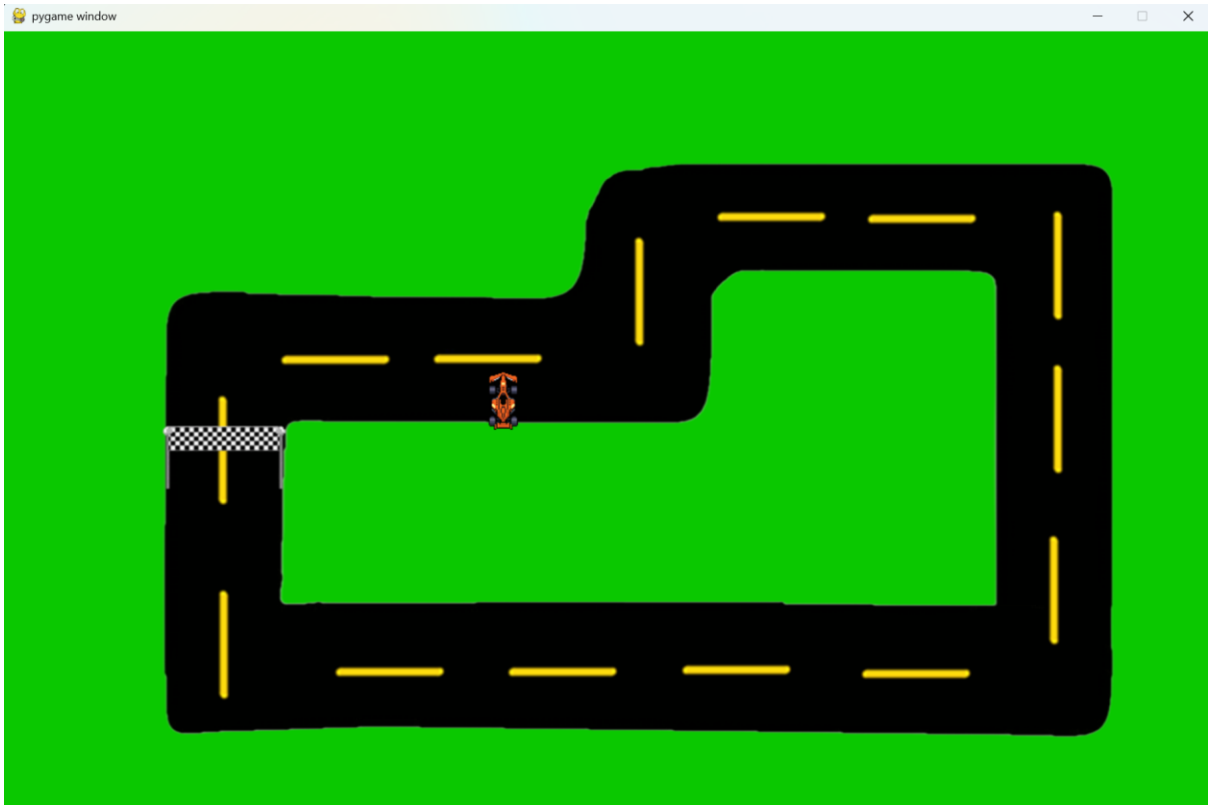
YOffset	int	The offset of the Y position of the button's text	None	Button
Surface	Surface	The image of the button	None	Button
Rect	rect	The rect of the button	None	Button
Text	text	The text object to be displayed onto the button	None	Button
Colour	String	The current colour of the button	None	Button

Programming:

To begin stage 3, I decided to implement all the gameplay features before the various screens and menus. I chose to do this because the results screen implementation requires the game to know when the car has won before it can be displayed. Additionally, the settings menu will allow the user to modify the TotalLaps variable and I would personally prefer for this value to remain constant until the gameplay has been tested and works correctly.

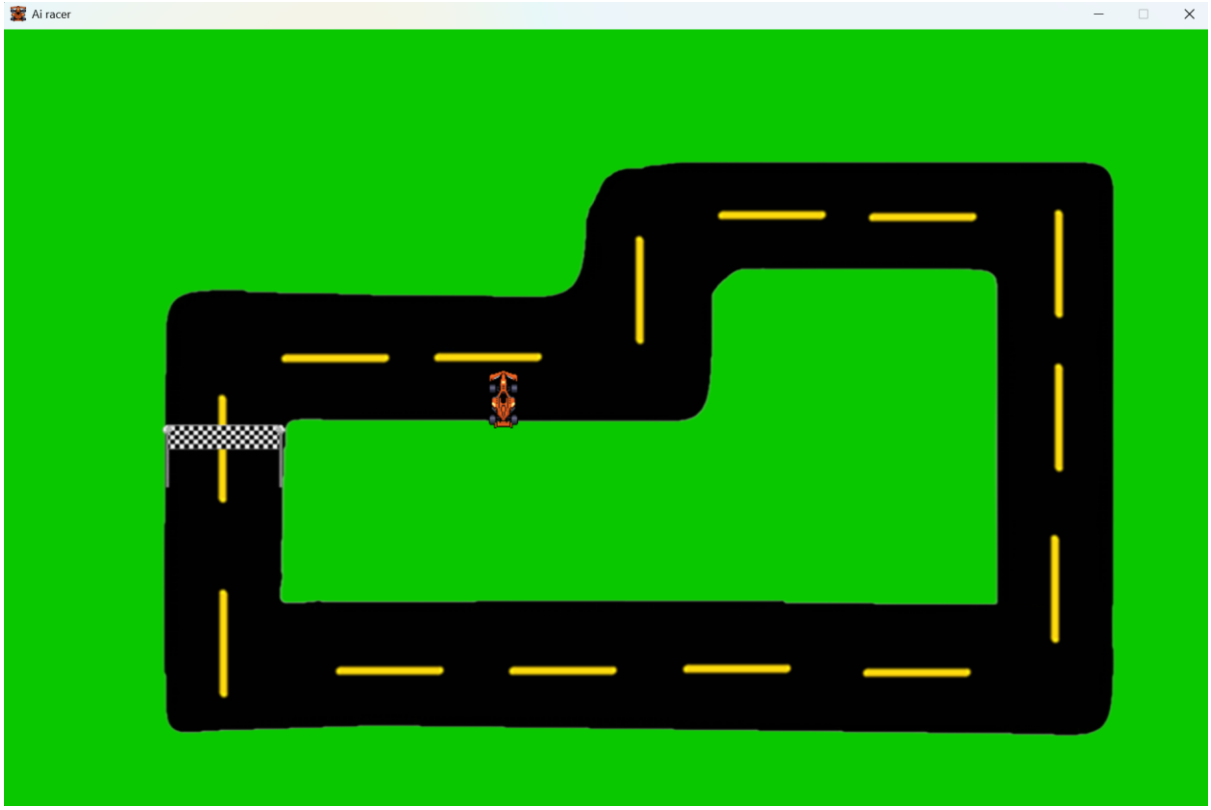
To begin the development, I decided to remove the placeholder pink rectangle image used for the car for a final image of a pixelated race car. I chose to do this at this stage as now all the GUI features will be implemented so it is a suitable time to ensure that all the assets which are used in the game are up to a good standard. I used the following image of a race car from https://www.vecteezy.com/vector-art/33530269-pixel-art-illustration-f1-car-pixelated-race-f1-car-f1-car-race-vehicle-pixelated-for-the-pixel-art-game-and-icon-for-website-and-video-game-old-school-retro:

I reduced its resolution in Microsoft paint so that it would be the same dimensions as the pink image previously used. Here is an image of the new race car image in the game:



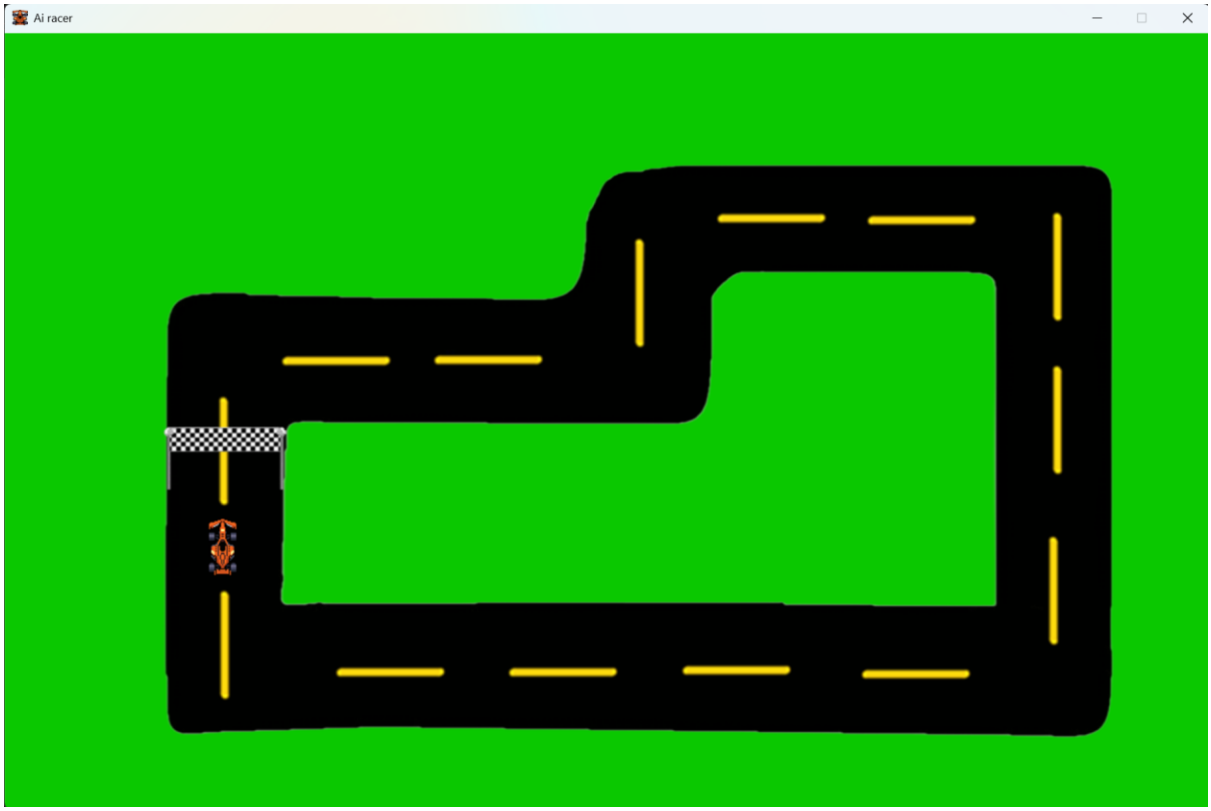
I also used the pygame 'set_caption' and 'set_icon' subroutines to customise the name and icon of the display window so it no longer says "pygame window":

```
10  #Customising pygame window
11  pygame.display.set_caption("Ai racer")
12  icon = pygame.image.load('Racecar.png')
13  pygame.display.set_icon(icon)
```



Finally, I updated the start position of the car so that it would begin in front of the finish line (however its initial value of LastCheckpoint will be set to 0 so that it will not receive a lap being complete when first crossing the finish line).

```
201    #Instantiating the car and racetrack objects
202    Car1 = Car(210,500,0,CarImage)
203    Track1 = Track("TEMP racetrack.png", 100,100)
204    Finishline1 = FinishLine("finishline.png", 159,400)
```



Now I will begin creating the gameplay. The first step to doing this is creating the checkpoints across the track.

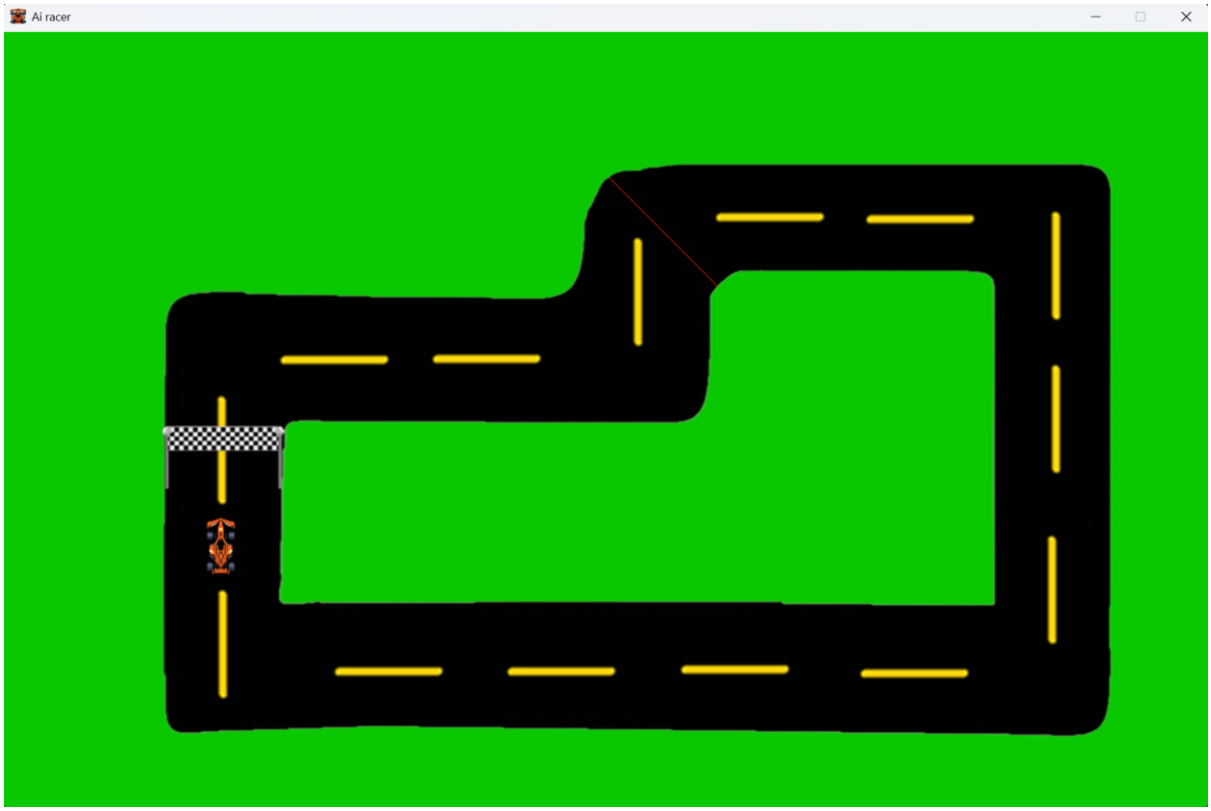
pygame.transform — pygame v2.6.0 documentation

```
188  class CheckPoint:
189      def __init__(self,x,y,rotation,width):
190          self.image = pygame.Surface.convert_alpha(pygame.image.load("Checkpoint.png"))
191          self.image = pygame.transform.scale(self.image,(width,1))
192          self.image = pygame.transform.rotate(self.image,rotation)
193          self.XPos = x
194          self.YPos = y
195          self.rect = self.image.get_rect()
196          self.rect.topleft = (self.XPos,self.YPos)
197          self.mask = pygame.mask.from_surface(self.image)
198
199
200      def get_rect(self): #Getter for the rect of the checkpoint
201          return self.rect
202
203      def get_mask(self): #Getter for the mask of the checkpoint
204          return self.mask
205
206      def get_image(self): #Getter for the image of the checkpoint
207          return self.image
208
209      def get_XPos(self): #Getter for the mask of the checkpoint
210          return self.XPos
211
212      def get_YPos(self): #Getter for the image of the checkpoint
213          return self.YPos
214
215      def display_checkpoint(self): #Method to display the checkpointto the screen
216          screen.blit(self.image,(self.XPos,self.YPos))
217
```

Firstly, I created the checkpoint class following the class diagram I had designed. In the constructor for the class, I loaded in “Checkpoint.png” which is an image comprised of a single red pixel, then I update the width and height of this image to the passed in parameter for the width and 1 for the height so that the height is always 1 and the width can vary. After this, I used pygame’s rotate function to rotate the image anticlockwise using the rotation parameter passed in so that the checkpoints are correctly covering the track.

```
241    Check1 = CheckPoint(625,150,-45,160)
```

To test this class, I instantiated a checkpoint named Check1 with parameters I decided upon after testing, and I also made it so the display_checkpoint() method would be called during the game loop. After altering the values of the parameters to be suitable, I ran the code and the checkpoint was displayed onto the screen correctly:



Next, I created the array “CheckArray”. This will store all the checkpoints on the track in the order which they appear on the track. This will make it so that all the checkpoints are able to be iterated through in the array so that collision detections can be done with each checkpoint or each checkpoint can be displayed efficiently.

I also created the variable “TotalChecks” which stores the total number of checkpoints on the track. It is much better to iterate through all the checkpoints to perform operations on all of them as there are many different checkpoints which would require a large amount of repeated code to process each one individually.

Next, I instantiated a CheckPoint object for all the 19 checkpoints I decided should be on the track. With each one, I entered arguments for their position, rotation and width after running the program and displaying them so that each one could be in the correct position. It was a time-consuming process to create each checkpoint in the correct spot; however, this was necessary as each checkpoint must have unique values for all their parameters which must be manually entered.

I decided to manually add every single CheckPoint into the CheckArray. This potentially could have also been automated using a for loop, however it was fast to copy and paste Check1 19 times and then update the number so I decided against this, however, if I

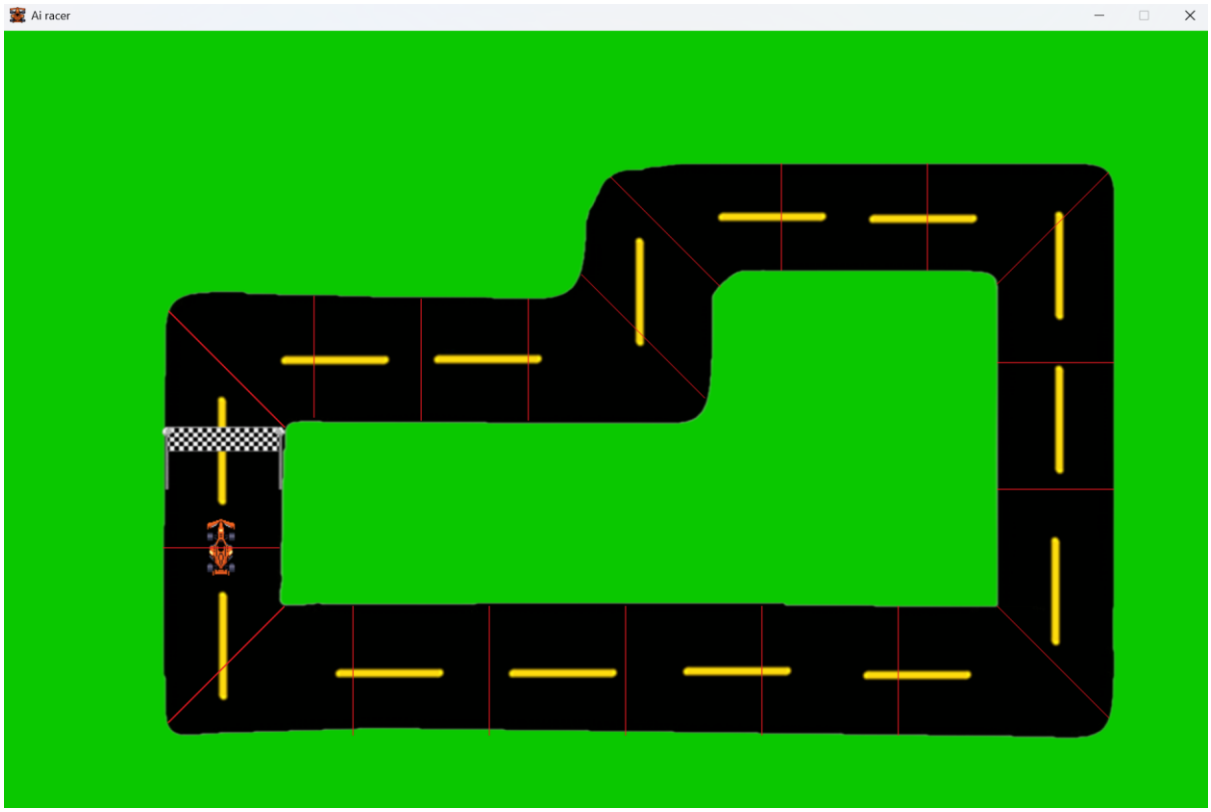
were to add more tracks in the future with different checkpoint positions then this automation would speed up this process slightly.

```
#Instantiating all of the checkpoints
Check1 = CheckPoint(171,287,-45,170)
Check2 = CheckPoint(320,272,90,125)
Check3 = CheckPoint(430,275,90,125)
Check4 = CheckPoint(540,275,90,125)
Check5 = CheckPoint(595,250,-45,180)
Check6 = CheckPoint(625,150,-45,160)
Check7 = CheckPoint(800,136,90,110)
Check8 = CheckPoint(950,136,90,110)
Check9 = CheckPoint(1022,145,45,163)
Check10 = CheckPoint(1022,340,0,120)
Check11 = CheckPoint(1022,470,0,120)
Check12 = CheckPoint(1022,590,-45,163)
Check13 = CheckPoint(920,590,90,133)
Check14 = CheckPoint(780,590,90,133)
Check15 = CheckPoint(640,590,90,133)
Check16 = CheckPoint(500,590,90,133)
Check17 = CheckPoint(360,590,90,133)
Check18 = CheckPoint(170,590,45,170)
Check19 = CheckPoint(165,530,0,120)


TotalChecks = 19

CheckArray = [Check1,Check2,Check3,Check4,Check5,Check6,Check7,Check8,Check9,Check10,Check11,Check12,Check13,Check14,Check15,Check16,Check17,Check18,Check19]

```

Here is the temporary code in the game loop which displays every checkpoint:

```
    for i in range(TotalChecks): #displays every checkpoint on the screen (for testing)
        CheckArray[i].display_checkpoint()
```

Here is how the track looked after every checkpoint had been introduced:



I realised that, if I wanted to ensure that all the checkpoints must be reached in the correct order (first checkpoint, then second checkpoint, etc so that you cannot go backwards around the track) then every checkpoint must have an attribute which stores

their checkpoint number, so I created that as follows in the CheckPoint class (This was also planned in the algorithm design):

```
self.CheckNo = CheckNo
```

I then included the checkpoint number for each checkpoint when I instantiate it:

```
#Instantiating all of the checkpoints
Check1 = CheckPoint(171,287,-45,170,1)
Check2 = CheckPoint(320,272,90,125,2)
Check3 = CheckPoint(430,275,90,125,3)
Check4 = CheckPoint(540,275,90,125,4)
Check5 = CheckPoint(595,250,-45,180,5)
Check6 = CheckPoint(625,150,-45,160,6)
Check7 = CheckPoint(800,136,90,110,7)
Check8 = CheckPoint(950,136,90,110,8)
Check9 = CheckPoint(1022,145,45,163,9)
Check10 = CheckPoint(1022,340,0,120,10)
Check11 = CheckPoint(1022,470,0,120,11)
Check12 = CheckPoint(1022,590,-45,163,12)
Check13 = CheckPoint(920,590,90,133,13)
Check14 = CheckPoint(780,590,90,133,14)
Check15 = CheckPoint(640,590,90,133,15)
Check16 = CheckPoint(500,590,90,133,16)
Check17 = CheckPoint(360,590,90,133,17)
Check18 = CheckPoint(170,590,45,170,18)
Check19 = CheckPoint(165,530,0,120,19)
```

I then created the collision detection between the car and the checkpoints. Firstly, I updated the collision detection between the car and the finish line. Now instead of calling the a placeholder "end_game" function, now if there is a mask collision, the checkpoint_reached function is called as planned in the algorithm design section, with the parameters of the car, the number of the checkpoints plus 1 (as the finish line will be 1 more than checkpoint 19) and the total number of checkpoints.

Additionally, I created a for loop which loops through every checkpoint and if any of them have a rectangle collision with the car then the checkpoint_reached function will be called with the parameters of the car, the checkpoint number of the checkpoint which was collided with and the total number of checkpoints. This will make it so any collisions between the car and the checkpoint are correctly detected.

```
400     #Rectangle collision detection between the finish line and the car
401     if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
402         #Mask collision detection between the finish line and the car
403         if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
404             if not Car1.new_lap(TotalLaps):
405                 #Car1 has won
406                 checkpoint_reached(Car1,TotalChecks + 1,TotalChecks)
407 
408     for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
409         if pygame.sprite.collide_rect(Car1,CheckArray[i]):
410             checkpoint_reached(Car1,i,TotalChecks)
```

I also made the checkpoint_reached function as designed in the algorithm design section for this stage. This function checks if the number of the collided checkpoint is 1 greater than the car's stored LastCheckpoint attribute, and if it is then it is the correct checkpoint. If it is the correct checkpoint, then if the checkpoint number is equal to the total number of checkpoints, then this must be the finish line and the Car's internal finished_lap() method is called. If it is not the finish line, then the car's next_checkpoint method is called. If it is not the correct checkpoint however, then the Car's wrong_checkpoint method is called.

```
def checkpoint_reached(TheCar,CheckNo,TotalChecks):
    if CheckNo == TheCar.get_checks() + 1: #If the checkpoint is correct
        if CheckNo == TotalChecks: #If this is the finish line
            TheCar.finished_lap()
        else:
            TheCar.next_checkpoint() #If not the finish line then run next_checkpoint
    else:
        TheCar.wrong_checkpoint() #If not the correct checkpoint then run wrong_checkpoint
```

I added the get_checks method into the Car class so that its last checkpoint can be returned (so that is can be used in the checkpoint_reached function)

```
70      def get_checks(self):
71          return self.LastCheckpoint
```

I also created the next_checkpoint and finished_lap methods which were designed in the algorithm design section of this stage. The next_checkpoint method simply increases the checkpoint number by 1 and the finished_lap method increases the lap counter by 1 and sets the sets the LastCheckpoint attribute to 0 so that the next checkpoint needed to be reached is Check1. Additionally, this method also checks if the Car's LapCount is equal to the global TotalLaps variable, and if it is then the

maximum number of laps has been reached which means that the global end_game function can be called to end the game. I also created the wrong_checkpoint method, however currently it does not need to do anything so it does nothing, however it may be needed to be used in the future for the neural network.

```
131     def next_checkpoint(self):  #method to move onto the next checkpoint being required to be reached
132         self.LastCheckpointc += 1
133
134
135     def finished_lap(self, TotalLaps): #Method which increases the number of laps which have been completed.
136         self.LapCount += 1
137         self.LastCheckpoint = 0
138         if self.LapCount >= TotalLaps:
139             end_game(self)
```

I tried running the code; however, I received the following error:

```
    if not Car1.new_lap(TotalLaps):
           ^^^^^^^^^^^^
AttributeError: 'Car' object has no attribute 'new_lap'
```

I realised that this was because the collision detection with the finish line was still using the old new_lap function which I had introduced in stage 2, however I have now replaced that function with the checkpoint_reached and finished_lap functions, so it is no longer needed.

To fix this issue, I removed that line from the collision detection as it was no longer needed:

```
425     #Rectangle collision detection between the finish line and the car
426     if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
427         #Mask collision detection between the finish line and the car
428         if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
429             checkpoint_reached(Car1,TotalChecks + 1,TotalChecks)
```

After attempting to run the code again, I was met with the following issue:

```
File "c:\Users\College\Github\Coursework\Ai racer.py", line 132, in next_checkpoint
  self.LastCheckpointc += 1
  ^^^^^^^^^^^^^^^^^^^^
```

I accidentally misspelt the LastCheckpoint attribute in the car class during the next_checkpoint function. To fix this issue, I removed the extra c at the end of the attribute.

```
131     def next_checkpoint(self):  #method to move onto the next checkpoint being required to be reached
132         self.LastCheckpoint += 1
```

After fixing this, there were no longer any syntax errors. I then introduced the print statement to print “Lap complete” every time the finished_lap method was called to test its functionality:

```
135     def finished_lap(self, TotalLaps): #Method which increases the number of laps which have been completed.
136         print("Lap complete")
137         self.LapCount += 1
138         self.LastCheckpoint = 0
139         if self.LapCount >= TotalLaps:
140             end_game(self)
```

However, when I drove the car across all the checkpoints and reached the finish line, nothing was printed. After reviewing my code, I noticed that when the checkpoint_reached procedure is called by the finish line, it passes in the CheckNo argument as 1 more than the total number of Checkpoints, however in the checkpoint_reached procedure, the finished_lap method is only called if the CheckNo parameter is equal to the total number of checkpoints and not 1 greater. To fix this, I updated the checkpoint_reached function to check if the CheckNo is 1 greater than the total number of checkpoints as intended:

```
def checkpoint_reached(TheCar,CheckNo,TotalChecks):
    if CheckNo == TheCar.get_checks() + 1: #If the checkpoint is correct
        if CheckNo == TotalChecks + 1: #If this is the finish line
            TheCar.finished_lap()
        else:
            TheCar.next_checkpoint() #If not the finish line then run next_checkpoint
    else:
        TheCar.wrong_checkpoint() #If not the correct checkpoint then run wrong_checkpoint
```

I also realised that when the collision detection is done between the checkpoints and the car that the variable i will start incrementing from 0 however the CheckNo's begin from 1. To fix this, I called the checkpoint_reached function with the CheckNo parameter as 1 more than i instead:

```
432     for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
433         if pygame.sprite.collide_rect(Car1,CheckArray[i]):
434             checkpoint_reached(Car1,i + 1,TotalChecks)
```

This fixed the previous issue, and now the finished lap method was able to be called. However, when I called the finished lap method from the checkpoint_reached function, I forgot to include the total number of laps as an argument, however this is a required parameter for the finished_lap method.

```
    TheCar.finished_lap()
    ~~~~~~~~~~~~~~~~~~~~^^
TypeError: Car.finished_lap() missing 1 required positional argument: 'TotalLaps'
```

To fix this, I included the global TotalLaps variable as an argument when the finished_lap method is called:

```
def checkpoint_reached(TheCar,CheckNo,TotalChecks):
    if CheckNo == TheCar.get_checks() + 1: #If the checkpoint is correct
        if CheckNo == TotalChecks + 1: #If this is the finish line
            TheCar.finished_lap(TotalLaps)
```

Now the text “Lap complete” is correctly displayed when the car crosses all the checkpoints and reaches the finish line and it is not displayed if the car reaches the finish line while missing some required checkpoints:

Now the gameplay is fully complete, I decided to no longer print “Lap complete” when completing a lap, and I no longer display all of the checkpoints onto the screen.

The next part of this stage is to create the main menu, settings menu and race complete screen.

To begin, I created the “game_loop” function, which included the majority of code created so far and will make it so that the game can be played from the beginning and reset multiple times without restarting the program:

```
def game_loop():
    """ Class definitions"""

    CarImage = pygame.image.load('Racecar.png') #Sets the Surface object CarImage equal to Car temp.png
    CarImage = pygame.Surface.convert_alpha(CarImage) #Converts that image so that it can contain pixel alphas
```

Next, I created the “menu” function, which will be used to display the main menu screen which was planned in the GUI design. Firstly, I knew that I needed to display text onto the screen, so I looked at pygame’s documentation (https://www.pygame.org/docs/ref/draw.html and pygame.font — pygame v2.6.0 documentation).

I created the following font objects which can be used to display text of varying sizes on the screen. I created the title_font which is a very large font to be used for either the title of the game or the name of the current screen (e.g. the word settings for settings). I decided to do this as it is conventional for a title or heading to be in a larger font than more specific text. Next, I created the very small body_font which will primarily be used to display the text on buttons. I chose this size as the font needed to be smaller than the buttons themselves but large enough so that the text is easily readable for the users. Finally, I created medium_font which is a font in between the other two in size. This font will be used for additional text which needs to be displayed on screen which needs to be larger than the buttons, so it is more easily readable, but smaller than the title so that the title stands out:

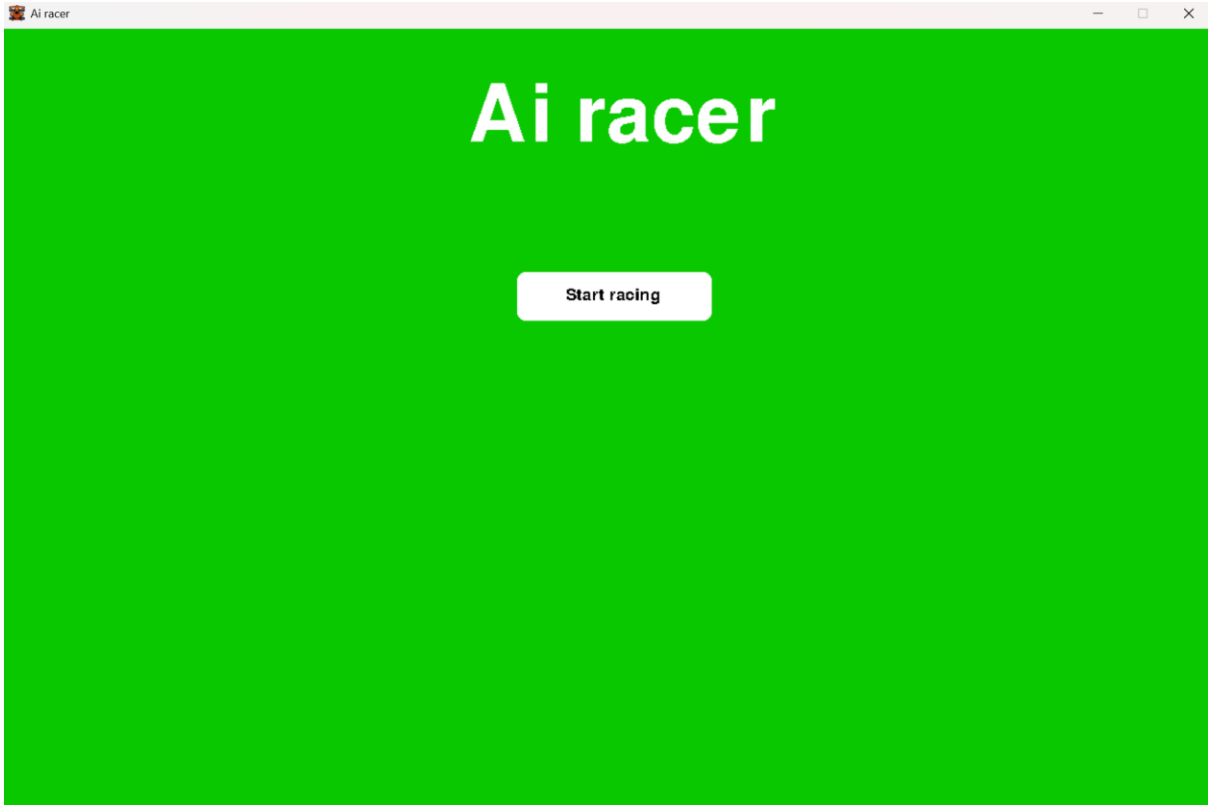
```
pygame.init() #Initialises pygame so its functionality can be used
pygame.font.init() #Creating fonts to be displayed on screen
title_font = pygame.font.SysFont('Aptos', 140)
body_font = pygame.font.SysFont('Aptos', 25)
medium_font = pygame.font.SysFont('Aptos', 63)
```

Next, I created the menu function. Firstly, I began another repeated loop and inside of it the words “Ai racer” which is the name of the game is displayed using the title_font in position (480,50). Next, to create the menu buttons, I created a rectangle of size (200,50) at position (530,250) with a border thickness of 8. Next, I created body_text called Button1Text with the words ‘start racing’ in white as outlined in the GUI design. I then displayed that text 52 pixels to the right and 15 pixels down from the top left position of the button so that the text appears in the middle of the button.

After displaying this on screen, I checked if the X button was pressed, which will quit the program, and for testing purposes, if the R button is pressed then that will return R signalling for the racing to begin and if the S button is pressed this will return S signalling for the settings menu to begin. However, I have designed for the racing to begin when the start racing button is clicked with the mouse and the keyboard pressing is only temporary until this feature is introduced:

```
def menu():
    running = True
    while running: #Repeated loop for the main menu
        screen.fill((10,200,0))
        title = title_font.render('Ai racer', False, (255,255,255)) #Displays the game's title on screen in white
        screen.blit(title,(480,50))

        Button1 = pygame.Rect(530,250,200,50) #Creates a rectangle object with parameters x,y,width,height
        pygame.draw.rect(screen, 'white',Button1,border_radius=8)
        Button1Text = body_font.render('Start racing', False, (0,0,0)) #Creates black text to be displayed on the button
        screen.blit(Button1Text,(Button1.x + 52,Button1.y+ 15)) #Creates the text 52 to the right and 15 pixels down from the button's top left pixel


        for event in pygame.event.get(): #event handling
            if event.type == pygame.QUIT:
                running = False

            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_r: #This means it was the s key
                        running = False
                        return 'R'
                if event.key == pygame.K_s: #This means it was the s key
                        running = False
                        return 'S'


        pygame.display.update()
```

After this, I called the main() function as soon as the program begins and this was displayed on screen:

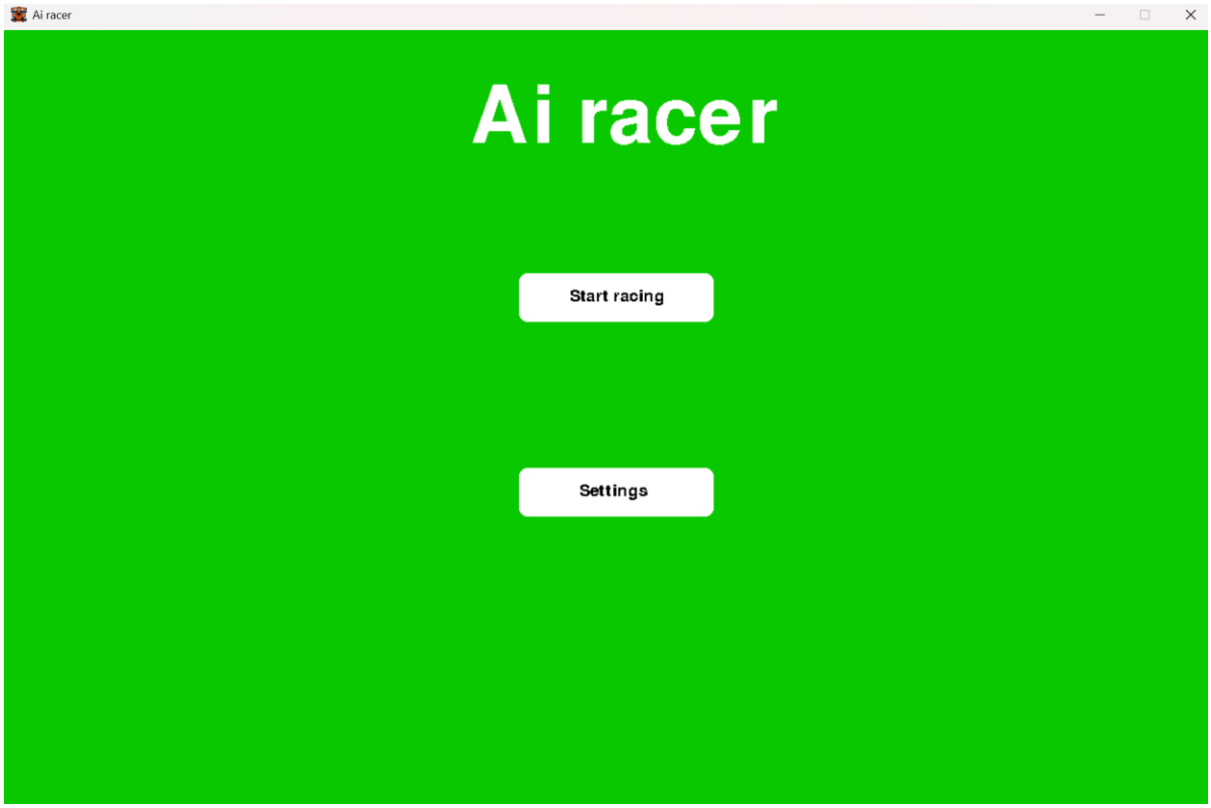


As you can see, the Title text and the button is displayed on screen correctly so I can continue to develop the main menu.

Next, I created Button 2, reusing the code I used to create button 1 (Note that I accidentally called the text Button1Text again, this worked correctly as the program overrode the previous text and displayed the new text after, however I fixed this issue later):

```
Button2 = pygame.Rect(530,450,200,50) #Creates a rectangle object with parameters x,y,width,height
pygame.draw.rect(screen, 'white',Button2,border_radius=8)
Button1Text = body_font.render('Settings', False, (0,0,0)) #Creates black text to be displayed on the button
screen.blit(Button1Text,(Button2.x + 62,Button2.y+ 15)) #Creates the text 62 to the right and 15 pixels down from the button's top left pixel
```

Then I ran the program and the menu looked like this:



The new settings button was displayed correctly, so I moved on to allowing buttons to be pressed.

I wanted to make it so the colour of buttons would change when you hovered over them, as this is conventional for the GUI of games, so I attempted to use the .fill method for surfaces on the buttons to change their colour when they collided with the mouse:

```
38          mousePos = pygame.mouse.get_pos()
39          if Button1.collidepoint(mousePos):
40              Button1.fill('yellow')
41          else:
42              Button1.fill('white')
```

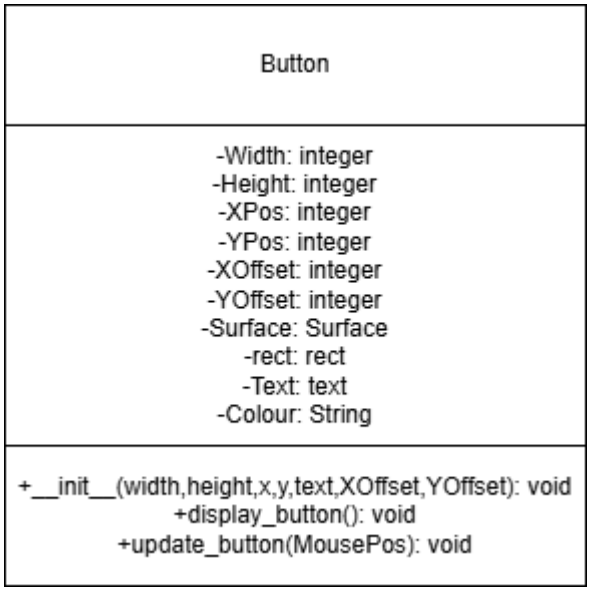
However, that method only works for surface objects and not rect objects which the buttons are, so I was met with this error:

```
    Button1.fill('white')
    ^^^^^^^^^^^^
AttributeError: 'pygame.rect.Rect' object has no attribute 'fill'
```

I decided that my method of creating buttons was likely inefficient as code had to be copy and pasted frequently, so I viewed this tutorial (How to Make Buttons in PyGame -

The Python Code). Here, they created a class for the buttons and then simply reused it which is a highly efficient method to reduce code reuse and to ensure that all the buttons function the same, so I decided to do the same for my buttons.

I created the following Class diagram for the Button class:



The attributes are fairly self-explanatory, with the width and heigh being the width and height of the button, the XPos and YPos being the X and Y positions of the button, The XOffset and YOffset being the offset of the button's text from its top left pixel, the Surface being the Surface of the button (so that the colour of the button can be changed using the .fill method), the rect being the pygame rect of the button, the Text being the instantiated text object for the button and the Colour being the colour of the button.

The __init__ method is the button's constructor which simply sets many of the attributes to passed in parameters and instantiates all attributes. The colour of the button will always be instantiated as white but it is an attribute so that it can be changed to yellow when the mouse collides with it.

The display_button() method will display the button onto the screen, which requires many less lines of code than the previous method without classes.

The update_button() method will check if the mouse is colliding with the button, and if it is then it will make it yellow and if not, it will return it to white. The method will also be used to check if the button has been pressed.

I created the class in code here:

```
def menu():
    class Button():
        def __init__(self,width,height,x,y,text,XOffset,YOffset):
            self.Width = width
            self.Height = height
            self.XPos = x
            self.YPos = y
            self.XOffset = XOffset
            self.YOffset = YOffset

            self.Surface = pygame.Surface((self.Width,self.Height))
            self.Rect = pygame.Rect(self.XPos,self.YPos,self.Width,self.Height)
            self.Text = text
            self.Colour = 'white'

        def display_button(self):
            pygame.draw.rect(screen, self.Colour,self.Rect,border_radius=8) #draws the button onto the screen
            screen.blit(self.Text,(self.XPos +self.XOffset,self.YPos + self.YOffset)) #draws the button's text onto the screen, adjusted by the offset
```

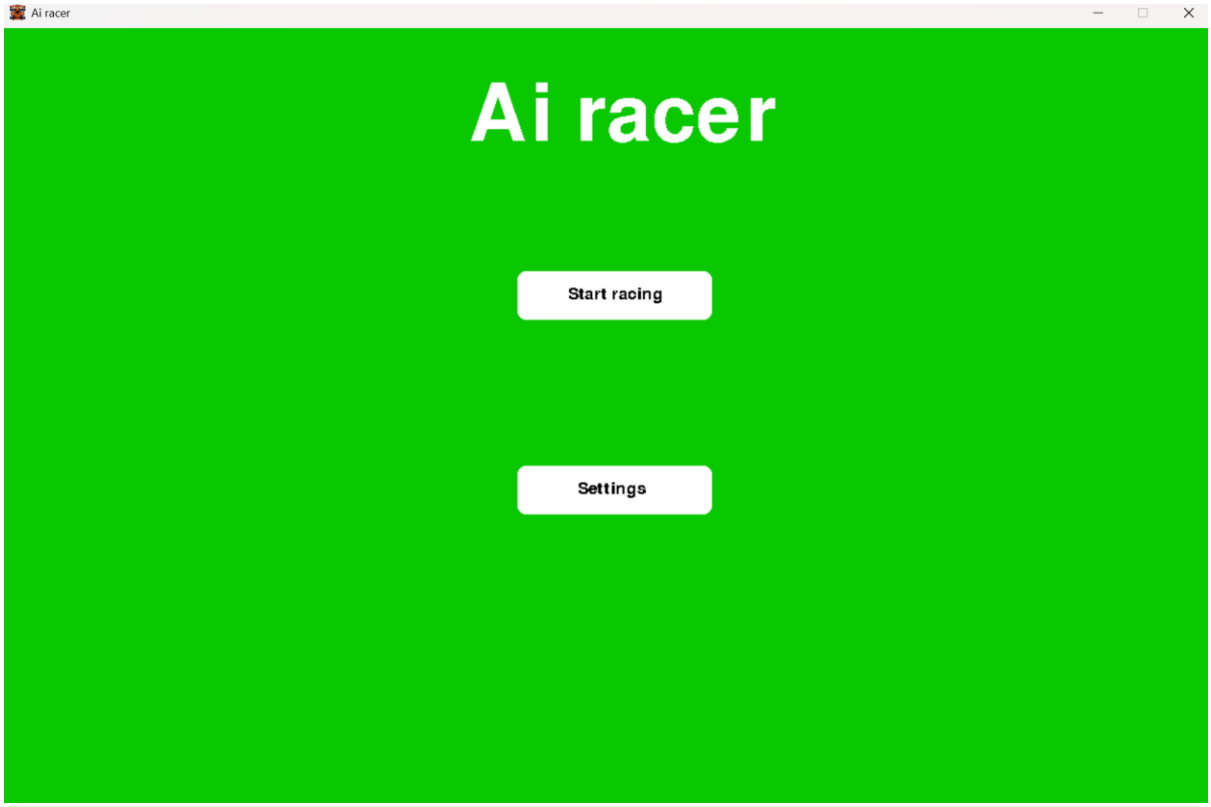
To begin with, I did not create the update_button method so that I could see if I could get the same functionality as before with the button class instead of also introducing a new feature.

In the while running loop for the main menu, I wrote the following code to create button 1 and 2:

```
        Button1Text = body_font.render('Start racing', False, (0,0,0)) #Creates black text to be displayed on the button

        Button1 = Button(200,50,530,250,Button1Text,52,15)

        Button1.display_button()


        Button2Text = body_font.render('Settings', False, (0,0,0)) #Creates black text to be displayed on the button
        Button2 = Button(200,50,530,450,Button2Text,62,15)

        Button2.display_button()
```

As you can see, this is many less lines of code than the previous method which means that using classes for the buttons is a much more maintainable and easier to read.

After running the code, the main menu looked like this:



The main menu functions the same as it did when not using classes, so now I can move on to adding additional functionality. Next, I created the update_button method. I used How to Make Buttons in PyGame - The Python Code to learn how to detect mouse clicks, however I modified their example so that it would function better for my program:

```
def menu():
    class Button():
        def __init__(self,width,height,x,y,text,XOffset,YOffset):
            self.Width = width
            self.Height = height
            self.XPos = x
            self.YPos = y
            self.XOffset = XOffset
            self.YOffset = YOffset

            self.Surface = pygame.Surface((self.Width,self.Height)) #Creates surface for the button
            self.Rect = pygame.Rect(self.XPos,self.YPos,self.Width,self.Height) #Creates rectangle for the button
            self.Text = text
            self.Colour = 'white'

        def display_button(self):
            screen.blit(self.Surface,self.Rect) #Displays the button's rectangle and surface to the screen
            screen.blit(self.Text,(self.XPos +self.XOffset,self.YPos + self.YOffset)) #draws the button's text onto the screen, adjusted by the offset

        def update_button(self,MousePos):
            self.Surface.fill('white') #By default the button is white
            if self.Rect.collidepoint(MousePos): #If the button and mouse collide
                self.Surface.fill('yellow') #The button is yellow
                if pygame.mouse.get_pressed(num_buttons=3)[0]: #If the mouse's left click is pressed
                    return True #Return true meaning the button was pressed
                else:
                    return False #Return False meaning the button was not pressed
            else:
                return False
```

This method will be run every frame, and it makes the button white by default, then the method checks if the mouse is colliding with the button, and if so, it fills the button in yellow. If the button has been pressed, then the function returns true, and if the button is not pressed, the function returns false. I did this because the exact functionality which occurs when the button is pressed is different for every button so the most that

can be done in the general class is check if the button has been pressed and return true or false for the menu itself to decide what then should be done for the button.

Now, in the while running loop for the menu, I renamed button1 and button 2 to RaceButton and SettingsButton so that they are more descriptive of their purpose.

After the race button and the settings button are created, the position of the mouse is found, and the update_button() method is called for both buttons. If the RaceButton is pressed, then R is returned by the main menu function and if the SettingsButton is pressed, then S is returned by the main menu function so that the program will know which screen should be displayed next. If the quit button is pressed the Q is returned instead which will quit the program.

```
while running: #Repeated loop for the main menu
    screen.fill((30,200,0))
    title = title_font.render('AI Racer', False, (255,255,255)) #Displays the game's title on screen in white
    screen.blit(title,(450,60))

    RaceButtonText = body_font.render('Start racing', False, (0,0,0)) #Creates black text to be displayed on the button
    RaceButton = Button(200,50,530,450,RaceButtonText,52,15)

    SettingsButtonText = body_font.render('Settings', False, (0,0,0)) #Creates black text to be displayed on the button
    SettingsButton = Button(200,50,530,650,SettingsButtonText,62,15)


    MousePos = pygame.mouse.get_pos() #Gets the mouse's position
    if RaceButton.update_button(MousePos): #If button 1 was pressed
        return 'R' #Returns R to begin the race
    if SettingsButton.update_button(MousePos): #If button 2 was pressed
        return 'S' #Returns S to go to settings

    for event in pygame.event.get(): #If the X button is pressed, the game closes
        if event.type == pygame.QUIT:
            running = False

    RaceButton.display_button()
    SettingsButton.display_button()
    pygame.display.update()
return 'Q' #returns Q to quit the game
```

Next, I created the system outside of all functions which decided which screens should be run. I did this following the flowchart outlined in algorithm design:

```
532    next_process = menu()
533    running = True
534    while running:
535
536        if next_process == 'R':
537            next_process = game_loop()
538        elif next_process == 'S':
539            next_process = settings()
540        elif next_process == 'O':
541            next_process = results()
542        elif next_process == 'M':
543            next_process == menu()
544        elif next_process == 'Q':
545            running = False
546    pygame.quit()
```

Here, the menu is immediately run and then the next_process variable is set to whatever character is returned from it. Then, a while running loop repeatedly checks what the next_process variable is set to and runs that subroutine and if next_process is Q, then pygame quits. I used the letter O for the results menu as R was already taken, and O could represent the word “over” as the game would be over at the time the results menu is run.

Now the main menu is complete, I decided to create the results menu.

Firstly, I updated the finished_lap method in the Car class in the game loop so that if the game is over, then the game loop returns ‘O’ which will cause the results screen to display, and the method will return ‘C’ otherwise to tell the game loop to continue:

```
def finished_lap(self, TotalLaps): #Method which increases the number of laps which have been completed.
    print("Lap complete")
    self.LapCount += 1
    self.LastCheckpoint = 0
    if self.LapCount >= TotalLaps:
        return 'O' #returns O meaning the game is over and the results screen should now be displayed
    return 'C' #returns C meaning the game should continue
```

Here is the updated checkpoint_reached function which now either returns the output of the finished_lap function if a lap has been completed (which will be O or C) or it returns C automatically if a lap has not been finished:

```
363         def checkpoint_reached(TheCar,CheckNo,TotalChecks):
364             if CheckNo == TheCar.get_checks() + 1:
365                 if CheckNo == TotalChecks + 1:
366                     return TheCar.finished_lap(TotalLaps)
367                 else:
368                     TheCar.next_checkpoint()
369                     return 'C'
370             else:
371                 TheCar.wrong_checkpoint()
372                 return 'C'
```

And finally, here is the collision detection with the car and the checkpoints where, if the result of the checkpoint reached function is O, then O is returned by the game loop, or else if it is C the game continues.

```
544
545         for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
546             if pygame.sprite.collide_rect(Car1,CheckArray[i]):
547                 result = checkpoint_reached(Car1,i + 1,TotalChecks)
548                 if result == 'O':
549                     return 'O'
```

After this has been implemented in the game loop, once the car completes all laps, the game should open the results menu. This means that I can now create the results() function which runs the results menu.

Here is the button class inside the results menu which has been copied from the main menu:

```
84  def results():
85      class Button():
86          def __init__(self,width,height,x,y,text,XOffset,YOffset):
87              self.Width = width
88              self.Height = height
89              self.XPos = x
90              self.YPos = y
91              self.XOffset = XOffset
92              self.YOffset = YOffset
93
94              self.Surface = pygame.Surface((self.Width,self.Height)) #Creates surface for the button
95              self.Rect = pygame.Rect(self.XPos,self.YPos,self.Width,self.Height) #Creates rectangle for the button
96              self.Text = text
97              self.Colour = 'white'
98
99          def display_button(self):
100             screen.blit(self.Surface,self.Rect) #Displays the button's rectangle and surface to the screen
101             screen.blit(self.Text,(self.XPos +self.XOffset,self.YPos + self.YOffset)) #draws the button's text onto the screen, adjusted by the offset
102
103         def update_button(self,MousePos):
104             self.Surface.fill('white') #By default the button is white
105             if self.Rect.collidepoint(MousePos): #If the button and mouse collide
106                 self.Surface.fill('yellow') #The button is yellow
107                 if pygame.mouse.get_pressed(num_buttons=3)[0]: #If the mouse's left click is pressed
108                     return True #Return true meaning the button was pressed
109                 else:
110                     return False #Return False meaning the button was not pressed
111             else:
112                 return False
```

Here is the while running loop inside the results menu which is very similar to the main menu. Here, the words “race finish” are displayed as a title onto the screen. Next, the buttons, RaceButton (which allows the user to begin a new race) and MenuButton (which allows the user to return to the menu) have been created on screen, and then if they are clicked they return either R to begin racing or M to return to the menu.

This was very quick to implement as I reused the design from the main menu.

```
    running = True
    while running:
        screen.fill((10,200,0))
        title = title_font.render('Race finish', False, (255,255,255)) #Displays the words 'Race finish' at the top of the screen
        screen.blit(title,(480,50))

        RaceButtonText = body_font.render('New race', False, (0,0,0)) #Creates black text to be displayed on the button
        RaceButton = Button(200,50,530,250,RaceButtonText,52,15)

        MenuButtonText = body_font.render('Settings', False, (0,0,0)) #Creates black text to be displayed on the button
        MenuButton = Button(200,50,530,450,MenuButtonText,62,15)

        MousePos = pygame.mouse.get_pos() #Gets the mouse's position
        if RaceButton.update_button(MousePos): #If button 1 was pressed
            return 'R' #Returns R to begin the race
        if MenuButton.update_button(MousePos): #If button 2 was pressed
            return 'M' #Returns M to go to the menu

        for event in pygame.event.get(): #If the X button is pressed, the game closes
            if event.type == pygame.QUIT:
                running = False

        RaceButton.display_button()
        MenuButton.display_button()
        pygame.display.update()
    return 'Q'
```

I attempted to run the program, the main menu displayed correctly and the “start racing” button on the main menu worked and began the game loop. However, when I completed the maximum number of laps, the results screen was never reached.

To try and find the place in my code where the error occurred, I added the print statement “Final lap done” in the finished_lap function so that I could see if this piece of code is ever reached when running the game:

```
def finished_lap(self, TotalLaps): #Method which increases the number of laps which have been completed.
    print("Lap complete")
    self.LapCount += 1
    self.LastCheckpoint = 0
    if self.LapCount >= TotalLaps:
        print("Final lap done")
        return 'O' #returns O meaning the game is over and the results screen should now be displayed
    return 'C' #returns C meaning the game should continue
```

I also did the same in the checkpoint_reached function where the words “checkpoint reached final round” are printed if O is being returned by the game loop.

```
386     def checkpoint_reached(TheCar,CheckNo,TotalChecks):
387         if CheckNo == TheCar.get_checks() + 1: #If the checkpoint is correct
388             if CheckNo == TotalChecks + 1: #If this is the finish line
389                 result = TheCar.finished_lap(TotalLaps) #Run the finished lap subroutine
390                 if result == 'O': #If the game is over
391                     print("checkpoint reached final round")
392                     return 'O'
393                 else:
394                     return 'C'
395             else:
396                 TheCar.next_checkpoint() #If not the finish line then run next_checkpoint
397                 return 'C'
398         else:
399             TheCar.wrong_checkpoint() #If not the correct checkpoint then run next_checkpoint
400             return 'C'
```

When running the program and completing the total number of laps, these lines were never printed which means that the error must have come before them.

I then realised that I never included the return ‘O’ statement when there is a collision between the car and the finish line (which obviously is the only time that a lap could finish) which means that the game loop is never able to finish.

```
567         #Rectangle collision detection between the finish line and the car
568         if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
569             #Mask collision detection between the finish line and the car
570             if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
571                 checkpoint_reached(Car1,TotalChecks + 1,TotalChecks)
572 
573         for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
574             if pygame.sprite.collide_rect(Car1,CheckArray[i]):
575                 result = checkpoint_reached(Car1,i + 1,TotalChecks)
576                 if result == 'O':
577                     return 'O'
```

I then fixed this as follows:

```
567         #Rectangle collision detection between the finish line and the car
568         if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
569             #Mask collision detection between the finish line and the car
570             if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
571                 result = checkpoint_reached(Car1,TotalChecks + 1,TotalChecks)
572                 if result == 'O':
573                     return 'O'
574 
575         for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
576             if pygame.sprite.collide_rect(Car1,CheckArray[i]):
577                 result = checkpoint_reached(Car1,i + 1,TotalChecks)
578                 if result == 'O':
579                     return 'O'
```

However, even after implementing this, the results screen still was not reached. When I ran the program and completed the total number of laps, the following text was displayed:

```
Lap complete
Lap complete
Lap complete
Final lap done
checkpoint reached final round
going to results screen
```

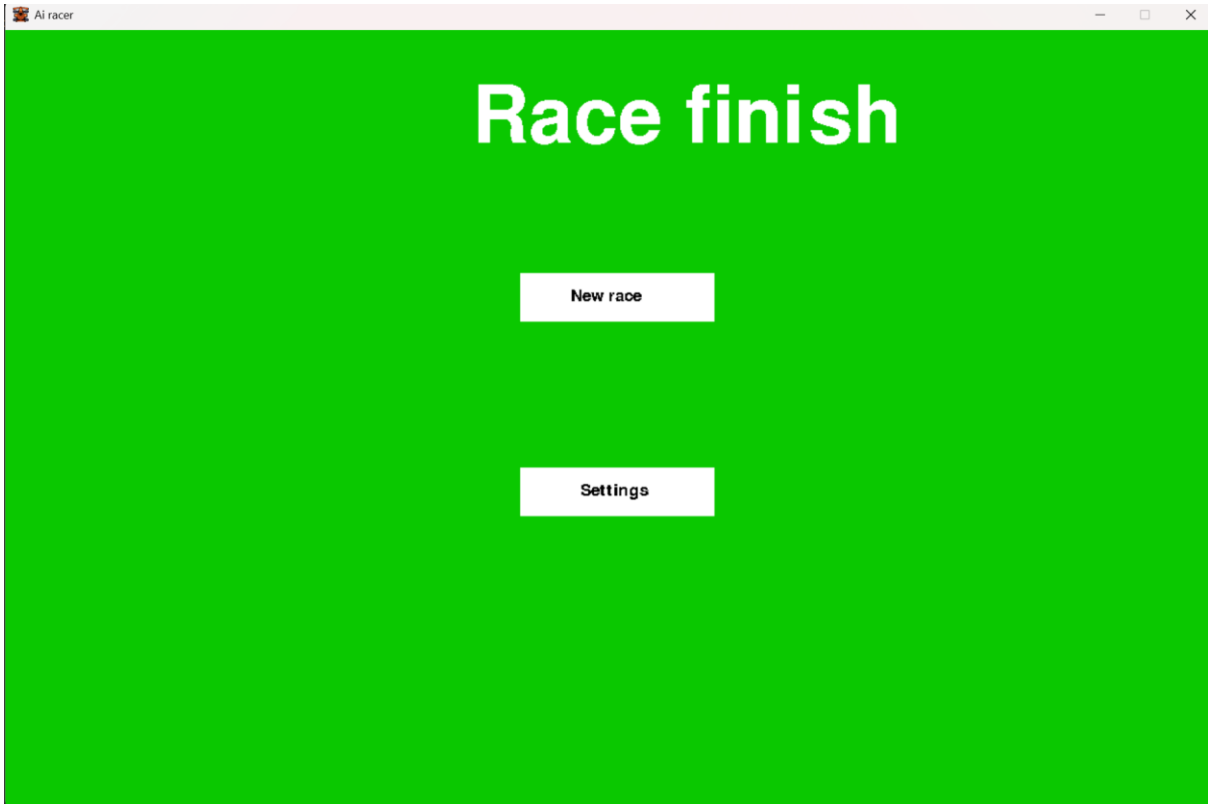
Every single print statement that I have included was displayed correctly, which means that the game loop works correctly when deciding when to return to the results screen so the error must be outside of the game loop.

To confirm this, I added a manual call of the results screen

```
567         #Rectangle collision detection between the finish line and the car
568         if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
569             #Mask collision detection between the finish line and the car
570             if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
571                 result = checkpoint_reached(Car1,TotalChecks + 1,TotalChecks)
572                 if result == 'O':
573                     print("going to results screen")
574                     results()
575                     return 'O'
```

After I manually sent it to the results screen it worked, therefore there is an issue with going between the main menu and the results screen.

Here is what the results screen looks like when it was reached:



To help identify where the error is, I inserted the following print statement into the code which switches which game state is run:

```
609    while running:
610
611        if next_process == 'R':
612            next_proccess = game_loop()
613            print(next_proccess)
614        if next_process == 'S':
615            next_proccess = settings()
616        if next_proccess == 'O':
617            next_process = results()
618        if next_process == 'M':
619            next_proccess == menu()
620        if next_proccess == 'Q':
621            running = False
622            pygame.quit()
```

After also replacing the elifs with just ifs, some of the transitions between states seemed to work correctly for now. However, I was unsure why this happened and assumed this was a temporary fix which would not work for all transitions.

After opening the settings menu, the following error occurred:

```
    LapsText = Medium_font.render('Total laps: ' + TotalLaps,False,(255,255,255))
                                                   ^^^^^^^^^
UnboundLocalError: cannot access local variable 'TotalLaps' where it is not associated with a value
```

The error states that it cannot access the local variable "TotalLaps" however this is actually a global variable which stores the total number of laps to be played in the game. To make it so that the TotalLaps variable can be accessed in the settings subroutine, I created the global subroutines get_totalLaps() and set_totalLaps():

```
475     def get_totalLaps():
476         return TotalLaps
477     def set_totalLaps(newValue):
478         TotalLaps = newValue
```

I also updated the code for the settings menu to use these subroutines instead of attempting to directly access the TotalLaps variable:

```
if LapIncreaseButton.update_button(MousePos): #If button 1 was pressed
    if get_totalLaps() <10:
        set_totalLaps(get_totalLaps + 1)
if LapDecreaseButton.update_button(MousePos): #If button 1 was pressed
    if get_totalLaps() >1:
        set_totalLaps(get_totalLaps) - 1
```

Using these fixed the previous error. However, when attempting to run the code I got this error instead:

```
    LapsText = Medium_font.render('Total laps: ' + get_totalLaps(),False,(255,255,255))
                                                   ~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

I realised that I was trying to concatenate the string "Total laps:" and the integer of the total number of laps. To fix this issue, I used the str() function to turn the integer into a string:

```
LapsText = Medium_font.render('Total laps: ' + str(get_totalLaps()),False,(255,255,255))
screen.blit(LapsText,(480,120))
```

After doing this, there were no longer any syntax errors.

However, now when I pressed any button to transition to another screen, the game froze and was unable to be interacted with at all.

To try and find the issue, I wrote more print statements at all of the transition points between states to see when the program attempted to open a new state:

```
691  next_process = menu()
692  running = True
693  while running:
694      print(next_process)
695      if next_process == 'R':
696          print("Game loop")
697          next_proccess = game_loop()
698      if next_process == 'S':
699          print("Settings")
700          next_proccess = settings()
701      if next_proccess == 'O':
702          print("Results")
703          next_process = results()
704      if next_process == 'M':
705          print("Menu")
706          next_proccess == menu()
707      if next_proccess == 'Q':
708          running = False
709          pygame.quit()
710  pygame.quit()
```

Now, when running the code and pressing the “Start racing” button, the following text was printed:

```
M
Results
Menu
M
Results
Menu
M
```

This made it seem like the game was continuously switching between the main menu and the results screen instead of loading one and staying in it when pressing the “main menu” button on the results screen.

After inspecting the code, I noticed that I had accidentally misspelt next_process multiple times to write next_proccess instead which meant that multiple variables were in use to decide which screen should be run when it should only be stored in a single variable. To fix this, I renamed every instance of next_proccess to next_process:

```
691  next_process = menu()
692  running = True
693  while running:
694      print(next_process)
695      if next_process == 'R':
696          print("Game loop")
697          next_process = game_loop()
698      if next_process == 'S':
699          print("Settings")
700          next_process = settings()
701      if next_process == 'O':
702          print("Results")
703          next_process = results()
704      if next_process == 'M':
705          print("Menu")
706          next_process == menu()
707      if next_process == 'Q':
708          running = False
709          pygame.quit()
710  pygame.quit()
```

The transitioning between states was still not working after this, so I reread this section of code once again. I noticed that in line 706, I used a == when instead this should be a single = to assign the value of next_process:

```
next_process == menu()
```

After changing this to a single =, the code still could not open the settings menu from the main menu and produced the following output when I attempted to do so:

```
M
Menu
S
Settings
M
Menu
S
Settings
M
Menu
S
Settings
M
```

Additionally, one time I was able to press the settings button and end up on the settings menu; however, this was not consistent to do. I then noticed that I had placed the button to return to the main menu on the settings menu at the same point that I had placed the button to go to the settings menu from the main menu. This made me realise that when I pressed the button to go to the settings menu, that would also press the button on the settings menu, and this would repeat continuously.

To fix this, I decided to add a time delay after pressing a button before the next one can be pressed. This would mean that after pressing a button, another one, or the same one is not immediately pressed.

An alternative method I could have used would be to press a button the first time that it is clicked and only count a second click once the mouse has let go of the left click and then clicked it again, however this is a more complicated process and it would have similar results to the time delay method so I decided against it.

Firstly, I created a global variable called LastButtonPress which will store the time on the system's clock that the last button was pressed.

```
16  LastButtonPress = 0 #Is set equal to the current time using the system's clock
```

Then, I made the function get_LastButtonPress() which returns the time on the clock that the last button was pressed and I created the procedure update_LastButtonPress() which sets the LastButtonPress to be equal to the current time on the systems clock:

```
38    def get_LastButtonPress(): #returns the last time a button was pressed
39        return LastButtonPress
40
41    def update_LastButtonPress(): #sets the last time a button was pressed to the current time
42        global LastButtonPress
43        LastButtonPress = time.perf_counter()
```

This will make it so that I am able to check if enough time has passed since a button was pressed so that a new one can be.

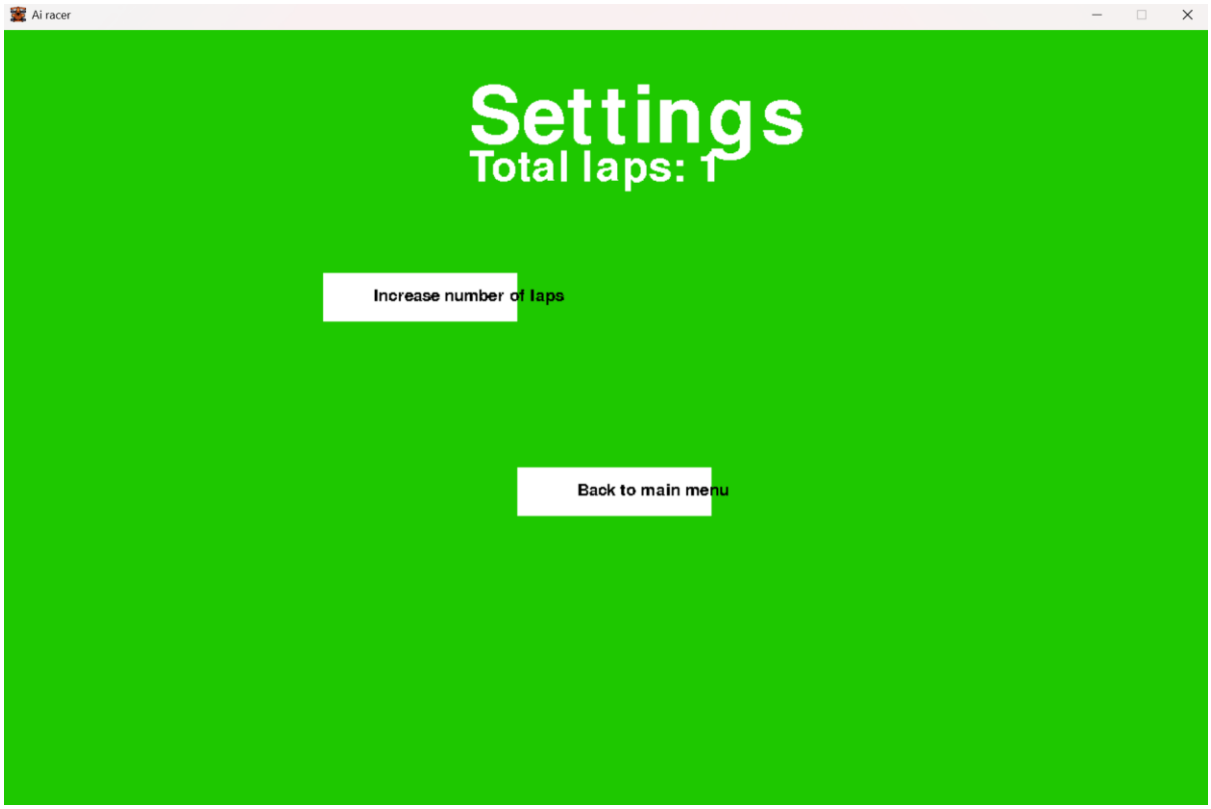
I modified the update_button subroutine in the button class so that it only returns true if 0.0165 seconds have passed since the last button press. This will also call the update_LastButtonPress() subroutine as the current time will be the new time that a button was last pressed:

```
def update_button(self,MousePos):
    self.Surface.fill('white') #By default the button is white
    if self.Rect.collidepoint(MousePos): #If the button and mouse collide
        self.Surface.fill('yellow') #The button is yellow
        if pygame.mouse.get_pressed(num_buttons=3)[0]: #If the mouse's left click is pressed
            if time.perf_counter() > get_LastButtonPress() + 0.0165000000:
                update_LastButtonPress()
                return True #Return true meaning the button was pressed



    return False #Return False meaning the button was not pressed
```

This fixed the issue. However, it still switched between game states very quickly, so I modified the time to 0.8 seconds:

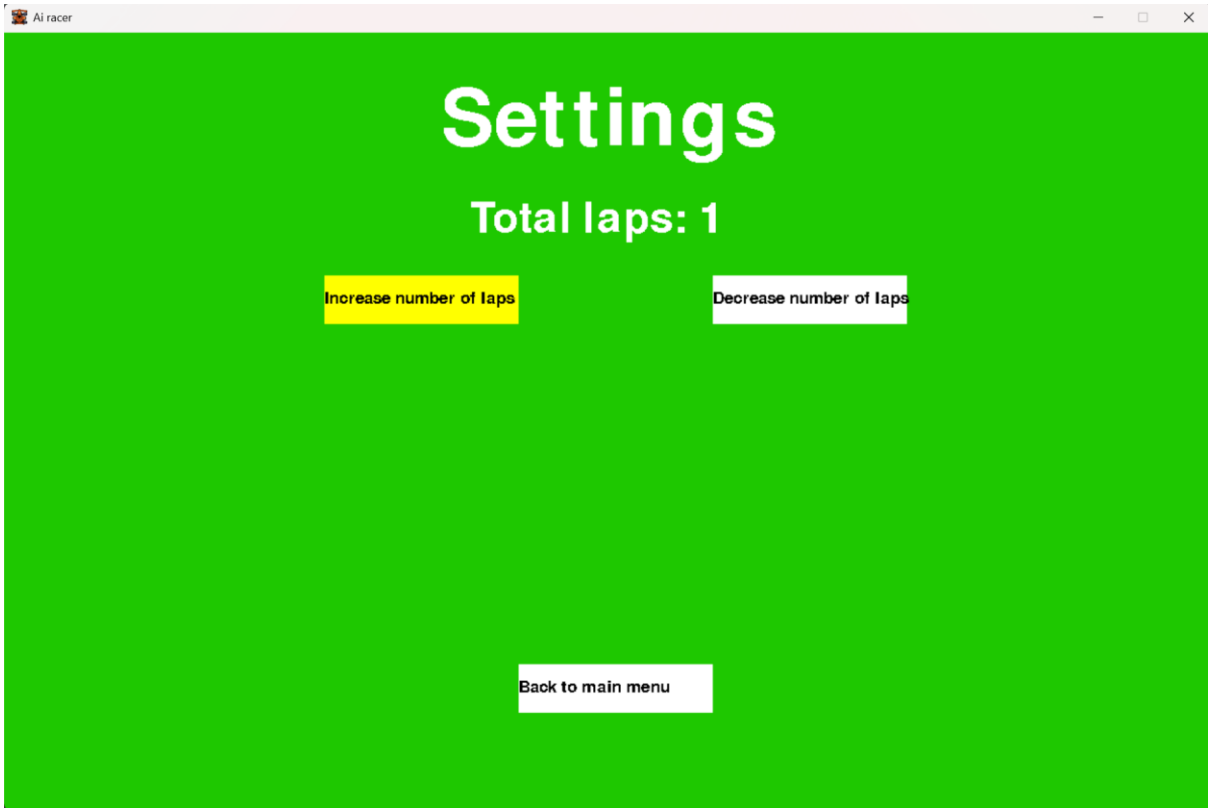
```
66          def update_button(self,MousePos):
67              self.Surface.fill('white') #By default the button is white
68              if self.Rect.collidepoint(MousePos): #If the button and mouse collide
69                  self.Surface.fill('yellow') #The button is yellow
70                  if pygame.mouse.get_pressed(num_buttons=3)[0]: #If the mouse's left click is pressed
71                      if time.perf_counter() > get_LastButtonPress() + 0.8000000000:
72                          update_LastButtonPress()
73                          return True #Return true meaning the button was pressed
74
75
76
77              return False #Return False meaning the button was not pressed
```

Now all buttons correctly opened the correct game state. Now, when I reached the settings screen, it looked like this:



I accidentally placed the lap increase button and the lap decrease button in the same position, so I modified all the positions of the elements. Additionally, I updated the XOffset and YOffset values for the buttons so that the text was correctly displayed.

Note that my mouse cursor was hovering over the “increase number of laps” button which is why it was highlighted yellow as intended:



I attempted to press the “increase number of laps” button and I was met with this error:

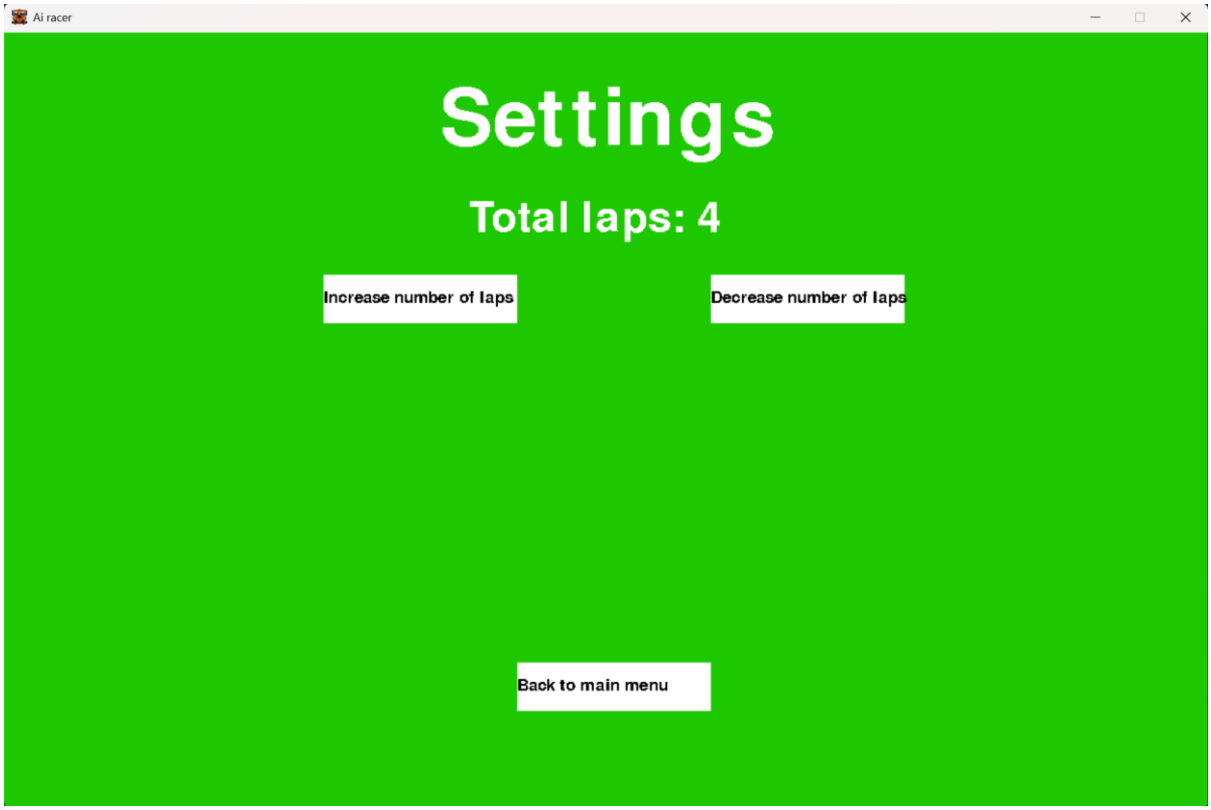
```
    set_totalLaps(get_totalLaps + 1)
                  ~~~~~~~~~~~~~~~^~~
TypeError: unsupported operand type(s) for +: 'function' and 'int'
```

I realised that this was because I had called the get_totalLaps function without adding in brackets. I fixed that as follows:

```
165         if LapIncreaseButton.update_button(MousePos): #If the lap increase button was pressed
166             if get_totalLaps() <10:
167                 set_totalLaps(get_totalLaps() + 1)
168         if LapDecreaseButton.update_button(MousePos): #If the lap decrease button was pressed
169             if get_totalLaps() >1:
170                 set_totalLaps(get_totalLaps() - 1)
```

Now, pressing the increase and decrease number of laps buttons work correctly and update the variable.

Here is an example where I pressed the increase number of laps button until it reached 4:



Now all the essential features of this stage have been completed.

At this point during development, I had spent much more time on stages 1,2 and 3 than I had initially planned for. This was because my initial plans for these stages had seemed simple, however during development I discovered that in order to achieve their success criteria, additional features needed to be created that I had not considered, such as the necessity to implement a set frame rate system so that the car's speed values could remain constant, the many factors which needed to be included with the movement of the car so that it was able to drive smoothly, and the fixes for certain issues which required extensive debugging to figure out.

This meant that, in this stage of development, there was not enough time remaining with this project to both implement and train a neural network manually. Considering this, I made the decision to instead of leaving the game in its current state and rushing to create a neural network which likely could not be completed in time, I will improve upon the game which has been created so far and create additional features which are important to stakeholders, so that the game is in the most enjoyable and completed state possible. I will also cover this decision in more detail during the evaluation stage of my project.

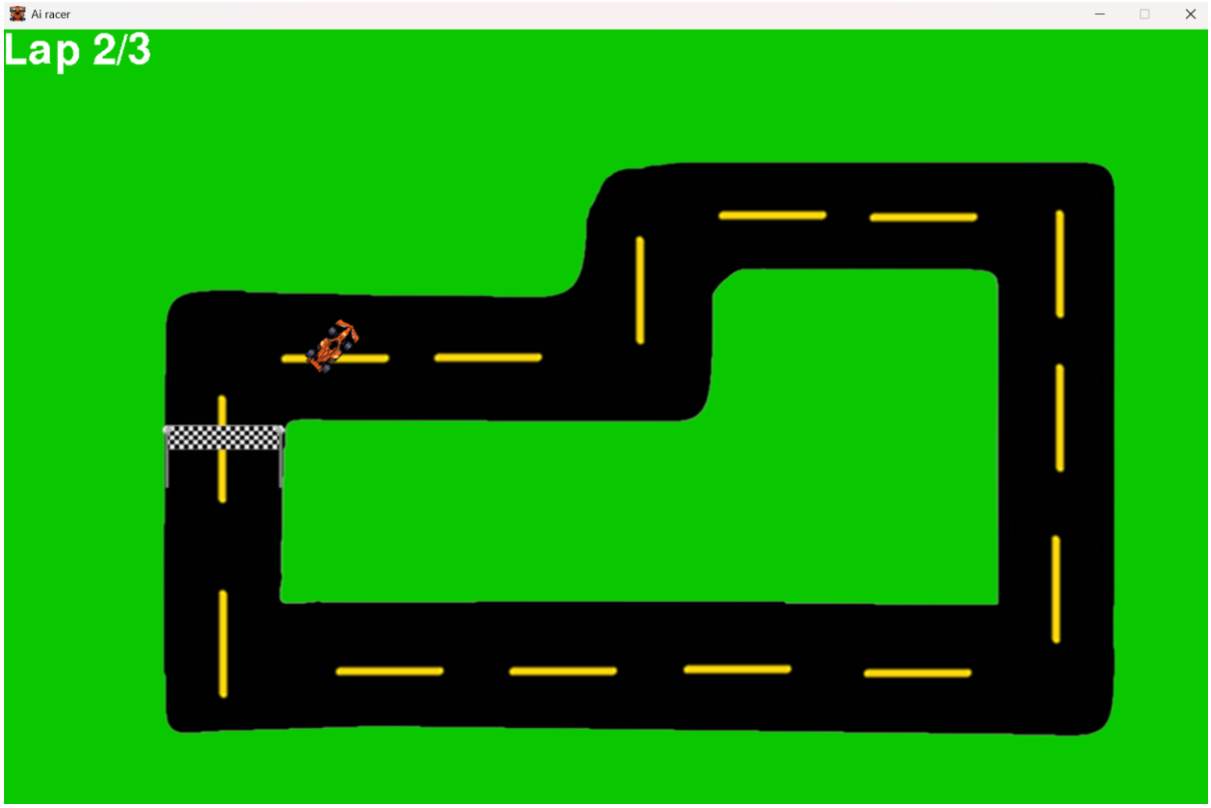
I believe that this is a better approach than moving onto stage 4, as even if that stage is able to be completed, without the entirety of stage 5 the neural network will be unable to be trained and therefore it will simply make random decisions and drive across the track randomly which is not a useful feature to have, nor will it be enjoyable or informative to my users. Whereas, improving upon the stages which have already been developed will result in the game being a fully operational project which will be enjoyable for the users to engage with.

Now, I decided to implement some of the non-essential features which are relevant to this stage. Firstly, I decided to implement the lap counter which tracks the number of laps which have been completed during the game loop. I chose to implement this feature as it would be greatly beneficial to the player as it will visually display to them their progress in the race as well as giving the player a clear indication if it counted as a lap when they reach the finish line. This is because the lap counter will update if the player reached all the checkpoints and the lap counter will not update if the player did not reach all the checkpoints.

I used the medium font to create the lap text in the game loop which uses the Car's get_laps() method and the global get_totaLaps() method so that the laps which the car have completed and the total number of laps is displayed:

```
696         LapsText = Medium_font.render('Lap ' + str(Car1.get_laps()) + "/" + str(get_totalLaps()),False,(255,255,255))
697         screen.blit(LapsText,(0,0))
```

This is what the lap counter looks like in the game when 2 laps have been completed:



Another feature to be displayed on the screen during the race is the race timer. This feature is important to be implemented as it is displayed both during the race and on the results screen and it allows the user to compare how well they performed on different races.

To do this, I created the variable TimerStart variable which records the exact time when the game loop began using the system's clock. I also used python's round function to remove some of the unnecessary decimal places:

```
571     TimerStart = time.perf_counter()
```

The TimerDisplay is updated every time that the game loop repeats, and it is set to the current time – TimerStart. This will record how much time has passed since the game loop began. I also used the medium font to display the TimerDisplay variable onto the screen.

```
701     TimerDisplay = round(time.perf_counter() - TimerStart,3) #Finds the difference between the current time and the time that the game started and rounds it to 1 millisecond
702     TimerText = Medium_font.render('Timer: ' + str(TimerDisplay),False,(255,255,255)) #Displays the text of the timer on the screen
703     screen.blit(TimerText,(975,0)) #Displays the timer in the top right of the screen
```

After this, I decided to create an additional setting to be modified in the settings menu. The setting will allow the user to toggle whether the checkpoints in the game will be displayed onto the screen on and off. I decided that this was important to make as the game will be hard to play if you do not know where the checkpoints are on the track so

the “Display checkpoints” setting will allow the user to view the location of all of the checkpoints and then turn it off when they have finished. Firstly, I created the DisplayCheckpoints variable which will store a Boolean value which represents if the checkpoints will be displayed or not.

```
17    DisplayCheckpoints = False
```

I created the switch_DisplayCheckpoints global procedure which will be used to change the DisplayCheckpoints variable from True to False and from False to True.

```
47  def switch_DisplayCheckpoints():
48      global DisplayCheckpoints
49      DisplayCheckpoints = not DisplayCheckpoints #Toggles the checkpoints from being displayed on screen to not
```

I then created the get_DisplayCheckpoints function which will return the current value of the DisplayCheckpoints variable to see if they are currently being displayed.

```
51  def get_DisplayCheckpoints(): #getter for the display checkpoints variable
52      global DisplayCheckpoints
53      return DisplayCheckpoints
```

At the end of the game loop, I added the following lines of code which display all of the checkpoints onto the screen only if the DisplayCheckpoints variable is True.

```
704         if DisplayCheckpoints:
705             for i in range(TotalChecks): #Displays every checkpoint on the screen so that the user can understand their positions
706                 CheckArray[i].display_checkpoint()
```

I then added a ToggleCheckpointDisplayButton onto the settings menu which, when pressed, will make it so the checkpoints start being displayed or stop being displayed.

```
177     ToggleCheckpointDisplayButtonText = body_font.render('Toggle checkpoint display', False, (0,0,0)) #Creates black text to be displayed on the button
178     ToggleCheckpointDisplayButton = Button(230,50,530,350,ToggleCheckpointDisplayButtonText,0,15)
```

I then added the following lines of code also into the settings menu which will display the text “Checkpoints are being displayed” if they are currently being displayed and “Checkpoints are not being displayed” if they are not currently being displayed.

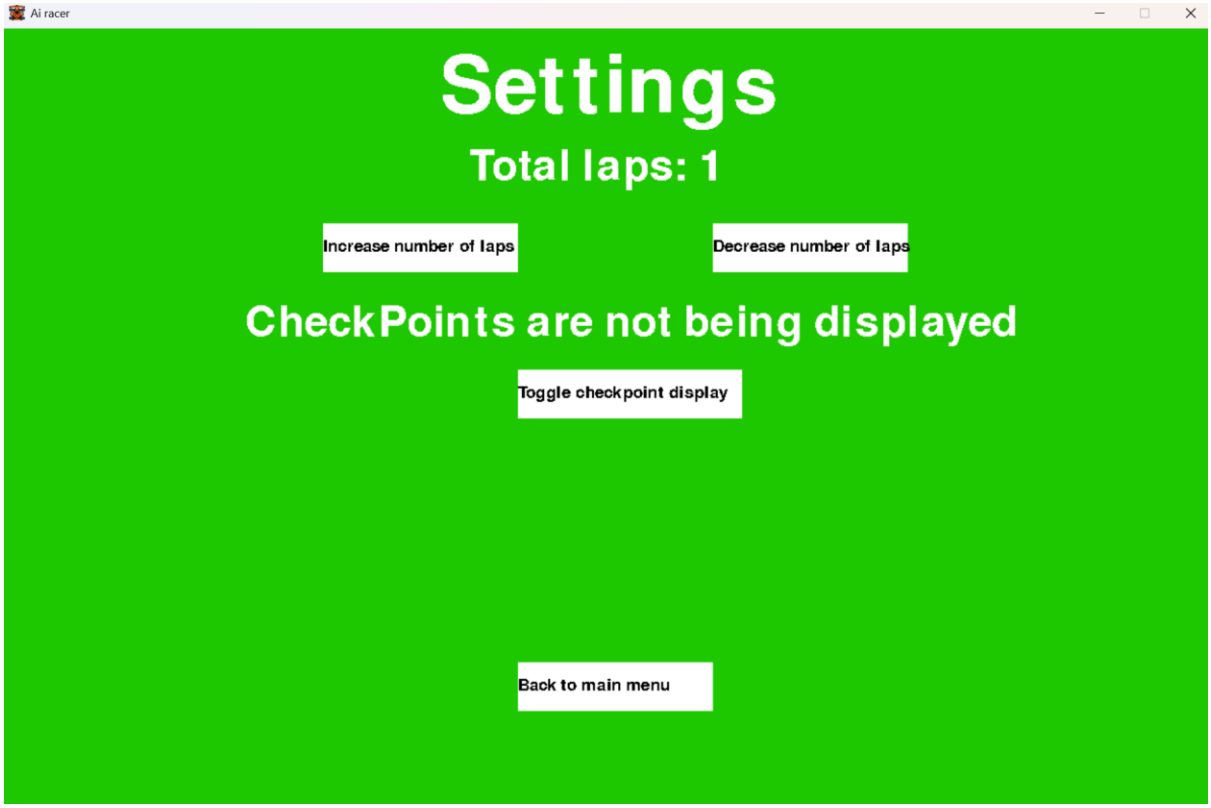
```
164         if get_DisplayCheckpoints():
165             DisplayCheckpointText = Medium_font.render('CheckPoints are being displayed',False,(255,255,255))
166             screen.blit(DisplayCheckpointText,(250,280))
167         else:
168             DisplayCheckpointText = Medium_font.render('CheckPoints are not being displayed',False,(255,255,255))
169             screen.blit(DisplayCheckpointText,(250,280))
```

I also called the display_button method for the ToggleCheckpointDisplayButton and the update_button method and if the button is pressed then the switch_DisplayCheckpoints() method is called:

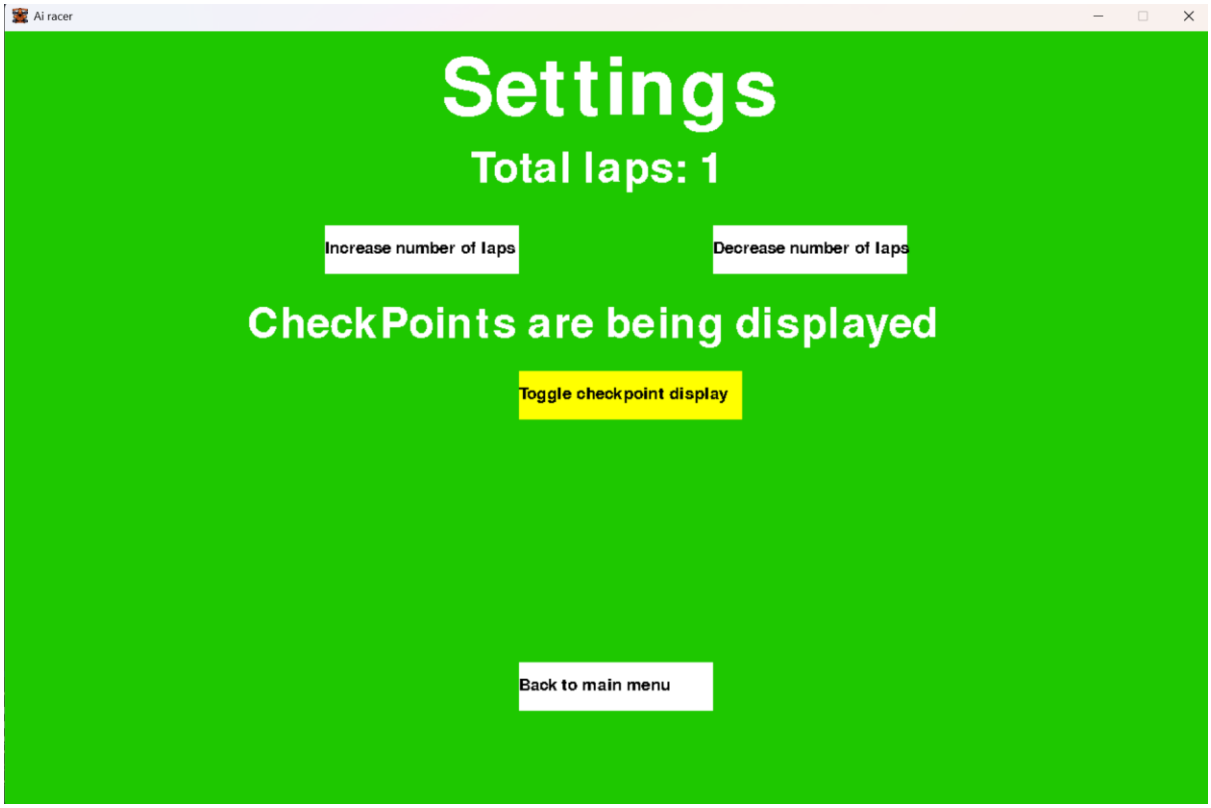
```
190             ToggleCheckpointDisplayButton.display_button()
```

```
179        if ToggleCheckpointDisplayButton.update_button(MousePos):
180            switch_DisplayCheckpoints()
```

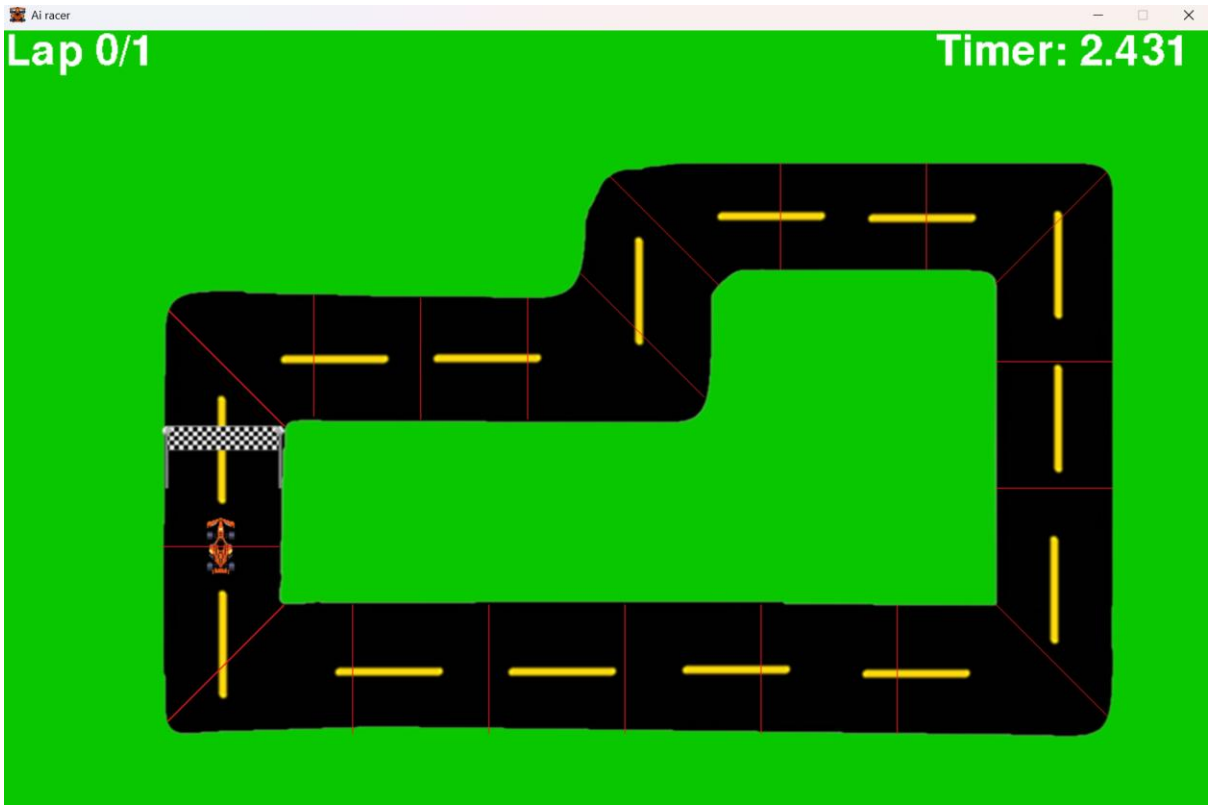
This is what the settings menu looks like with the new text and button when the checkpoints are not being displayed.



And this is what the settings menu looks like when the button has been pressed and the checkpoints are being displayed (The button is yellow because the mouse cursor was hovering over it):



Now that these features have been introduced, this is what the game loop looks like with the newly added timer and the DisplayCheckpoints variable being set to True using the settings menu:



After considering what features would be the most beneficial to the game, I noticed that, without an AI to play against, the only gameplay which is available is the car driving around the track by itself. To make the game more entertaining, I decided that introducing a 2-player mode would allow multiple players to compete against each other to get a better time, which would make the game much more enjoyable.

To begin creating this feature, I created the IsSinglePlayer global variable, which is very similar to the DisplayCheckpoints variable as it decides whether the game mode is single player or 2 players:

```
18    IsSinglePlayer = True #True for single player, False for 2 player
```

I then created the subroutines switch_GameMode and get_GameMode which switches the value of IsSinglePlayer or returns its value by reusing the code from DisplayCheckpoints.

```
56  def switch_GameMode():
57      global IsSinglePlayer
58      IsSinglePlayer = not IsSinglePlayer #Toggles the game modr from 1 players to 2 players.
59
60  def get_GameMode(): #getter for the IsSinglePlayer variable
61      global IsSinglePlayer
62      return IsSinglePlayer
```

Inside the settings menu, firstly I added text in medium font which displays “1 player is selected” if there is one player and “2 players are selected” if there are two players:

```
180         if get_GameMode():
181             GameModeText = Medium_font.render('1 player is selected',False,(255,255,255))
182             screen.blit(GameModeText,(440,450))
183         else:
184             DisplayCheckpointText = Medium_font.render('2 players are selected',False,(255,255,255))
185             screen.blit(DisplayCheckpointText,(440,450))
```

I then created the button on the settings menu called “ToggleGameModeButton” which switches if the game mode is single player or 2 players.

```
196         ToggleGameModeButtonText = body_font.render('Toggle 1/2 players', False, (0,0,0)) #Creates black text to be displayed on the button
197         ToggleGameModeButton = Button(200,50,540,550,ToggleGameModeButtonText,0,15)
```

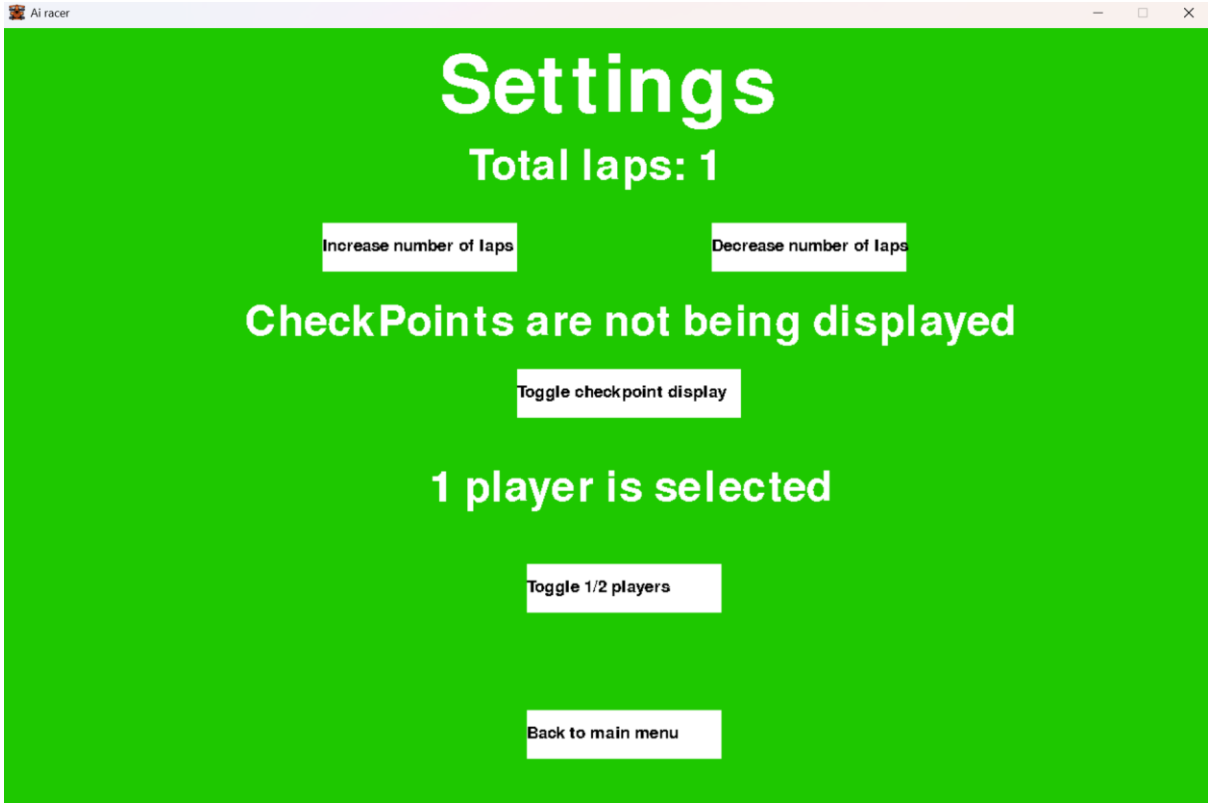
I added the if statement which will check if the ToggleGameModeButton is pressed then the global switch_GameMode() procedure will be called to update how many players will be in the game:

```
212             if ToggleGameModeButton.update_button(MousePos):
213                 switch_GameMode()
```

Finally, I displayed the ToggleGameModeButton onto the screen using the display_button() method:

```
225             ToggleGameModeButton.display_button()
```

This is how the settings menu now looked with the additional text and button:



Now that the setting can be updated, I will implement the feature to add multiple players. This will require large amounts of the game loop to be modified as previously, many parts of the game loop directly involve Car1 and do processes with it, however, to add a second car, this must instead be done with both Car1 and Car2.

To begin, I identified all the code in the game loop which does processing with car 1 which instead should be done with both cars, and I put it into a function called “car_processing” with the parameter of TheCar. Now I will be able to reuse this functionality multiple times using the car as an argument:

```
642         StartTime = NewTime
643
644         #print(StartTime)
645
646         def car_processing(TheCar):
647             if not IsGoingUp or not IsGoingDown: #If both IsGoingUp and IsGoingDown are true, then the speed remains the same
648                 if IsGoingUp:
649                     Car1.set_speed((Car1.get_ResultantSpeed() + Acceleration + 0.00003 * Car1.get_ResultantSpeed()))
650                 elif IsGoingDown:
651                     Car1.set_speed((Car1.get_ResultantSpeed() - Acceleration - 0.00003 * Car1.get_ResultantSpeed()))
652
653             if not IsTurningLeft or not IsTurningRight:#If both IsTurningLeft and IsTurningRight are true, then the angle remains the same
654                 if IsTurningLeft:
655                     Car1.set_image(Car1.rotate_car(-RotationAmount *Car1.get_ResultantSpeed(), Car1.get_image()))
656                 elif IsTurningRight:
657                     Car1.set_image(Car1.rotate_car(RotationAmount *Car1.get_ResultantSpeed(), Car1.get_image()))
658
```

I used VsCode’s refactor tool to refactor the variable Car1 inside this function into the parameter TheCar so that it performs all the functionality on whichever car is passed in.

I added the attribute PlayerNo to the car class so that each car can store which car it is:

```
304         def __init__(self,XPos,YPos,Rotation,CarImage,PlayerNo):
305             pygame.sprite.Sprite.__init__(self)
306             self.XPos = XPos
307             self.YPos = YPos
308             self.Rotation = Rotation
309             self.XSpeed = 0 #Sets speeds to 0 as the car is initially not moving
310             self.YSpeed = 0
311             self.ResultantSpeed = 0
312             self.CarImage = pygame.Surface.convert_alpha(CarImage) #Makes it so pixels can be transparent
313             self.DisplayCarImage = self.CarImage
314             self.rect = self.DisplayCarImage.get_rect()
315             self.rect.topleft = (self.XPos,self.YPos)
316             self.mask = pygame.mask.from_surface(self.DisplayCarImage)
317             self.LapCount = 0
318             self.LastCheckpoint = 0
319             self.PlayerNo = PlayerNo #Number of the player, 1 for player 1, 2 for player 2 etc
```

I then created the method get_PlayerNo which will return the Car's player number:

```
432         def get_PlayerNo(self):
433             return self.PlayerNo
```

After that, I instantiated both Car1 and Car2 onto the track in the beginning of the game loop:

```
563     #Instantiating the car and racetrack objects
564     Car1 = Car(210,500,0,CarImage,1)
565     Car2 = Car(180,500,0,CarImage,2)
```

When I first created the game loop, I made the variables IsGoingUp, IsGoingDown, IsTurningLeft, IsTurningRight and Friction not encapsulated inside of the car class as they were local variables inside of the game loop class. However, now that there are multiple cars, there must be multiple values of each of these variables for each car as these values will vary depending on the specific car:

```
606
607     #Definitions of global variables used in the game
608
609     running = True
610     IsGoingUp = False
611     IsGoingDown = False
612     IsTurningLeft = False
613     IsTurningRight = False
614     Friction = 0.0095
615     Acceleration = 0.055
616     RotationAmount = 0.45
617     StartTime = time.perf_counter()
618     FrameRate = 0.0165000000
619     '''Variables only used to test the mean and standard deviation of time between frames
620     count = 0
621     total = 0
622     sigmaXSquared = 0
623     '''
624     NormalFriction = 0.0095
625     OffTrackFriction = 0.04
626     TimerStart = time.perf_counter()
```

To fix this, I turned all these variables into attributes inside of the Car class so that there are unique values of them for each car:

```
342             self.PlayerNo = PlayerNo #Number of the player, 1 for player 1, 2 for player 2 etc
343             self.IsGoingUp = False #Boolean to store if the car is currently moving up
344             self.IsGoingDown = False #Boolean to store if the car is currently moving down
345             self.IsTurningLeft = False #Boolean to store if the car is currently turning left
346             self.IsTurningRight = False #Boolean to store if the car is currently turning right
347             self.Friction = 0.0095 #Value of friction being applied on this specific car
```

I then created getters and setters for each of these variables so that they can be accessed and modified by the game loop:

```
366         def get_IsGoingUp(self): #Getter for if the car is going up
367             return self.IsGoingUp
368 
369         def get_IsGoingDown(self): #Getter for if the car is going down
370             return self.IsGoingDown
371 
372         def get_IsTurningLeft(self): #Getter for if the car is turning left
373             return self.IsTurningLeft
374 
375         def get_IsTurningRight(self): #Getter for if the car is turning right
376             return self.IsTurningRight
377 
```

```
383         def set_IsGoingUp(self,NewValue): #Setter for if the car is going up
384             self.IsGoingUp = NewValue
385 
386         def set_IsGoingDown(self,NewValue): #Setter for if the car is going down
387             self.IsGoingDown = NewValue
388 
389         def set_IsTurningLeft(self,NewValue): #Setter for if the car is turning left
390             self.IsTurningLeft = NewValue
391 
392         def set_IsTurningRight(self,NewValue): #Setter for if the car is turning right
393             self.IsTurningRight = NewValue
```

```
378         def get_Friction(self): #Getter for the friction of the car
379             return self.Friction
380 
381         def set_Friction(self,NewValue): #Setter for the friction of the car
382             self.Friction = NewValue
```

I then rewrote the game loop so that it uses TheCar's getters and setters to access its attributes instead of using the local variables (The next few screenshots are areas of the game loop's code where TheCar's getters and setters are used instead of local variables):

```
686         if not TheCar.get_IsGoingUp() or not TheCar.get_IsGoingDown(): #If both IsGoingUp and IsGoingDown are true, then the speed remains the same
687             if TheCar.get_IsGoingUp():
688                 TheCar.set_speed((TheCar.get_ResultantSpeed() + Acceleration + 0.00003 * TheCar.get_ResultantSpeed()))
689             elif TheCar.get_IsGoingDown():
690                 TheCar.set_speed((TheCar.get_ResultantSpeed() - Acceleration - 0.00003 * TheCar.get_ResultantSpeed()))
691 
692         if not TheCar.get_IsTurningLeft() or not TheCar.get_IsTurningRight():#If both IsTurningLeft and IsTurningRight are true, then the angle remains the same
693             if TheCar.get_IsTurningLeft():
694                 TheCar.set_image(TheCar.rotate_car(-RotationAmount *TheCar.get_ResultantSpeed(), TheCar.get_image()))
695             elif TheCar.get_IsTurningRight():
696                 TheCar.set_image(TheCar.rotate_car(RotationAmount *TheCar.get_ResultantSpeed(), TheCar.get_image()))
```

```
for event in pygame.event.get(): #event handling
    if event.type == pygame.QUIT:
        running = False
        pygame.quit()
    if event.type == pygame.KEYDOWN: # This means any key has been PRESSED
        if event.key == pygame.K_a: #This means it was the a key
            TheCar.set_IsTurningLeft(True)

        if event.key == pygame.K_d: #This means it was the d key
            TheCar.set_IsTurningRight(True)

        if event.key == pygame.K_w: #This means it was the w key
            TheCar.set_IsGoingUp(True)

        if event.key == pygame.K_s: #This means it was the s key
            TheCar.set_IsGoingDown(True)



    if event.type == pygame.KEYUP: # This means any key has been LET GO OF
        if event.key == pygame.K_a: #This means it was the a key
            TheCar.set_IsTurningLeft(False)

        if event.key == pygame.K_d: #This means it was the d key
            TheCar.set_IsTurningRight(False)
        if event.key == pygame.K_w: #This means it was the w key
            TheCar.set_IsGoingUp(False)
        if event.key == pygame.K_s: #This means it was the s key
            TheCar.set_IsGoingDown(False)
```

```
741         #Adding frictional forces to the car's speed
742         if TheCar.get_ResultantSpeed() != 0:
743             TheCar.set_speed(TheCar.get_ResultantSpeed() - TheCar.get_Friction() *TheCar.get_ResultantSpeed())
```

```
751         #Rectangle collision detection between the track and the car
752         if pygame.sprite.spritecollide(Track1, CarGroup, False):
753             #Mask collision detection between the track and the car
754             if pygame.sprite.spritecollide(Track1, CarGroup,False, pygame.sprite.collide_mask):
755                 TheCar.set_Friction(NormalFriction)
756             else:
757                 TheCar.set_Friction(OffTrackFriction)
758
759         else:
760             TheCar.set_Friction(OffTrackFriction)
```

Outside of the car_processing() function, the function is called once or twice, once for Car1 and if 2 player mode is being used, then once for Car2. If either of these functions return ‘O’ then that car has completed all of its laps and that car has won:

```
785         if car_processing(Car1,Car1Group) == 'O':
786             return 'O'
787 
788         if not get_GameMode():
789             if car_processing(Car2,Car1Group) == 'O':
790                 return 'O'
```

Next, I updated the instantiation of car 1 and 2 so that Car2 is only instantiated if 2 player mode is being used:

```
603     #Instantiating the car and racetrack objects
604     Car1 = Car(210,500,0,CarImage,1)
605     if not get_GameMode():
606         Car2 = Car(180,500,0,CarImage,2)
```

I ran the code and this worked, however the cars were displayed underneath the track so I made sure to display the track before the cars so that the cars are displayed at the front of the screen:

```
794     #Displaying objects onto screen
795     Track1.display_track()
796     Car1.display_car()
797     if not get_GameMode():
798         Car2.display_car()
799 
800 
801     Finishline1.display_FinishLine()
```

This fixed the issue and both cars were able to be displayed and move. However, both cars currently move at the same time with the same inputs as the input detection only checks if w,a,s,d is being pressed and moves both cars due to it.

To fix this, I will set up different inputs for each player:

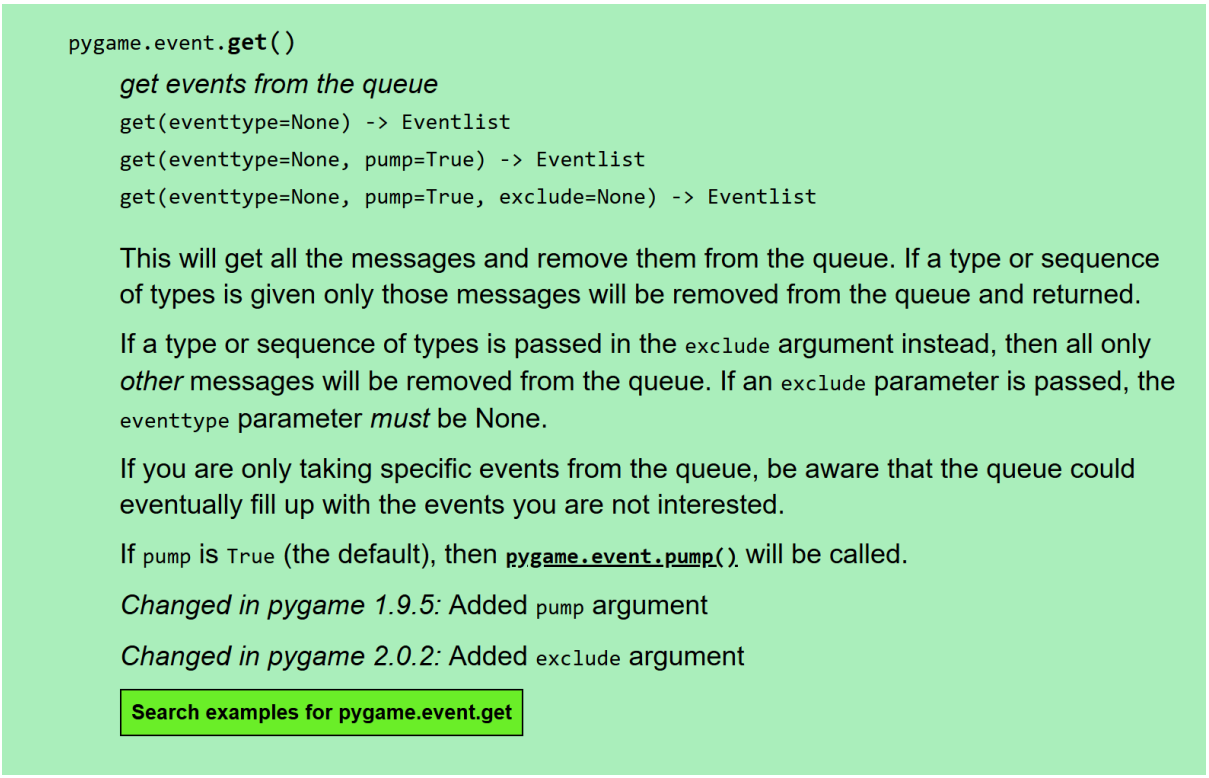
Here, if the current player's car number is 1, then it checks the w,a,s,d inputs to decide how Car1 should move and if the current player's car number is 2, then it checks the arrow key inputs to decide how Car2 should move:

```
711             for event in pygame.event.get(): #event handling
712                 if event.type == pygame.QUIT:
713                     running = False
714                     pygame.quit()
715                 if event.type == pygame.KEYDOWN: # This means any key has been PRESSED
716                     if TheCar.get_PlayerNo() == 1:
717                         if event.key == pygame.K_a: #This means it was the a key
718                             TheCar.set_IsTurningLeft(True)
719 
720                         if event.key == pygame.K_d: #This means it was the d key
721                             TheCar.set_IsTurningRight(True)
722 
723                         if event.key == pygame.K_w: #This means it was the w key
724                             TheCar.set_IsGoingUp(True)
725 
726                         if event.key == pygame.K_s: #This means it was the s key
727                             TheCar.set_IsGoingDown(True)
728                     elif TheCar.get_PlayerNo() == 2:
729                         if event.key == pygame.K_LEFT: #This means it was the a key
730                             TheCar.set_IsTurningLeft(True)
731 
732                         if event.key == pygame.K_RIGHT: #This means it was the d key
733                             TheCar.set_IsTurningRight(True)
734 
735                         if event.key == pygame.K_UP: #This means it was the w key
736                             TheCar.set_IsGoingUp(True)
737 
738                         if event.key == pygame.K_DOWN: #This means it was the s key
739                             TheCar.set_IsGoingDown(True)
```

```
744                 if event.type == pygame.KEYUP: # This means any key has been LET GO OF
745                     if TheCar.get_PlayerNo() == 1:
746                         if event.key == pygame.K_a: #This means it was the a key
747                             TheCar.set_IsTurningLeft(False)
748 
749                         if event.key == pygame.K_d: #This means it was the d key
750                             TheCar.set_IsTurningRight(False)
751                         if event.key == pygame.K_w: #This means it was the w key
752                             TheCar.set_IsGoingUp(False)
753                         if event.key == pygame.K_s: #This means it was the s key
754                             TheCar.set_IsGoingDown(False)
755                     elif TheCar.get_PlayerNo() == 2:
756                         if event.key == pygame.K_LEFT: #This means it was the a key
757                             TheCar.set_IsTurningLeft(False)
758                         if event.key == pygame.K_RIGHT: #This means it was the d key
759                             TheCar.set_IsTurningRight(False)
760                         if event.key == pygame.K_UP: #This means it was the w key
761                             TheCar.set_IsGoingUp(False)
762                         if event.key == pygame.K_DOWN: #This means it was the s key
763                             TheCar.set_IsGoingDown(False)
764 
```

After implementing this, car 1 was able to move correctly, however car 2 did not move at all. After reading pygame.event — pygame v2.6.0 documentation I learnt that "This will get all the messages and remove them from the queue." This means that, when the car_processing() function is run with the first player, this will get all of the events which include all w,a,s,d presses and all the arrow key presses and it will remove those from the queue, meaning that when player 1 gets the events, this will also remove the arrow key presses from the event list and when player 2 attempts to process the inputs, the

event list will be empty.



pygame.event.**get()**

get events from the queue

`get(eventtype=None) -> Eventlist`

`get(eventtype=None, pump=True) -> Eventlist`

`get(eventtype=None, pump=True, exclude=None) -> Eventlist`

This will get all the messages and remove them from the queue. If a type or sequence of types is given only those messages will be removed from the queue and returned.

If a type or sequence of types is passed in the `exclude` argument instead, then all only *other* messages will be removed from the queue. If an `exclude` parameter is passed, the `eventtype` parameter *must* be None.

If you are only taking specific events from the queue, be aware that the queue could eventually fill up with the events you are not interested.

If `pump` is `True` (the default), then **`pygame.event.pump()`** will be called.

Changed in pygame 1.9.5: Added `pump` argument

Changed in pygame 2.0.2: Added `exclude` argument

Search examples for pygame.event.get

(image from pygame.event — pygame v2.6.0 documentation)

To fix this, I removed the if statement which processed the different inputs depending on which car was currently being processed and simply processed both the w,a,s,d inputs and the arrow key inputs no matter which car is being processed as this will remove the inputs from the queue:

```
710
711             for event in pygame.event.get(): #event handling
712                 if event.type == pygame.QUIT:
713                     running = False
714                     pygame.quit()
715                 if event.type == pygame.KEYDOWN: # This means any key has been PRESSED
716                     if event.key == pygame.K_a: #This means it was the a key
717                         Car1.set_IsTurningLeft(True)
718 
719                     if event.key == pygame.K_d: #This means it was the d key
720                         Car1.set_IsTurningRight(True)
721 
722                     if event.key == pygame.K_w: #This means it was the w key
723                         Car1.set_IsGoingUp(True)
724 
725                     if event.key == pygame.K_s: #This means it was the s key
726                         Car1.set_IsGoingDown(True)
727 
728                     if event.key == pygame.K_LEFT: #This means it was the a key
729                         Car2.set_IsTurningLeft(True)
730 
731                     if event.key == pygame.K_RIGHT: #This means it was the d key
732                         Car2.set_IsTurningRight(True)
733 
734                     if event.key == pygame.K_UP: #This means it was the w key
735                         Car2.set_IsGoingUp(True)
736 
737                     if event.key == pygame.K_DOWN: #This means it was the s key
738                         Car2.set_IsGoingDown(True)
739 
740
```

```
742 
743                 if event.type == pygame.KEYUP: # This means any key has been LET GO OF
744                     if event.key == pygame.K_a: #This means it was the a key
745                         Car1.set_IsTurningLeft(False)
746 
747                     if event.key == pygame.K_d: #This means it was the d key
748                         Car1.set_IsTurningRight(False)
749                     if event.key == pygame.K_w: #This means it was the w key
750                         Car1.set_IsGoingUp(False)
751                     if event.key == pygame.K_s: #This means it was the s key
752                         Car1.set_IsGoingDown(False)
753                     if event.key == pygame.K_LEFT: #This means it was the left key
754                         Car2.set_IsTurningLeft(False)
755                     if event.key == pygame.K_RIGHT: #This means it was the right key
756                         Car2.set_IsTurningRight(False)
757                     if event.key == pygame.K_UP: #This means it was the up key
758                         Car2.set_IsGoingUp(False)
759                     if event.key == pygame.K_DOWN: #This means it was the down key
760                         Car2.set_IsGoingDown(False)
761 
```

This fixed the issue and now both car 1 and 2 can be controlled separately with w,a,s,d and the arrow keys depending on the car. Now, I will implement a feature which was outlined during the GUI design stage which was intended for the game mode when the player could play against the AI. On the results screen, the program will display which player won the game and the time that the fastest player took to beat it.

Firstly, I created an array called LastResult. This array will store a float which is the time in seconds that the winning player took to complete the race, and it will also store the player number of the player who won:

```
19  LastResult = [0.0,0] #Array holding the time taken for the last game and the player who won
```

I created the procedure set_results to set the LastResult array to new values of the time taken and the player number of the winner.

```
65  def set_results(TimeTaken, Winner):
66      global LastResult
67      LastResult = [TimeTaken,Winner]
```

I also created a function to return the LastResult array so that it can be displayed.

```
69  def get_results():
70      global LastResult
71      return LastResult
```

In the results screen, I created two texts in medium font which will display firstly which player won and the time that it took them using the get_results() function:

```
PlayerWon = Medium_font.render('Player' + get_results()[1] + ' won!', False, (255,255,255)) #Displays which player won the game
screen.blit(PlayerWon,(450,150))

TimeLaps = Medium_font.render('It took' + get_results()[0] + ' time to beat ' + get_totalLaps() + ' laps', False, (255,255,255)) #Displays the time taken and how many laps were raced
screen.blit(TimeLaps,(450,250))
```

Now, I needed to update the values of the LastResult array when a player won. I did this by using the set_results procedure whenever a player won the game with the correct arguments:

```
804
805         #Rectangle collision detection between the finish line and the car
806         if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
807             #Mask collision detection between the finish line and the car
808             if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
809                 result = checkpoint_reached(TheCar,TotalChecks + 1,TotalChecks)
810                 if result == 'O':
811                     set_results(NewTime-StartTime,TheCar.get_PlayerNo())
812                     return 'O'
813
814         for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
815             if pygame.sprite.collide_rect(TheCar,CheckArray[i]):
816                 result = checkpoint_reached(TheCar,i + 1,TotalChecks)
817                 if result == 'O':
818                     set_results(NewTime-StartTime,TheCar.get_PlayerNo())
819                     return 'O'
```

I attempted to run the program; however I was met with this error:

```
    PlayerWon = Medium_font.render('Player' + get_results()[1] + ' won!', False, (255,255,255)) #Displays which player won the game
                                   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
TypeError: can only concatenate str (not "int") to str
```

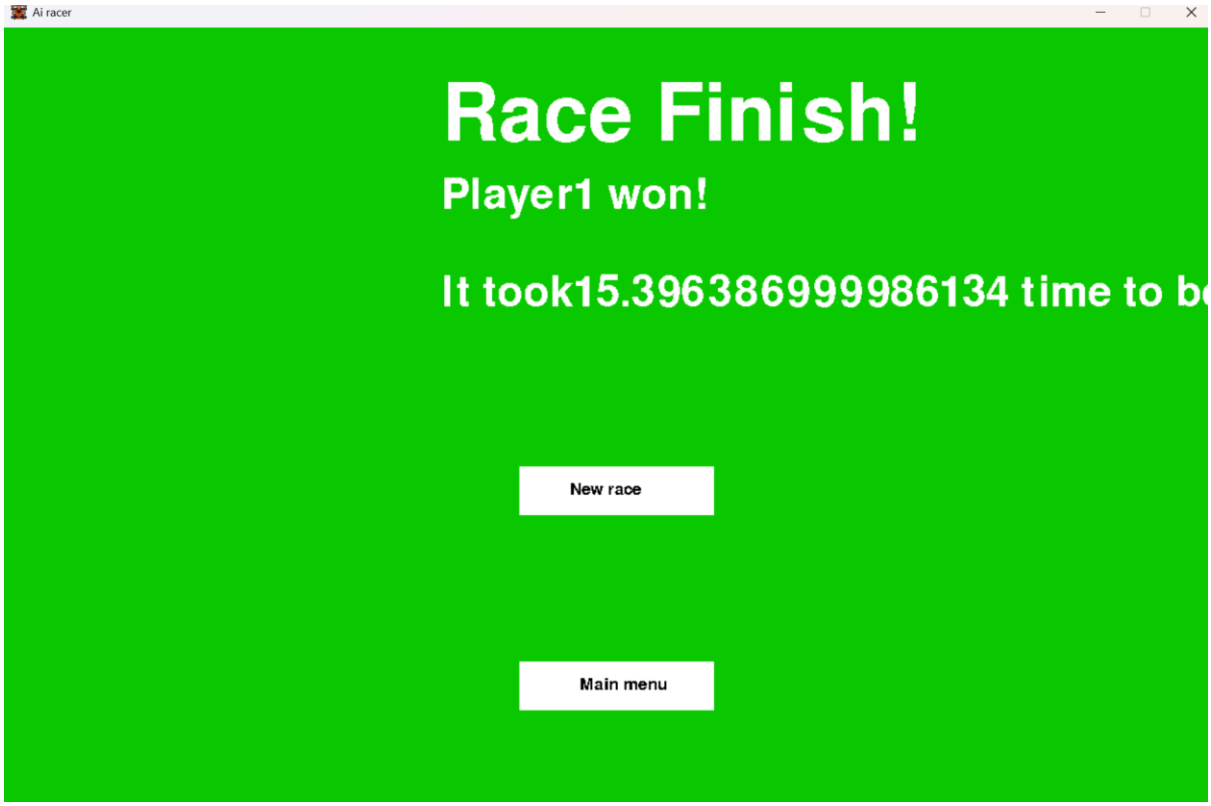
This was because I attempted to concatenate a string and an integer, so I used the str() function again to turn the player number and time taken into strings to be displayed:

```
running = True
while running:
    screen.fill((10,200,0))
    title = title_font.render('Race Finish!', False, (255,255,255)) #Displays the words 'Race finish' at the top of the screen
    screen.blit(title,(450,50))

    PlayerWon = Medium_font.render('Player' + str(get_results()[1]) + ' won!', False, (255,255,255)) #Displays which player won the game
    screen.blit(PlayerWon,(450,150))

    TimeLaps = Medium_font.render('It took' + str(get_results()[0]) + ' time to beat ' + str(get_totalLaps()) + ' laps', False, (255,255,255)) #Displays the time taken and how many laps were raced
    screen.blit(TimeLaps,(450,250))
```

This fixed the error and allowed the results screen to display. However, the time taken only displayed 0.0 seconds despite this not being the correct value.

```
804
805             #Rectangle collision detection between the finish line and the car
806             if pygame.sprite.spritecollide(Finishline1, CarGroup, False):
807                 #Mask collision detection between the finish line and the car
808                 if pygame.sprite.spritecollide(Finishline1, CarGroup,False, pygame.sprite.collide_mask):
809                     result = checkpoint_reached(TheCar,TotalChecks + 1,TotalChecks)
810                     if result == 'O':
811                         set_results(NewTime-TimerStart,TheCar.get_PlayerNo())
812                         return 'O'
813
814             for i in range(TotalChecks): #Checks if any of the checkpoints have collided with the car
815                 if pygame.sprite.collide_rect(TheCar,CheckArray[i]):
816                     result = checkpoint_reached(TheCar,i + 1,TotalChecks)
817                     if result == 'O':
818                         set_results(NewTime-TimerStart,TheCar.get_PlayerNo())
819                         return 'O'
```

This was because I used the StartTime variable instead of TimerStart as the StartTime variable holds the time when this current frame began, and the TimerStart variable holds the time when the game loop began. After fixing this, this is what the race finish screen looked like:

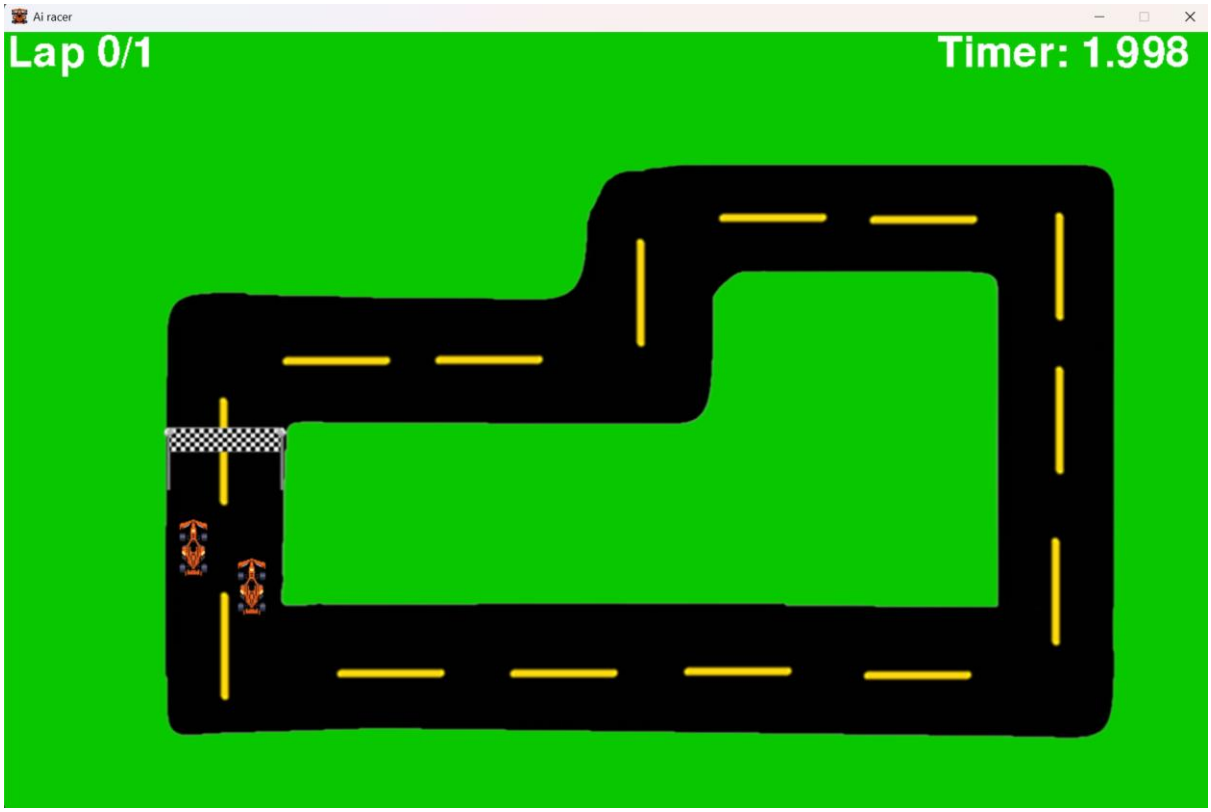


I had not rounded the time taken so the number of decimal places was very long which was not intended, so I rounded the time taken to 3 decimal places and I fixed the spacing with the word Player and 1.

After testing 2 player mode, I noticed that one of the players would begin on the track more to the right than the other player, however this will give them an unfair advantage as they are closer to the first turn. To fix this, I made it so the rightmost player started lower down than the leftmost player so that the starting positions were as fair as possible:

```
622     Car1 = Car(240,540,0,CarImage,1)
623     if not get_GameMode():
624         Car2 = Car(180,500,0,CarImage,2)
```

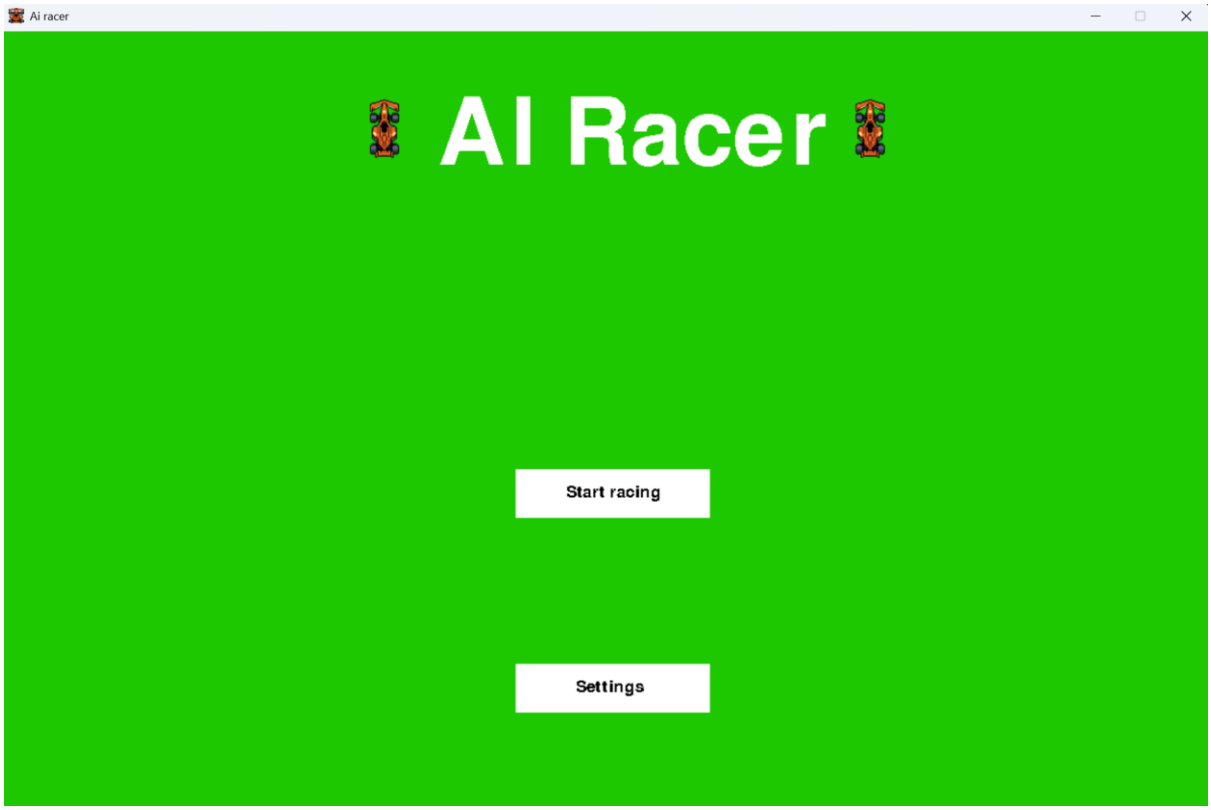
Here is an image of the newly balanced start positions:



I noticed that the main menu of the game looked rather bland, however it would be more engaging to the users if the title screen was more visually interesting and this is conventional for video games. To do this I loaded the Car image into the main menu subroutine, and I displayed the image of the car next to the title.

```
111     CarImage = pygame.image.load('racecar.png')
112     CarImage = pygame.Surface.convert_alpha(CarImage)
```

```
124         screen.blit(CarImage,(380,70))
125         screen.blit(CarImage,(880,70))
```



I am now happy with the state of the game and all the features in it which means that stage 3 of development has been completed.

Now that stage 3 has been completed, I will test its functionality using the test table below which I created in the design section. This will allow me to evaluate if the success criteria for this stage have been met. **SEE EVIDENCE OF ALL TESTS BELOW:**

Test number	Required inputs	Test description	Success criteria	Pass/Fail
1	Run the program.	The main menu screen should appear instead of the usual racetrack.	The main menu screen should appear instead of the usual racetrack.	Pass
2	Run the program and click the “start racing” button with the mouse.	This button should cause the racetrack to appear and the gameplay to begin.	The racetrack then appears, and the gameplay begins.	Pass
3	Run the program, press “start racing” on the main menu and drive the car around the track	When the car drives over the finish line for the	The race finish screen appears.	Pass

	so that it passes over the finish line 3 times.	third time, the race finish screen should appear.		
4	Run the program, press “start racing” on the main menu.	A timer should appear at the top of the screen which counts the time passed after the race began.	The timer appears and counts down.	Pass
5	Run the program, press “start racing” on the main menu.	A lap counter should appear at the top of the screen which counts how many laps have occurred.	A lap counter should appear at the top of the screen which begins at 1/3	Pass (However, I originally planned for the counter to begin from 1 but I changed it to begin from 0)
6	Run the program, press “start racing” on the main menu and race a single lap.	The lap counter should increment by one when the finish line is reached.	A lap counter should appear at the top of the screen which begins at 1/3 and reaches 2/3 when the finish line is first reached	Pass
7	Run the program, press “start racing” on the main menu and race two laps.	The lap counter should increment by one when the finish line is reached each time.	A lap counter should appear at the top of the screen which begins at 1/3 and reaches 2/3 when the finish line is first reached and 3/3 when it is reached a second time.	Pass
8	Run the program, press “start racing” on the main menu and attempt to drive the car in the opposite direction through the finish line.	The game should not allow the car to simply drive back through the finish line resulting in a completed lap so this should not affect the lap counter.	The lap counter is unaffected	Pass

Test 1:

See “Stage 3 testing.mp4” at 0 seconds. Immediately, as the program was loaded the main menu screen was displayed and not the racetrack, so this test is a pass.

Test 2:

See “Stage 3 testing.mp4” from 4 seconds to 5 seconds. When the “Start racing button” was pressed with the mouse, the main menu screen changed to the game loop. Therefore, this test is a pass.

Test 3:

See “Stage 3 testing.mp4” from 4 seconds to 60 seconds. When the Car completed 3 full laps and the lap counter reached 3, the race finish screen appeared, so this test is a pass. (note that in the video I changed the settings to make the total number of laps equal to 3 as the feature to modify the total number of laps was not planned for when I created this test table in the stages of development plan).

Test 4:

See “Stage 3 testing.mp4” from 4 seconds to 10 seconds. As you can see in the top right corner of the screen, there is a timer during the game loop which records how much time has been spent in the game loop. Therefore, this test is a pass.

Test 5:

See “Stage 3 testing.mp4” from 4 seconds to 60 seconds. As you can see in the top left corner of the screen, there is a lap counter which records how many laps have been completed by the car. The original plan was for the lap counter to begin at 1 as this would mean you are on the first lap; however, I changed this so that it shows you how many laps you have completed. This test is a pass as the success criteria has been met fully, only its implementation has been done slightly differently as initially planned.

Test 6:

See “Stage 3 testing.mp4” from 13 to 15 seconds. As you can see, when the car completes a lap fully, the lap counter increments by 1. This means that this test is a pass.

Test 7:

See “Stage 3 testing.mp4” from 4 to 60 seconds. As you can see, every time the car crosses the finish line after completing a full lap (not including the lap where the car went backwards) the lap counter incremented by 1. This means that this test is a pass.

Test 8:

See “Stage 3 testing.mp4” from 28 to 40 seconds. Here, the car does a full lap on the track going backwards across the track. When the car crosses the finish line, the lap counter does not increment by 1 which means that this test is a pass.

pygame.transform — pygame v2.6.0 documentation

pygame.sprite — pygame v2.6.0 documentation

How to Make Buttons in PyGame - The Python Code

pygame.mouse — pygame v2.6.0 documentation

String Comparison in Python - GeeksforGeeks

pygame.key — pygame v2.6.0 documentation

End testing

Now that the stages of development have been completed, I will begin the post-development testing of the program. This will be split into 3 categories. Functionality testing where I will test how the different features of the program work which have not yet been tested during the stages of development. Robustness testing where I will test how the program performs in situations which are different to the environment in which it was developed in or if the user does actions which are different than what is typically expected of them. Usability testing where I will have a group of stakeholders use my program and individually evaluate how accessible it is and simple to use.

Robustness:

Test number	Required inputs	Test description	Success criteria	Met/Partially met/Unmet	Evidence of test
1	Include the code which records the mean and standard deviation of the time between the frames. Then run the program and begin the game loop. Drive the car around the track during the game loop for 1 minute.	Run the program on a lower spec pc and record the mean and standard deviation of the time between frames.	If the program is robust, then the frame rate on a lower spec pc should be the same on a higher one as I have previously limited the minimum time between frames (see evidence)		
2	Run the program, navigate to the settings menu and press the “Decrease number of laps” button until the number of laps reaches 0 or further.	Attempt to select unintended values for settings in the settings menu	If the program is robust, it will prevent the user from being able to select a 0 or negative number of laps as they are not valid values		
3	Run the program, navigate to the settings menu and press the “Increase number of laps” button until the number of laps reaches 11 or further.	Attempt to select unintended values for settings in the settings menu	There is an intended limit of 10 total laps maximum for a certain game so that the games do not take too long. If the program is robust, it will prevent the user from being able to select an 11 or higher number of laps as they		

			are not valid values.		
4	Run the program, begin racing. Attempt to drive the car off all the edges of the screen.	Attempt to drive the car off the screen.	The car is prevented from crossing over the edge of the screen and is kept inside of the display window.		
5	Run the program, put the game into 2 player mode and begin racing. Attempt to drive the 2nd car off all the edges of the screen.	Attempt to drive the player 2 car off the screen.	The car is prevented from crossing over the edge of the screen and is kept inside of the display window.		
6					

Usability:

Now I will evaluate the usability of the program. I planned some questions to ask stakeholders in the post development test plan section of the design stage so I will mostly be using those, however I will not use the questions which were about the planned neural network as that is no longer a feature and I added an additional 2 questions about controlling the car as a user as that is now how the game is played.

I gave multiple stakeholders my program and allowed them to use it. Then I made them fill out a Microsoft form which allowed them to answer my targeted questions about how they found the project. I decided that someone was a stakeholder if they were between the ages of 14-18 and have an interest in computer science so I only gave the survey to people who fit that criteria. Additionally, I ensured that I had responses from multiple genders and age groups as well as people with varying interest in computer science from moderate to very strong to ensure that I had a broad set of results.

Here are the results from the survey:

1. How easy was your experience navigating the in game menus?

Every single person voted that it was incredibly easy to navigate the in-game menus. This suggests that the system of pointing at a button with the mouse cursor and clicking it to navigate through the menus was highly effective (see page) and improved the usability of the game.

2. To what extent did the available settings improve your experience of the game?

The respondents picked between 3 and 4 for this question. This suggests that the available settings did benefit the user experience and usability of the program, however there is still room for improvement and potentially more settings could improve the usability of the game further.

3. Are there any additional settings you would like to introduce to improve your experience of the game?

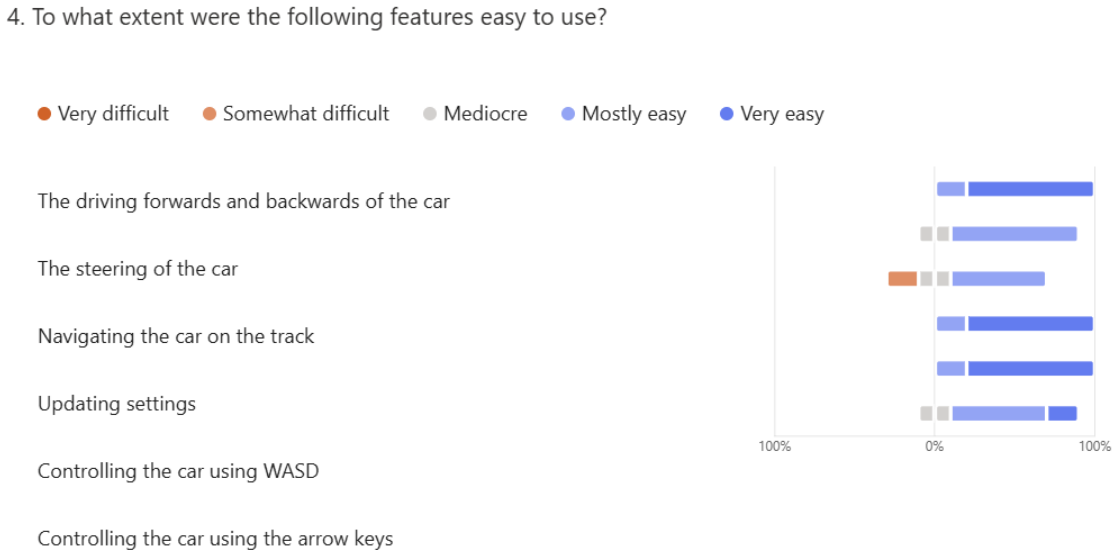
5 Responses

ID ↑	Name	Responses
1	anonymous	I did intuitively assume WASD driving, but this may not be the same for everyone. I would like a countdown too, I was suprised.
2	anonymous	Speed/Friction
3	anonymous	different speed settings would be good! it was also be fun if you had colour customisation
4	anonymous	Add speed pads / variation in game mechanics
5	anonymous	More game modes

For this question, three of the users suggested some form of setting which modifies the game's driving speed. I did spend extensive time during stage 1 attempting to make the driving of the car as easy as possible; however, it is very difficult to do this precisely when there are many different forces being used and the stakeholder feedback suggests that this was not done perfectly. This suggests that the useability of the game may have been impacted by the speed and friction used with the car, especially since stakeholders have previously highlighted that accurate physics are highly important in the analysis stage (page).

There are also requests for additional features/more game modes which understandably would make the game more enjoyable, however were not able to be implemented with the scope of this project. Finally, the first response says that they assumed that the controls would be w,a,s,d as these are conventional for many video games, however this may not be the same for all users and there is no explanation in game of which buttons do what which may impact the usability of the program.

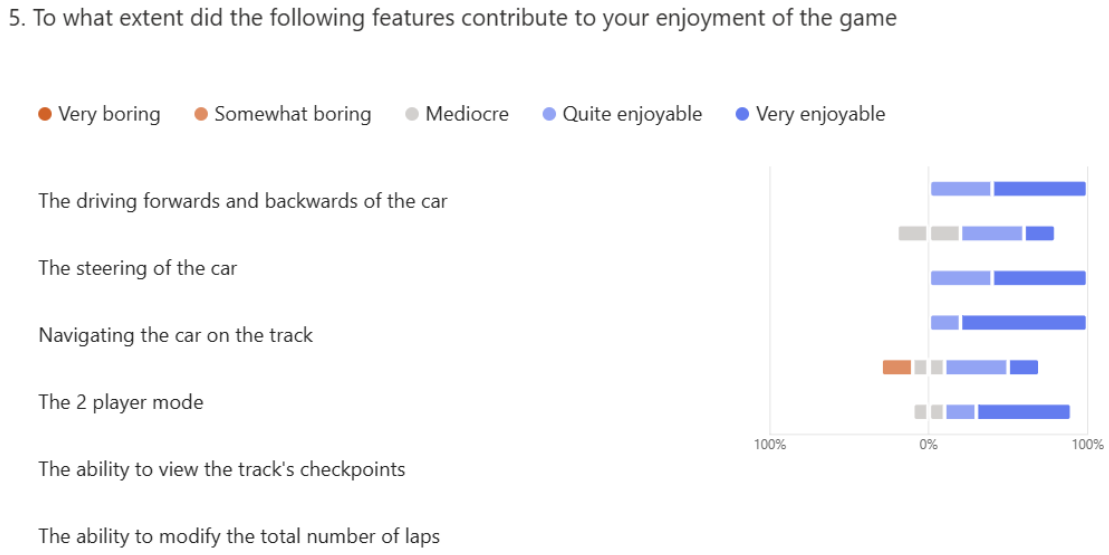
This next question was not planned in the post development test plan section of the design stage as it is about the car being controlled by the user, instead of by an Ai:



I created this question so that I could understand specifically which areas of the game had the best usability, and which areas had the worst usability.

The areas with the best usability were the driving forwards and backwards of the car, updating the settings and controlling the car using w,a,s,d. This is understandable as these features require very simple inputs and little coordination, apart from controlling the car with w,a,s,d which is likely controls that most users were used to as people interested in computer science.

The area with the worst usability was navigating the car on the track. This is also understandable as to perform this, you must drive the car, steer the car and ensure that it remains on the track. The answers to these questions follow what you would expect to see considering the difficulty to perform these different tasks. This suggests that as there are no features which are rated greatly easier/harder than you would expect and there are no individual features in this selection which have been poorly designed to the point that they effect usability.



Question 5 is structured very similarly to question 4, however it addresses how enjoyable certain features were instead of their ease of use.

The most enjoyable features rated were the navigation of the car on the track, the driving of the car and very strongly the 2-player mode. This is understandable as the 2 player makes the game competitive and is designed to be very fun and the navigation of the car on the track adds a high level of difficulty, however this may be enjoyable to a lot of users. However, the driving forwards and backwards of the car was rated very highly despite in a previous question many people asked for the option to modify the driving speed. This could possibly be because I intended this question to mean how enjoyable was my implementation of this feature, however, some stakeholders may have assumed that I was asking how enjoyable is it that the feature exists at all which of course is very important for the driving of the car itself.

The least enjoyable feature was the ability to view the track's checkpoints. This is understandable as I only intended for this feature to allow the players to learn the checkpoint locations however the feature does not modify the gameplay itself in any way.

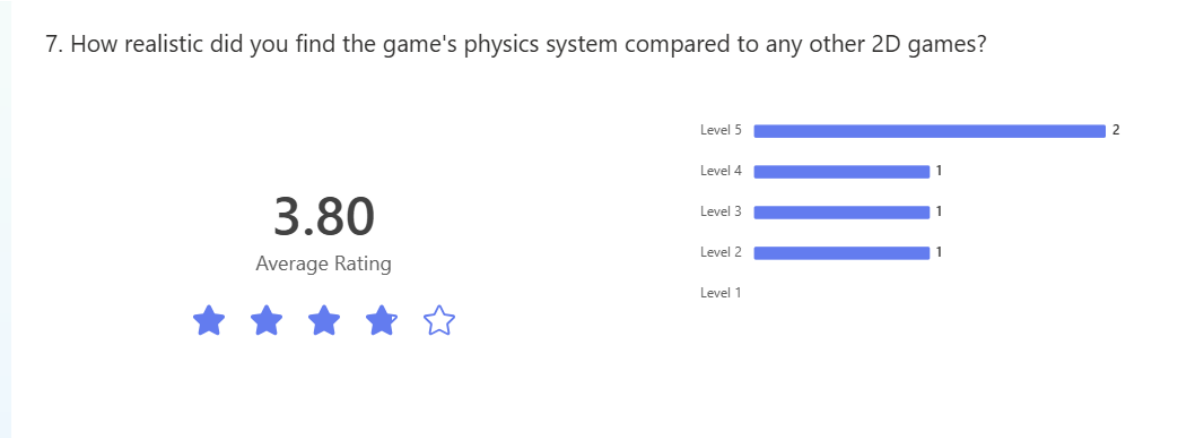
6. Do you have any additional comments about your experience with those features?

3 Responses

ID ↑	Name	Responses
1	anonymous	Checkpoint counter to know if I missed one and an individual lap timer please.
2	anonymous	the turning is very difficult if you are going slowly
3	anonymous	Make it so you can see if you have passed a checkpoint

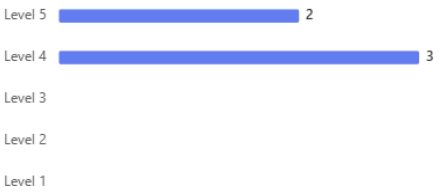
Question 6 asked for long answer responses about the areas assessed in the last 2 questions. Multiple people mentioned that they would like to know when they have missed a checkpoint. This is because, when you are driving on the track, even if you have set the checkpoints to be displayed, there is no indication when you drive by a checkpoint whether you have collided with it which can cause the player to continue the entire lap until they reach the finish line and then realise they missed a checkpoint. This is worth considering as the lack of this feature seems to have negatively affected the usability of the program.

Additionally, one person mentioned that turning is difficult when you are travelling slowly. This is because, I made it so that the speed at which the car turns is proportional to the resultant speed of the car (see page) so that the car appears more similar to cars in real life, however this has impacted the usability of the program slightly as it is now harder to drive the car when you are driving slowly.



The next question asked users how realistic they found the game's physics system. There is a large variety of responses here which shows that stakeholders are conflicted on the matter, however the average score of 3.8 stars is very high which suggests that on average, most users found the physics system accurate.

8. How enjoyable was the game's physics system?



Question 8 asks how enjoyable the physics system was. This question received only 4 and 5-star ratings. This suggests that stakeholders found the game's physics system highly enjoyable and the game's physics system improved the usability of the program. This contrasts with previous responses where many people suggested that they should be able to modify the speed of the car. This suggests that the only area of the physics system which users found decreased the usability of the program was the speed of the car.

9. Are there any improvements that you would want to be implemented for the game's physics system?

5 Responses

ID ↑	Name	Responses
1	anonymous	drifting.
2	anonymous	Drifting
3	anonymous	I think you need to be faster when on the grass as it takes too long to get back onto the track.
4	anonymous	Allow drifting
5	anonymous	drifting

Question 9 allowed users to submit any improvements they would like to add to the game's physics system. There was an overwhelming majority who wanted to add a drifting feature into the game. This is understandable as drifting is a very common feature in many other games of the same genre and it would add more complexity and nuance to the driving mechanics. Due to the scope of this project I did not have time to implement this feature, however if I were to update this project, drifting clearly would improve the engagement from the users.

10. How would you rate the game's graphics and GUI?

This question asked the stakeholders to rate the game's graphics and GUI. Overall, the game scored poorly at this question. As the focus for this project was to implement as much functionality as possible, I prioritized that over additional work on the GUI, however, it seems that the current GUI is worsening the usability of the program.

11. Do you have any concerns with the accessibility of the game?

5 Responses

ID ↑	Name	Responses
1	anonymous	N/A
2	anonymous	no
3	anonymous	no I do not
4	anonymous	Add controls in settings
5	anonymous	The ability to customise controls and change the speed/friction would be nice

The final question asked stakeholders if they had any concerns with the accessibility of the program. Most stakeholders said that they had no concerns, however some stakeholders pointed out that it would be beneficial if it was possible to modify the controls which are used in the program. This is understandable, as although w,a,s,d is conventional to use for many games, some users will prefer to use different buttons so allowing them to modify this would improve the usability of the program for them.

Evaluation:

Now that all tests have been completed, I will evaluate how well my final program meets my initial success criteria and usability requirements.

Here are the success criteria which was outlined in the analysis stage:

	Feature	Explanation	Justification	Importance
1.	Display window.	The car racing along the track will be visually shown in a main window which will also contain the user interface.	This must be displayed to the user so that they can see and understand the AI's decisions.	Essential
2.	A car which can be controlled.	There will be a simple car which can be controlled in a top down 2d environment with simple abilities such as accelerating/ decelerating and turning.	Having a car to be controlled by the Ai is a crucial part of the project as this is what the Ai will learn to use.	Essential
3.	A racetrack.	There will be a track for the car to race on with a defined start, finish and sides of the track.	Having a complex track layout with turns will develop a more sufficient Ai which can respond to changes on the track which both makes the program more engaging for users but also shows more of the capabilities of machine learning.	Essential
4.	Win detection	There will be a system which detects when the car has completed a lap, and when the car has completed enough laps to win the game.	It is essential that the Ai is detected when it completes laps on the track as this is the entire goal which the Ai is training for. Additionally, it is conventional in video games for the "player" to be able to win the game, so having	Essential

			the Ai being able to win after completing a certain number of laps will make the game more engaging.	
5.	A main menu	There will be a main menu screen which will allow the user to begin watching the Ai race on the track or open the settings menu to modify any settings.	This feature is essential to implement as it is necessary that the user is given the option between beginning the game or modifying the settings. Additionally, it is conventional for a video game to have a main menu screen before beginning the game, otherwise the Ai may immediately begin the race before the user is ready to watch it.	Essential
6.	A settings menu	There will be a settings menu which will allow the user to modify various aspects of the game such as accessibility features, enabling certain additional features and modifying some essential features.	This is an essential feature as the settings menu is required so that the user can access many of the important/Optional features such as the game mode to play against the Ai. This is also important so that the user can toggle some accessibility features such as a colour blindness mode and modify essential features which means that the settings menu itself is essential.	Essential
7.	A neural network	There will be a neural network which will take various information about the car in the input layer and output probabilities of the car making	The neural network is a fundamental part of this program and will be used to control the car and show the user an example of machine learning being used in action.	Essential

		various actions such as accelerating.		
8.	More advanced physics	Including more advanced physics will include the ability for the car to drift on the race train and the effects of resistive forces on the car.	The implementation of more advanced physics gives is rated as highly important to my stakeholders, so it is a desirable feature for my project. The Ai more options of how to traverse the course as well as making the game a more realistic simulation of real life which will be more engaging for the user to view.	Important
9.	Ability to play against the Ai	This will allow for the user to control their own car and try to beat the neural network in a race.	The implementation of this feature is rated as the most important to my stakeholders, so it is a very important feature to implement to make the project more engaging. This gives the user interactivity with the AI which will cause the users to spend more time on the program and have a better experience.	Important
10.	Multiple racetracks	This will allow the AI to perform and race on different tracks with different layouts which means that the AI will need an understanding of the game mechanics in general and not just a specific track.	The implementation of this feature is rated as moderately important to my stakeholders so it is a feature which will add benefit to the engagement of my users. As this causes the AI to have to perform well even on different track layouts, it will make the AI more robust and stronger which will aid in the teaching of machine learning.	Optional

11.	Select Ais with different amounts of training	This will allow the user to watch Ais with a different amount of time having been spent on its training and compare how they perform on the track.	This feature will aid in the education of the users as the ability to compare the neural network after different amounts of training will demonstrate how the time spent training impacts the performance of the agent.	Optional

1) **Display window:**

This feature was completely met. A display window is correctly displayed onto the screen, and all the functionality of the game is displayed in this game window such as the GUI, the car, etc. On pg 54, you can see when the display menu was first displayed and in "Full gameplay.mp4" you can see that all of the gameplay takes place inside of the display window as required.

Improvements with further development:

This success criteria is very small and therefore it was able to be completely met with ease. There are no improvements that I would want to implement for this success criteria as it works fully and cannot be expanded upon.

2) **A car which can be controlled:**

This feature was completely met. In "Test recording 1" you can see the car accelerate, decelerate and turn in a top-down 2d environment as required by this success criteria.

Additionally, on pg 61-62 you can see that I implemented a feature to make it so the car cannot drive off the edge of the screen. However, on pg 62 I discuss how there is a graphical bug where the car may appear to slightly go off the screen or be stopped early by an invisible wall depending on the angle of the car on some of the edges of the display window. I could not fix this bug and although it is not an essential feature for this success criteria, if the user encountered this bug, it would make the game slightly less engaging for them.

On pg 91-95 I also improved this feature by creating a stable frame rate which meant that the car's movement appeared consistent on the screen and whatever the speed that the car was set to would appear the same even when more features were added to the program. This feature worked completely throughout all of the development and it caused the car to drive much more smoothly so the usability feature to add a frame rate into the game was a success.

Improvements with further development:

After surveying my stakeholders, many of them requested that an additional setting could be added to vary the speed of the car. This suggests that they were not happy with the current speed. So, if I were to have more time with this project, I would do further research into real life physics and mechanics to see if there are any physical laws which I have not included in my game which would improve, and I would develop the requested setting to allow users to modify the acceleration value. As the acceleration value is a float with many decimal places, I would not be able to implement a simple button in the settings menu to increase the acceleration by 1 as that would be very imprecise, so instead I would create a slider which allows you to change the value of the acceleration from a minimum to a maximum value and anywhere in between.

3) **A racetrack:**

This feature was completely met. On pg 98, you can see that there is a racetrack for the car to drive on with clearly defined sides and the finish line acts and the start and finish of the track.

In the development plan for Stage 2 on pg 23, I outlined that the car will have a greater friction off the track than on the track. On pg 95-97, I implement this feature and you can see in "Test recording 1.mp4" that the friction for the track is higher off the track than on it. This is highly beneficial for the game, as the higher friction off of the track is realistic which makes the game more immersive, and it punishes the player from driving off of the track which adds a fun challenge to try and go as fast as possible while remaining on the track at all times.

Improvements with further development:

The success criteria has been fully met, however, when I surveyed my stakeholders, a large number of them rated the GUI and graphics of the game lowly which I believe is mostly due to the plain green background which is present off of the track as well as the track being poorly drawn. If I had more time with this project to implement more features, I would replace the plain green background with a more detailed and engaging background image with various structures like tires and mud. This would improve the usability of the game and to improve it further, I would add collision detection with some of the background structures using collision detection code I have previously made in this project. Then, I would make it so various effects occur if you collide with those structures such as using a timer to make the car unable to move for a second after it touches the mud or the car's rotation attribute being set to a random value when the car collides with the tires so it appears to bounce off in a random direction. This would improve the game by both improving the plain background by making it interesting and by adding dangerous effects for bumping into obstacles.

4) **Win detection:**

This success criteria has been completely met. On pg 116-124, I created checkpoints across the track and then I created subroutines to detect when the car finishes a lap and then finally when the car wins the game.

When I created the 2 player game mode, I decided to make it so that a single player would win out of the two, and that their time taken to win the race and their player number was recorded to make that game mode highly competitive as shown on pages 160-163.

Improvements with further development:

5) **A main menu:**

This success criteria has been completely met. The requirements for the main menu are for there to be a button which opens the game loop, and a button which opens the settings menu. As you can see in "Stage 3 testing.mp4", from the main menu, you are able to open up both the game loop and the settings menu. These features were implemented on pages 127-142.

Improvements with further development:

6) **A settings menu:**

This success criteria has been partially met. I was successfully able to create the settings menu between pages 143 and 152. The settings I ended up implementing were: Updating the total number of laps (see page 145), toggling whether or not checkpoints are displayed (see page 150) and toggling the 2 player mode (see page 152). These settings improve the user experience of the game. The ability to modify the total number of laps allows the user to play the exact amount of laps that they want instead of being forced to do a certain amount. The display checkpoint toggle allows the user to see exactly where the checkpoints are so that they can play effectively but can turn off the setting when they get used to it and the 2 player mode is a highly engaging mode which was voted upon very highly by my stakeholders.

However, there are some additional settings which I decided not to implement considering the scope of the project. For example, in the success criteria and GUI design, I planned that there would be accessibility settings such as a colour blindness mode, however the colours that were already being used (green, orange, black) are very contrasting and a colour blindness mode would not make the game that much easier to understand to a colour blind person.

Additionally, as there is no longer a neural network, none of the settings which involve the neural network could be implemented.

Improvements with further development:

If I was given more time to develop the project, I would introduce more settings into the settings menu to develop it more. In the stakeholder usability survey, someone requested that the controls of the game be able to be changed in the settings menu. This feature would have a great impact on accessibility, as certain users with physical disabilities may have difficulty reaching either w,a,s,d or the arrow keys, however being able to “remap” the buttons would greatly improve the usability for them and the accessibility of the program. To implement this, I would change in the code where I directly refer to the arrow keys or w,a,s,d and I would instead replace each of the input keys with a variable. Then, each of the variables would be instantiated to w,a,s,d and the arrow keys, however in settings I would make it so you could select an input e.g. player 1, moving up which is initially W but then you could type in a new key e,g, L and then the button to move up would be replaced with L instead.

7) **A neural network:**

This success criteria has been not met. During development, I had spent much more time on stages 1,2 and 3 than I had initially planned for. This was because my initial plans for these stages had seemed simple, however during development I discovered that in order to achieve their success criteria, additional features needed to be created that I had not considered, such as the necessity to implement a set frame rate system so that the car's speed values could remain constant, the many factors which needed to be included with the movement of the car so that it was able to drive smoothly, and the fixes for certain issues which required extensive debugging to figure out.

This meant that there was not enough time remaining with this project to both implement and train a neural network manually once I had gotten to stage 3. Considering this, I made the decision to instead of leaving the game in an unfinished state and rushing to create a neural network which likely could not be completed in time, resulting in an unfinished game and neural network, I improved upon the game which had been created and I made additional features which are important to stakeholders, so that the game is in the most enjoyable and completed state possible.

I believe that this is a better approach than moving onto stage 4, as even if that stage is able to be completed, without the entirety of stage 5 the neural network will be unable to be trained and therefore it will simply make random decisions and drive across the track randomly which is not a useful feature to have, nor will it be enjoyable or informative to my users. Whereas, when I improved upon the stages which already were developed resulted in the game being a fully operational project which will be enjoyable for the users to engage with.

The neural network was planned to be a very large part of the project as a whole which meant that no longer including this feature shifted the focus of the project from being an entertaining and educational game where you can learn about neural networks into the focus being to create a highly enjoyable and competitive racing game.

Improvements with further development:

If I were to develop the project further, the first feature I would implement would undoubtedly be the neural network. To do this, I would firstly create a generalised neural network which does not interact with the gameplay at all at first, inside of it would be an input layer which would be used to take in inputs from the game such as the car's

velocity, distance from the edges of the track, etc, multiple hidden layers which use weights to transform the inputs from the game into values between 0 and 1 and then an output layer which would output the probability between 0 and 1 for the car to do any of its possible inputs e.g. driving forward or turning right etc. After creating a general neural network, I would then link the input layer to actual inputs from the game and the output layer to actual outputs to the car. Firstly all of the weights would be randomised but by using reinforcement learning, the weights would slowly converge towards values which maximise the probability of the car heading towards the finish line.

8) **More advanced physics:**

This success criteria has been not met. This feature was highly requested by stakeholders and it would be highly enjoyable in the game, however it does not improve any other feature of the game and it is non-essential so I chose to prioritize more essential features.

Improvements with further development:

9) **Ability to play against the AI:**

This success criteria has been partially met. As there is no Ai, there is no mode to allow the player to play against the Ai. However, I have created a 2 player mode which allows two players to race along side each other and as the implementation for that feature is very similar to what it would have been if there was an Ai, I decided to adapt this additional feature into an enjoyable game mode which has greatly improved the enjoyment of the game.

Improvements with further development:

If the neural network was created, this would be very easy to adapt to allow for the player to play against the AI instead. This is because much of the code is now generalised so that it will perform all of the necessary functions on any passed in car object which means that if a car object was being controlled by the neural network, it would still easily be able to interact with the rest of the game and compete against the player.

10) **Multiple racetracks:**

This success criteria has been not met. This feature does not improve any other feature of the game and it is non-essential so I chose to prioritize more essential features.

Improvements with further development:

This feature would be difficult to implement currently as the racetrack and all 19 checkpoints are currently hard-coded into the software which is not very maintainable, so, I would have to implement a feature to more easily switch tracks out.

1) **Select AI's with different amounts of training:**

This success criteria has been not met. This feature does not improve any other feature of the game and it is non-essential so I chose to prioritize more essential features.